PSTAT 131/231 Midterm Sheet

Fall 2026 • Midterm Wed Oct 28, 5:00–6:15 pm, CHEM 1179

Coverage (Canvas *Midterm Information Page*): Slide Set 1 (Intro to ML; bias–variance tradeoff; KNN) • Slide Set 2 (Linear models; feature engineering; EDA) • Slide Set 3 (Logistic regression; LDA/QDA; model evaluation, *up until Slide 42*) • Reading: *ROC Curves Deep Dive* • HW 1–3 • Labs 1–3. Format: ≤12 multiple choice + 4 short answer; open book/notes, paper only. Also included: practice exam + solutions, Sai's graded midterm from last quarter, and Slide Set 3 slides 43–93 (LDA/QDA; past the Slide-42 cutoff but tested by practice Q20, Lab 3 and HW 3).

Index: last pages. **Bold blue** locations = distilled notes, plain = raw transcription; 12 : C4 = page 12, grid column C (of A–H, left→right), row 4 (of 1–11, top→bottom).

Contents

PART I DISTILLED NOTES 1

1 ML foundations, bias–variance, KNN 1
1.1 What ML is; the basic model . . . 1
1.2 Training vs. test error; overfitting . . . 2
1.3 Bias–variance decomposition . . . 3
1.4 Classification: error rate and the Bayes classifier . . . 3
1.5 K-nearest neighbors (KNN) . . . 4

2 Linear models, feature engineering, EDA 5
2.1 Exploratory data analysis (EDA) . . . 5
2.2 Missing data & imputation . . . 5
2.3 Data splitting . . . 6
2.4 Linear regression: model and least squares . . . 6
2.5 Categorical predictors: dummy variables . . . 6
2.6 Interactions and nonlinear terms . . . 7
2.7 Regression diagnostics . . . 7
2.8 Feature engineering with recipes . . . 8
2.9 Linear regression vs. KNN regression . . . 8

3 Classification models: logistic regression, LDA, QDA, naive Bayes 8
3.1 Classification set-up; why not linear regression? . . . 8
3.2 The logistic regression model . . . 9
3.3 Estimating the coefficients: maximum likelihood . . . 9
3.4 Multiple logistic regression and confounding . . . 9
3.5 Predicted probabilities and the threshold . . . 9
3.6 Discriminant analysis: Bayes' theorem and the idea . . . 10
3.7 LDA with one predictor ($p = 1$) . . . 10
3.8 LDA with several predictors ($p > 1$) . . . 11
3.9 Quadratic discriminant analysis (QDA) . . . 11
3.10 Comparing classifiers . . . 11
3.11 Naive Bayes (Lab 3) . . . 11
3.12 HW3: four classifiers on the Titanic data . . . 12

4 Model evaluation: metrics, thresholds, ROC/AUC 12
4.1 Train vs. test: where performance is measured . . . 12
4.2 Regression metrics . . . 12
4.3 From probabilities to classes: the threshold . . . 13
4.4 The confusion matrix and its rates . . . 13
4.5 Class imbalance: why accuracy can mislead . . . 13
4.6 ROC curve and AUC . . . 13
4.7 Reading "ROC Curves Deep Dive": Titanic Model A vs. B . . . 14
4.8 yardstick code and output . . . 14

5 R code reference: tidyverse + tidymodels 15
5.1 The pipeline: split → recipe → spec → workflow → fit → predict → metrics 15
5.2 dplyr / tidyverse verbs . . . 15
5.3 ggplot2 for EDA . . . 15
5.4 rsample: splitting . . . 16
5.5 recipes: steps, selectors, prep/bake . . . 16
5.6 parsnip: model specs, engines, modes . . . 16
5.7 fit, predict, augment, tidy . . . 17
5.8 yardstick: metrics . . . 17
5.9 Common bugs / exam traps . . . 17

6 Exam prep: past questions, patterns, quick answers 18
6.1 Practice exam: every question, one line . . . 18
6.2 Sai's graded midterm: every question, where points were lost . . . 18
6.3 Question patterns and how to answer them . . . 19
6.4 60-second facts . . . 20
6.5 HW answers at a glance . . . 20

PART II RAW TRANSCRIBED MATERIAL (only what Part I does not cover) 22

7 Slide Set 1 — Intro to ML, bias–variance tradeoff, KNN (pp. 1–58) 22
7.1 Course overview and introduction . . . 22
7.2 The learning set-up: response, predictors, f and error . . . 23
7.3 Parametric vs. non-parametric; uses of models . . . 23
7.4 Bias–variance tradeoff: flexibility, training MSE and test MSE . . . 23

8 Slide Set 1 (cont.) — pp. 59–116 23
8.1 Training MSE vs. test MSE; overfitting . . . 23
8.2 Bias-variance decomposition . . . 24
8.3 Classification setting: error rates and the Bayes classifier . . . 24

9 Slide Set 2 — Linear models, feature engineering, EDA 24
9.1 Exploratory data analysis (EDA) . . . 24
9.2 Linear regression . . . 24
9.3 Multiple linear regression . . . 25
9.4 Practical considerations: qualitative predictors . . . 25
9.5 Extensions of the linear model: interactions, polynomial regression . . . 25

10 Slide Set 3 — Logistic regression & model evaluation (pp. 1–42, the tested part) 25
10.1 Overview: classification . . . 25
10.2 Estimating the coefficients . . . 25
10.3 Confounding . . . 25
10.4 Classification performance: confusion matrix . . . 25
10.5 ROC curve and AUC . . . 26

11 Slide Set 3 (cont.) — Discriminant analysis: LDA & QDA (pp. 43–93; past the Slide-42 cutoff) 26
11.1 Motivation: why LDA? . . . 26
11.2 The geometric picture: the gene-expression example . . . 26

12 Reading — ROC Curves Deep Dive 26
12.1 Data . . . 26
12.2 Model A: ROC Curve . . . 27
12.3 Model B: ROC Curve: . . . 27
12.4 Conclusions . . . 27

13 Lab 1 — R basics, dplyr, EDA (diamonds), data splitting 27
13.1 Setup . . . 27
13.2 Basics of Data Processing . . . 28
13.3 Exploratory Data Analysis . . . 29
13.4 Data Splitting . . . 29

14 Lab 2 — Model fitting with tidymodels: linear regression, KNN, missing data 30
14.1 Introduction: Model Fitting . . . 30
14.2 Linear Regression . . . 30
14.3 *k*-Nearest Neighbors . . . 30
14.4 Missing Data . . . 31
14.5 Resources . . . 32

15 Lab 3 — Intro to binary classification (logistic, LDA, QDA, naive Bayes) 32
15.1 Logistic Regression . . . 32
15.2 LDA . . . 33
15.3 QDA . . . 33
15.4 Naive Bayes . . . 33
15.5 Comparing Model Performance . . . 33
15.6 Fitting to Testing Data . . . 33
15.7 Resources . . . 33
15.8 Source . . . 33

16 Homework 1–3: prompts, my submissions, solutions 33
16.1 Homework 1 – ML main ideas; EDA on mpg . . . 33
16.2 Homework 2 – linear regression and KNN (abalone) . . . 34
16.3 Homework 3 – binary classification (Titanic) . . . 36

17 Practice exam (Fall 2026) and Sai's graded midterm (last quarter) 38
17.1 Practice exam with official solutions . . . 38
17.2 Sai's midterm (graded, 20/26) . . . 38

INDEX – page : grid cell 40

Index 40

PART I DISTILLED NOTES

1 ML foundations, bias–variance, KNN

1.1 What ML is; the basic model

Machine learning vs. statistical learning (S1.7–18)

machine learning = applying learning algorithms and statistical methods to real-world data, i.e. how a machine can use data to *learn*; "all about making predictions or classifications" (S1.10).

- **ML**: subfield of AI in CS; algorithmic perspective; focus on AI and *prediction accuracy*.
- **Statistical learning**: subfield of statistics; statistical perspective; focus on interpretability and inference (S1.14).
- Classical programming: rules + data → answers. ML: data + answers → *rules* (S1.17).
- Rough workflow (S1.25): **Goal** → **Data** → **Learn** → **Evaluate**, looping back ("a different method/model?").

The model everything is built on (S1.29–33)

$$Y = f(X_1, \ldots, X_p) + \varepsilon$$

- Y = **response** (outcome, output, target, dependent variable).
- $X_1, \ldots, X_p$ = **predictors** (inputs, features, covariates, independent variables); p = # predictors, n = # observations, $\mathbf{x}_i = (x_{i1}, \ldots, x_{ip})$.
- f = fixed, unknown (non-random) function; ε = random error ("noise"), mean zero.
- **Goal: learn** f from data; the estimate is $\hat{f}$, prediction $\hat{y} = \hat{f}(\mathbf{x})$.
- Ex. (S1.29): Y=wage, X_1=age, X_2=education, X_3=year.

Supervised vs. unsupervised (S1.35, 44–47; HW1 Q1)

supervised learning	unsupervised learning
Response Y *known* for training data; the observed Y is the "supervisor" (answer key)	No response: only $X_1, \ldots, X_p$; "you never see the answer key"
Learn f to predict/explain Y	Find structure/groups/patterns in X
Linear & logistic regression, KNN, decision trees, random forests, SVM, neural nets	PCA, k-means clustering, hierarchical clustering, neural nets
Evaluated against the true Y	No direct "right answer" to score against

[Figure: S1.47: same 2-D scatter twice; supervised = points coloured by known class, unsupervised = all black (response not known).]

Regression vs. classification; metrics (S1.42–43; HW1 Q2–3)

Decided by the type of the **response** Y (not the predictors):

- regression: Y **quantitative** / numeric (price, blood pressure, speed). Metrics: MSE, RMSE, MAE, R-squared.
- classification: Y **qualitative** / categorical (survived/died, spam/not spam). Metrics: accuracy, error rate, precision, area under ROC curve (AUC).

Supervised-learning goals (S1.41): **prediction** (accurately predict future response), **estimation** (how predictors affect response), **model selection** (find the "best" model), **inference** (assess quality of predictions/estimation).

Regression or classification?

- *Logistic regression* is a **classification** method (outputs class probabilities), despite the name.
- A 0/1 or 1/2/3 coded label is still categorical ⇒ classification (convert with `factor()`).
- KNN does **both**; in tidymodels you must say which with `set_mode()` (Lab 2).

Uses of models: descriptive / inferential / predictive (S1.50; HW1 Q4)

- **Descriptive**: choose a model to best *visually emphasize a trend* in data (e.g. a smooth line on a scatterplot).
- **Inferential**: "what features are significant?"; aim is to *test theories*; (possibly) causal claims; state relationship between outcome & predictor(s) (p-values, CIs, hypothesis tests).
- **Predictive**: "what combo of features fits best?"; aim is to predict Y with minimum **reducible error**; *not* focused on hypothesis tests.

HW1 Q6 (voter data). (i) "Given a voter's profile, how likely are they to vote for the candidate?" ⇒ **predictive** (we want an accurate probability for each voter; why does not matter). (ii) "How would support change if the voter had personal contact with the candidate?" ⇒ **inferential** (effect of one specific predictor on the outcome; a relationship/causal-type question).

Parametric vs. non-parametric (S1.49; HW1 Q5)

Mechanistic/ parametric model	Empirically-driven/ non-parametric model
Assume a form for f (e.g. $\beta_0 + \beta_1 X_1 + \cdots$), estimate the parameters	No/few assumptions about f; shape learned from data
Won't match the true unknown f ⇒ **higher bias**	Much more flexible by default ⇒ **low bias, higher variance**
Adding parameters = more flexibility; too many = overfitting	Requires a **larger #** of observations; can also overfit
Easier to interpret (few meaningful parameters)	Harder to interpret ("black box")
e.g. linear regression, logistic regression, LDA	e.g. KNN

Similar: both are predictive models that estimate f from data and both can overfit. **Which is easier to understand?** Mechanistic. **Bias–variance link:** mechanistic = rigid form → high bias / low variance; empirically-driven = flexible → low bias / high variance (needs more n to tame variance).

Flexibility vs. interpretability (S1.53–56)

Different methods give different $\hat{f}$, each with its own flexibility (complexity). Ranking, interpretability high → low / flexibility low → high: subset selection, lasso → least squares → GAMs, trees → bagging, boosting → SVMs (S1.55).

- Why ever pick a *less* flexible method? Better **interpretability** and, sometimes, **smaller test error**.
- **No single method is best for all data sets**; choosing one depends on *evaluating* its performance (S1.56).

"The most flexible model is always best" — wrong (PE Q3)

(1) If the goal is interpretation/inference, flexible models are harder to interpret. (2) A very flexible model fits patterns in the training data that do not exist in the population ⇒ overfitting: low training error, high test error.

1.2 Training vs. test error; overfitting

Training MSE vs. test MSE (S1.57–60)

Training set $\{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}$; test set $\{(\tilde{\mathbf{x}}_1, \tilde{y}_1), \ldots, (\tilde{\mathbf{x}}_m, \tilde{y}_m)\}$ *not seen or used to train* $\hat{f}$.

$$\text{Training MSE} = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \hat{f}(\mathbf{x}_i)\right)^2 \qquad \text{Test MSE} = \frac{1}{m}\sum_{i=1}^{m}\left(\tilde{y}_i - \hat{f}(\tilde{\mathbf{x}}_i)\right)^2$$

$\hat{f}$ is obtained on the **training** set. We want the $\hat{f}$ with the **smallest test MSE**. **Lowest training MSE ≠ lowest test MSE.**

How the errors move with flexibility (S1.61–66)

- Higher flexibility → matches training points better, more wiggly curve → **training MSE always decreases** (monotone, can reach 0).
- Higher flexibility ↛ better fit to the true f: **test MSE is U-shaped** — falls, hits a minimum, then rises.
- Test MSE can never fall below $\mathrm{Var}(\varepsilon)$ (dashed line in the plots); training MSE can.
- **overfitting** = small training MSE, large test MSE (the method chases noise). **underfitting** = too inflexible: both errors high (misses real structure).
- Speed-vs-yams toy (S1.36–40): a green squiggle interpolates the 5 training points (training error 0), but on the 4 red test points the plain straight line has the smaller total error $\sum_{i=6}^{9}|\tilde{y}_i - \hat{f}(\tilde{x}_i)|$.

Estimating test MSE (S1.68, 94): training MSE is easy; test data may be unavailable, or using it repeatedly turns it into "another training set". Fix: **cross-validation** — estimates test MSE using only training data.

Practice Q3 — which model? (PE Q3)

Model	Train acc.	Test acc.	Reading
A	71%	70%	too simple (both low)
B	86%	82%	**prefer: highest test acc.**
C	100%	68%	**overfitting** (huge gap)

Official solution. Prefer B (highest *testing* accuracy). C is overfitting; B shows a little overfitting too, but much less.

Lab 2 (diamonds, predict price) — RMSE (Lab 2)

Model	train	test	Reading
linear regression	1126	1146	no gap, but large error: high bias
KNN, $k = 5$ (default)	386	**711**	best test RMSE
KNN, $k = 2$	152	851	lower train, higher test: overfit

Smaller k = more flexible $\Rightarrow$ lower training error but a bigger train–test gap. The linear model imposes a form on f (even predicts negative prices); KNN learns f from the data.

Training-error traps (PE Q4)

- "A model with lower training error is always preferable" — **false**: training error is a poor estimate of test error; as training error $\to 0$, test error almost always increases.
- "A predictor with zero Pearson correlation with Y can't improve prediction" — **false**: zero correlation = no *linear* relation; nonlinear/empirically-driven models (e.g. KNN) can still use it.
- "Two highly correlated predictors necessarily interact" — **false**: an interaction term means the X_1–Y relationship changes with X_2; unrelated to correlation.

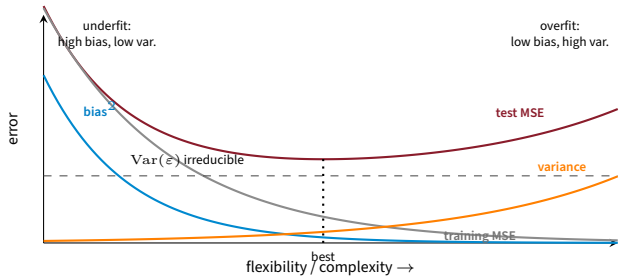
1.3 Bias–variance decomposition

Bias–variance decomposition (course form) (S1.70–79)

$$\underbrace{E\left[\left(\tilde{y}_0 - \hat{f}(\tilde{\mathbf{x}}_0)\right)^2\right]}_{\text{expected test MSE}} = \underbrace{\operatorname{Var}\left(\hat{f}(\tilde{\mathbf{x}}_0)\right)}_{\text{variance on test data}} + \underbrace{\left(\operatorname{Bias}\left(\hat{f}(\tilde{\mathbf{x}}_0)\right)\right)^2}_{\text{bias on test data}} + \underbrace{\operatorname{Var}(\varepsilon)}_{\text{irreducible error}}$$

- $\hat{f}$ learned from a training set; $(\tilde{\mathbf{x}}_0, \tilde{y}_0)$ = a test observation, *independent* of the training set, randomly sampled from $Y = f(X) + \varepsilon$ (f non-random, ε zero-mean noise).
- $\operatorname{Bias}(\hat{f}(\tilde{\mathbf{x}}_0)) = E[\hat{f}(\tilde{\mathbf{x}}_0)] - f(\tilde{\mathbf{x}}_0)$: how far the *average* fit is from the truth (error from wrong/too-simple assumptions).
- $\operatorname{Var}(\hat{f}(\mathbf{x}_0)) = E\left[(\hat{f}(\mathbf{x}_0) - E[\hat{f}(\mathbf{x}_0)])^2\right]$: how much $\hat{f}(\mathbf{x}_0)$ jumps around across different training sets.
- E averages over repeated training sets (each gives a different $\hat{f}$) and over ε.

The tradeoff picture (S1.80–84, 101)



Sketch: test MSE = bias^2 + variance + $\operatorname{Var}(\varepsilon)$; minimum (dotted) where the falling bias^2 and rising variance balance.

- **Simple model $\to$ high bias + low variance; flexible model $\to$ low bias + high variance** (S1.81). More complexity: variance ↑, bias ↓.
- Want *simultaneously* low variance and low (squared) bias — the challenge is finding such a method.
- Where the minimum sits depends on the true f (S1.82–84): true f moderately curved $\to$ intermediate flexibility best; f nearly linear $\to$ linear regression already near-optimal (bias≈0, extra flexibility only adds variance); f very nonlinear $\to$ bias falls steeply, flexible methods win.

Dartboards: bias = off-centre, variance = spread (S1.71–76, 80)

Bull's-eye $= \tilde{y}_0$; each dart $= \hat{f}(\tilde{\mathbf{x}}_0)$ from a different training set.

1. tight on centre	low bias	low var.
2. scattered around centre	low bias	**high var.**
3. tight but off-centre	**high bias**	low var.
4. scattered and off-centre	**high bias**	**high var.**

Reducible vs. irreducible error (S1.77, 85–87)

Variance + bias^2 = **reducible error** (depends on the method). $\operatorname{Var}(\varepsilon)$ = **irreducible error**: noise no model can remove (unmeasured variables, randomness). The "best" we can do: $\hat{f}(\mathbf{x}_0) = E[Y \mid X = \mathbf{x}_0]$ (the true regression function). Then variance $= 0$ and bias $= 0$, so expected test MSE is **minimized at** $\operatorname{Var}(\varepsilon)$. But $E[Y \mid X]$ is **unknown in practice** (the population joint distribution of X and Y is unknown) $\Rightarrow$ we must estimate it (e.g. KNN).

Bias–variance traps

- No method can get expected test MSE below $\operatorname{Var}(\varepsilon)$; training MSE *can* go below it (that is overfitting).
- Bias and variance describe the *method* over many possible training sets, not one fitted curve.
- The "bias" term enters **squared**; all three terms are ≥ 0.
- "Increasing flexibility lowers bias" is (generally) true; "…lowers test error" is **not** necessarily true.

1.4 Classification: error rate and the Bayes classifier

Training / test error rate (S1.88–93)

$$\text{Training error rate} = \frac{1}{n}\sum_{i=1}^{n} I(y_i \neq \hat{y}_i), \qquad I(y_i \neq \hat{y}_i) = \begin{cases} 1 & y_i \neq \hat{y}_i \\ 0 & y_i = \hat{y}_i \end{cases}$$

= fraction of *incorrect* classifications in the training set; $\hat{y}_i$ = class label $\hat{f}$ predicts. **Test error rate** $= \frac{1}{m}\sum_{i=1}^{m} I(\tilde{y}_i \neq \hat{\tilde{y}}_i)$ on the test set (slide writes $\frac{1}{n}$). accuracy $= 1 -$ error rate.

Worked example (S1.90)

y_i	dog	cat	bird	dog	dog
$\hat{y}_i$	cat	cat	dog	dog	dog
$I(y_i \neq \hat{y}_i)$	1	0	1	0	0

Training error rate $= 2/5 = 0.40$; training accuracy $= 1 - 2/5 = 0.60$. Same story as regression (S1.93): higher flexibility $\to$ lower training error rate, but $\not\to$ lower test error rate; **overfitting = large test error rate + small training error rate**.

Bayes classifier and Bayes error rate (S1.96–99)

Bayes classifier: assign each observation to the *most likely class given its predictor values*:

$$\hat{y}_0 = \operatorname{argmax}_j \operatorname{Prob}(Y = j \mid X = \mathbf{x}_0)$$

Ex.: $P(\text{dog} \mid \mathbf{x}_0) = 0.5$, $P(\text{cat} \mid \mathbf{x}_0) = 0.3$, $P(\text{bird} \mid \mathbf{x}_0) = 0.2 \Rightarrow \hat{y}_0 =$ dog. Two classes: $\hat{y}_0 = 1$ if $\operatorname{Prob}(Y = 1 \mid X = \mathbf{x}_0) > 0.5$, else 2. The Bayes decision boundary is where $\operatorname{Prob}(Y = 1 \mid X = \mathbf{x}_0) = 0.5$ (purple dashed curve on the slides).

$$\text{Bayes error rate} = 1 - E\left[\max_j \operatorname{Prob}(Y = j \mid X)\right]$$

= lowest possible test error rate; > 0 whenever classes overlap (even the Bayes classifier misclassifies points on the "wrong" side, red squares on S1.99). It is the classification analogue of $\operatorname{Var}(\varepsilon)$.

Regression vs. classification summary (S1.100)

	Regression	Classification
Y	quantitative	qualitative
error	training/test MSE	training/test error rate
best	$E[Y \mid X = \mathbf{x}_0]$	$\operatorname{argmax}_j \operatorname{Prob}(Y = j \mid X = \mathbf{x}_0)$
floor	$\operatorname{Var}(\varepsilon)$	$1 - E[\max_j \operatorname{Prob}(Y = j \mid X)]$

Bayes classifier

It is a theoretical gold standard: it needs the true conditional distribution of Y given X, which we never know. Real classifiers (KNN, logistic regression, LDA…) *estimate* $\operatorname{Prob}(Y = j \mid X)$ and try to approach it.

1.5 K-nearest neighbors (KNN)

KNN algorithm (S1.103–109)

We never know $E[Y \mid X]$ or $\operatorname{Prob}(Y = j \mid X)$ — **estimate** them locally: look at the K training points closest to $\mathbf{x}_0$.

$$\mathcal{N}_0 = \{i : \mathbf{x}_i \text{ is among the } K \text{ nearest neighbors of } \mathbf{x}_0\}$$

Regression (average): $\hat{f}_K(\mathbf{x}_0) = \dfrac{1}{K} \sum_{i \in \mathcal{N}_0} y_i$

Classification (majority vote): $\hat{y}_K = \operatorname{argmax}_j \left\{ \dfrac{1}{K} \sum_{i \in \mathcal{N}_0} I(y_i = j) \right\}$

(the class proportions among the neighbours estimate $\operatorname{Prob}(Y = j \mid X = \mathbf{x}_0)$). Steps: (1) compute the distance from $\mathbf{x}_0$ to every training point; (2) keep the K closest; (3) average their y / take a vote.

Distances (S1.104)

- Euclidean distance: $d(\mathbf{x}, \mathbf{y}) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$ (straight line; usual default).
- Manhattan distance: $d(\mathbf{x}, \mathbf{y}) = |x_1 - y_1| + |x_2 - y_2|$ (grid / city-block).
- Minkowski distance: $d(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{1/p}$, $p \geq 1$; $p = 1$ Manhattan, $p = 2$ Euclidean.

Worked KNN examples

3-NN regression (S1.107): house prices of the 3 nearest (by sq. ft) are 50, 55, 51 $\Rightarrow \hat{f} = (50 + 55 + 51)/3 = 52$.

Vote with $k = 1, 3, 5, 7$ (PE Q5). Neighbors ordered closest → farthest:

neighbor	1	2	3	4	5	6	7		
class		A	B	B	A	A	B	B	
$k = 1$:	A							(A 1–0)	⇒ **A**
$k = 3$:	A	B	B					(A1, B2)	⇒ **B**
$k = 5$:	A	B	B	A	A			(A3, B2)	⇒ **A**
$k = 7$:	A	B	B	A	A	B	B	(A3, B4)	⇒ **B**

Predictions can flip as k changes. Even k can tie (here $k = 2$: 1–1, $k = 4$: 2–2) — use odd k for two classes.

ISLR picture (S1.110): $K = 3$, the circle around × holds 2 blue + 1 orange ⇒ predict blue; repeating for every point of the plane draws the KNN decision boundary.

Choosing k: the KNN bias–variance tradeoff (S1.113–115)

For fixed $(\mathbf{x}_0, y_0)$, with $Y = f(X) + \varepsilon$:

$$E\left[(y_0 - \hat{f}(\mathbf{x}_0))^2\right] = \underbrace{\frac{\operatorname{Var}(\varepsilon)}{k}}_{\text{variance}} + \underbrace{\left[f(\mathbf{x}_0) - \frac{1}{k} \sum_{i \in \mathcal{N}_0} f(\mathbf{x}_i)\right]^2}_{\text{bias}^2} + \operatorname{Var}(\varepsilon)$$

	variance	bias^2	$\operatorname{Var}(\varepsilon)$
$k \uparrow$	decrease	increase	beyond our control
$k \downarrow$	increase	decrease	beyond our control

- **Small k = flexible**: jagged, island-filled decision boundary; $k = 1$ gives **training error 0** (each point is its own nearest neighbour) → low bias, high variance, overfits.
- **Large k = smooth / rigid**: boundary nearly linear at $K = 100$ → high bias, low variance, underfits. $k = n$: every prediction = overall mean / majority class.
- Flexibility grows with $1/K$ (S1.115): training error falls steadily to 0 at $1/K = 1$; test error is U-shaped (min near $K \approx 10$), never below the dashed Bayes error rate (≈ 0.13).
- k is a hyperparameter (tuning parameter): not estimated by fitting; choose it by test/cross-validation error, never by training error.

k direction traps (PE Q1; Sai Q1)

- KNN great on training, worse on new data (overfitting) ⇒ **increase** k. Decreasing k makes it worse; up-weighting large-variance variables makes it worse; adding an interaction term is pointless (KNN is non-parametric).
- **Increasing k makes KNN *less* flexible** (adding polynomial/interaction terms to a regression makes it *more* flexible).

Ties (S1.111–112)

Distance ties: several points equidistant at the k-th spot (which ones count?). The slide's table keeps all tied points: L → A B; M (3 equidistant) → A A B; N (1 obvious + 2 tied) → A A B B. *Vote ties* (A B, A A B B): common fixes are an odd k, choosing at random, or going with the nearest neighbour's class / distance-weighted votes.

Why standardize predictors (PE Q6)

KNN is **distance-based**: a predictor with a big spread dominates the distance. Ex.: age (years) vs. income ($): A=(25, 50,000), B=(65, 51,000), C=(26, 58,000). Unscaled, $d(A, B) = \sqrt{40^2 + 1000^2} \approx 1001$ but $d(A, C) = \sqrt{1^2 + 8000^2} \approx 8000$, so B (40 years older!) is A's "nearest" neighbour — age barely matters. Standardizing *only* age does not fix it: income must be standardized too. Fix: `step_normalize()` (center to mean 0, scale to SD 1) on **all** numeric predictors, so each contributes on the same scale.

Curse of dimensionality; KNN profile (S1.116)

KNN works well for **small** p (about $p \leq 4$) and **large** n. It **fails when p is very large**: there are very few (almost no) training points "near" $\mathbf{x}_0$, so the "nearest" neighbours are actually far away ⇒ **curse of dimensionality**. (Classic illustration: to capture 10% of uniform data in a p-dim cube, a neighbourhood needs side $0.1^{1/p}$: 0.10 for $p = 1$, 0.32 for $p = 2$, 0.79 for $p = 10$.) More sophisticated non-parametric models exist (e.g. kernel estimators).

- **Non-parametric**: no assumed form for f, no coefficients, no normality / equal-variance assumptions; can draw **nonlinear decision boundaries** because classification is decided *locally* by neighbours (Sai Q2).
- Costs: not interpretable (no coefficients to read); needs the training data at prediction time; sensitive to scale and to irrelevant/noisy predictors; categorical predictors must be dummy-coded (`step_dummy()`) so distances can be computed.

KNN in tidymodels (Lab 2; HW3 Q8)

Model: `nearest_neighbor()` with argument `neighbors` (k; default 5 — Lab 2's fit reports "Best k: 5"), engine `set_engine("kknn")` (the only engine; needs `library(kknn)`), and `set_mode()` *required* ("regression" or "classification", because KNN can do both).

```
knn_rec <- recipe(y ~ ., data = train) |>
  step_dummy(all_nominal_predictors()) |>
  step_normalize(all_numeric_predictors())

knn_mod <- nearest_neighbor(neighbors = 7) |>
  set_engine("kknn") |>
  set_mode("classification")    # or "regression"

knn_wf <- workflow() |>
  add_model(knn_mod) |>
  add_recipe(knn_rec)
knn_fit <- fit(knn_wf, data = train)

predict(knn_fit, new_data = test)        # .pred_class / .pred
predict(knn_fit, new_data = test, type = "prob")
augment(knn_fit, new_data = test) |>
```

```
  accuracy(truth = y, estimate = .pred_class)
```

Regression version (Lab 2): `set_mode("regression")`, then `predict()` + `bind_cols()` with the true `price` and a `metric_set(rmse, rsq, mae)`. `extract_fit_parsnip()` on a KNN fit shows little (no coefficients). (Engine detail: kknn weights neighbours by distance by default, "Best kernel: optimal".)

Spot-the-bug: KNN code (PE Q7)

- `y ~ .` silently includes an identifier variable (e.g. `application_id`) and post-outcome info (e.g. `default_date` = target leakage) ⇒ drop them.
- No `step_normalize()` ⇒ distances dominated by large-scale variables.
- Recipe defined but never used: `fit(defaulted ~ ., data = loans)` fits the raw data, so the `step_dummy()` never happens ⇒ put recipe + model in a `workflow()`.
- `one_hot = TRUE` is **fine** for KNN (no intercept, so no dummy-variable collinearity problem).
- Fitting/evaluating on all data (no train/test split) gives an optimistic error estimate.

2 Linear models, feature engineering, EDA

Slide map: S2.3–9 EDA • S2.10–39 linear regression • S2.40–46 categorical predictors • S2.47–56 interactions & polynomials • S2.57–75 diagnostics • S2.76–78 LR vs KNN, bias–variance.

2.1 Exploratory data analysis (EDA)

What EDA is for

Goals (S2.4): data cleaning & quality check; descriptive statistics; data visualization; ideas for feature engineering; discovering relationships among variables. **Iterative loop**: (1) generate questions about your data; (2) search for answers by visualizing, transforming and modeling; (3) use what you learn to refine questions / generate new ones. Not a fixed set of rules — "a state of curiosity about data" (HW1).

- Two questions that are always useful (S2.8): what **variation** occurs *within* my variables? what **covariation** occurs *between* my variables? You do not need to look at everything; ask questions, answer them, present the 3–5 most interesting results.
- Assess the data (S2.5, S2.9): # rows/columns; variable types (numeric, categorical, ordered); missing data (% by variable, why, impute?); unusual values — extremely large/small, impossible, unusual within context. Real data is messy.

Which plot for which question

Question	Plot	ggplot2 layer
Shape of one numeric var (skew, modes, outliers)	histogram, density plot	`geom_histogram()` (`bins=60`), `geom_density()`
Counts of one categorical var; class imbalance	bar plot	`geom_bar()`, `count()`
Numeric vs. categorical	boxplot (+ jitter)	`geom_boxplot()`, `geom_jitter(alpha=.3)`
Two numeric vars	scatterplot	`geom_point()`, `geom_smooth()`
All numeric pairs	correlation matrix	`cor()` + `corrplot()`

Boxplot: box = Q1–Q3 (IQR), line = median, whiskers to the most extreme points within 1.5 IQR, dots beyond = potential outliers. **skewness**: *right (positive) skew* = mass at low values, long right tail, mean > median: `diamonds price` (most < $\$10{,}000$) (Lab 1); `abalone` age (mean 11.4 > median 10.5, max 30.5) (HW2 Q1). Left skew = mirror image. Bimodal: `mpg` hwy (peaks ≈17 and ≈26 mpg: trucks/SUVs vs. small cars) (HW1 Ex1). Skewed outcome → stratify the split, consider a log transformation.

```
diamonds %>% select(where(is.numeric)) %>% cor() %>%
  corrplot(type = "lower", diag = FALSE, method = "color")
ggplot(diamonds, aes(x = price)) + geom_histogram(bins = 60)
ggplot(diamonds, aes(x = price, y = reorder(cut, price),
                     fill = color)) + geom_boxplot()
```

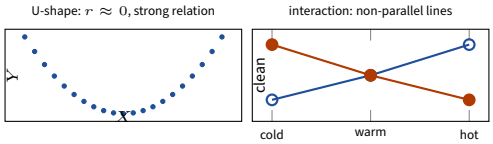
```
ggplot(mpg, aes(x = factor(cyl), y = hwy)) +
  geom_boxplot() + geom_jitter(alpha = 0.3)   # HW1 Ex4
ggplot(mpg, aes(y = fct_rev(fct_infreq(manufacturer)))) +
  geom_bar()        # HW1 Ex3: flipped bar plot, ordered by height
```

EDA findings to remember. `mpg` (HW1): hwy vs `cty` strong positive linear; median `hwy` falls with `cyl` (29/24/17 mpg for 4/6/8 cyl); `cty`–`hwy` and `displ`–`cyl` strongly +, each pair negatively correlated with the other. `diamonds` (Lab 1, S2.74): $r(\text{price}, \text{carat}) = 0.92$; x, y, z correlate 0.86–0.88 with price and 0.95–0.98 with carat (collinearity). Worst colour J costs *more* because J diamonds tend to be larger (carat) — a third variable drives both: confounding.

Pearson correlation

$$r_{xy} = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2 \sum_i (y_i - \bar{y})^2}} \in [-1, 1]$$

Measures the strength and direction of **linear** association *only*. $|r| = 1$ iff points lie exactly on a line; r is unitless, symmetric, unchanged by centering/scaling. In simple linear regression $R^2 = r^2$ (ISLR). `cor()` returns `NA` if any `NA` is present.



Left (PE Q8): $Y = X^2$, Pearson $r \approx 0$. Right (S2.50–51): cleanliness vs. water temperature, one line per detergent; the best detergent depends on temperature = interaction. Parallel lines = main effects only, no interaction.

Correlation myths (PE Q4, Q8, Q9)

- **Zero correlation ≠ useless predictor.** U-shaped scatterplot with $r = 0.03$ (PE Q8): correct answer = "little *linear* correlation despite a strong nonlinear relationship". NOT "unrelated", NOT "plot is wrong", NOT "X won't be useful" — X helps via an X^2 term, KNN or other flexible models (PE Q4a).
- **Correlated predictors ≠ interaction.** $r(X_1, X_2) = 0.91$ is evidence of collinearity, not of an interaction term. Interaction = the relationship between X_1 and Y *depends on the value of* X_2, e.g. positive for one level of X_2, negative for the other (PE Q4b, Q9). Skewness also says nothing about interaction.
- Correlation ≠ causation (confounding, e.g. colour J vs. carat). Lower training error is not always better (PE Q4c).

2.2 Missing data & imputation

Handling missing values

Look first (S2.5, Lab 2): `vis_miss()` (`naniar`) on `airquality`: 4.8% of all cells missing; Ozone 24.18%, `Solar.R` 4.58%, others 0%. Fine to inspect missingness before splitting. **Why missing?** (S2.6) (dog eats homework H; H^* = observed homework; A = student attribute): **MCAR**(missing completely at random) — dog eats *any* homework, missingness unrelated to anything; **MAR**(missing at random) — dog eats *some students'* homework, missingness depends on an observed attribute A; **MNAR**(missing not at random) — dog eats *bad* homework, missingness depends on the missing value itself. Patterns: univariate, monotone, general, planned. **Options** (Lab 2): (1) drop the variable (not if it is the outcome); (2) drop rows, `drop_na()`: `airquality` 153 → 111 rows (large loss); (3) imputation = predict the missing values. **Imputation steps** (S2.7): numeric: `step_impute_bag()`, `step_impute_knn()`, `step_impute_linear()`; categorical: bag, knn, `step_impute_mode()`; also `step_impute_mean()`/`median`. mean imputation (and median/-mode) **reduces variance** (all imputed values identical); KNN, bagging or linear imputation tend to perform better. **Rules**: (a) handle *all* missingness somehow (different methods per variable is fine), (b) consider *why* it is missing, (c) *report* what you did. **Gotchas**: `mean(x)` is `NA` if any `NA` (use `na.rm = TRUE`); `geom_histogram()` silently drops `NA` rows ("Removed 37 rows containing non-finite values").

```
air_recipe <- recipe(Ozone ~ ., data = airquality_train) %>%
  step_impute_linear(Ozone,
       impute_with = imp_vars(Wind, Temp, Month, Day)) %>%
  step_impute_linear(Solar.R,
       impute_with = imp_vars(Wind, Temp, Month, Day))
prep(air_recipe) %>% bake(new_data = airquality_train) %>%
  vis_miss()                    # now 0% missing
```

2.3 Data splitting

Train/test split with rsample

```
set.seed(3435)                  # same random split every run
diamonds_split <- initial_split(diamonds, prop = 0.80,
                                strata = price)
diamonds_train <- training(diamonds_split)
diamonds_test  <- testing(diamonds_split)
dim(diamonds_train); dim(diamonds_test)     # verify sizes
```

- `prop` = fraction that goes to the **training** set (default $3/4$). (HW2 Q2): `prop = 0.85, strata = age` on 4177 abalone → 3548 train / 629 test.
- `set.seed()`: splitting is random; without a random seed every re-run gives a different split and different results (reproducibility). Any number — just remember it.
- `strata` = **stratified sampling**: sample *within* strata of the outcome so train and test have approximately the same outcome distribution (same class proportions; a numeric outcome is binned into quartiles, `breaks = 4`). **Why**: with a skewed outcome or class imbalance (e.g. `titanic`: most passengers did not survive) a plain random split can leave too few rare cases / extreme values in one set → unrepresentative fit or test estimate (Lab 3, HW3, ROC reading).
- **Never touch the test set** until the very end: EDA-driven choices, recipe estimation (`prep()`: means, SDs, imputation models), fitting and tuning all use `training` only; the test set is used once to estimate test error. Splitting *before* EDA is "arguably better practice" (Lab 1).

2.4 Linear regression: model and least squares

The model and its assumptions

(S2.12–14) The best possible prediction under squared error is $\hat{f}(\mathbf{x}_0) = E[Y \mid X = \mathbf{x}_0]$ — unknown because the joint distribution of (X, Y) is unknown. So **make an assumption**:

$$Y = \beta_0 + X_1\beta_1 + \cdots + X_p\beta_p + \varepsilon, \quad E[\varepsilon] = 0,\ \varepsilon \perp (X_1, \ldots, X_p)$$

$\Rightarrow E[Y \mid X = \mathbf{x}_0] = \beta_0 + x_{01}\beta_1 + \cdots + x_{0p}\beta_p$: the conditional expectation is **linear in the parameters**. simple linear regression: $p = 1$, data $\{(x_1, y_1), \ldots, (x_n, y_n)\}$; multiple linear regression: $p > 1$, $\mathbf{x}_i = (x_{i1}, \ldots, x_{ip})$. ε = catch-all for what the simple model misses, mean zero, independent of everything else (S2.24). A parametric model: estimate $p + 1$ numbers instead of a whole function. **Assumptions** (S2.48, S2.58–61): (1) **linear** and **additive** in the predictors; (2) errors $\varepsilon_1, \ldots, \varepsilon_n$ **uncorrelated**; (3) **constant variance** $\mathrm{Var}(\varepsilon_i) = \sigma^2$; plus no influential outliers / severe collinearity (diagnostics below). **Picture** (S2.15–18): y = confirmed COVID cases, x = county population. intercept β_0 = mean y when $x = 0$; slope β_1 = change in mean y per one-unit increase in x (+10,000 people $\Rightarrow +10000\beta_1$ cases). Different (β_0, β_1) give different lines ($y = 8000 + 0.057x$ vs. $5000 + 0.061x$); residual for Riverside $= y_{RS} - \beta_0 - \beta_1 x_{RS}$ = observed − predicted.

Least squares estimates (S2.20–29)

Choose $\hat{\beta}$'s to minimize the sum of squared residuals

$$\mathrm{SSR} = \sum_{i=1}^{n} \left(y_i - \hat{\beta}_0 - \hat{\beta}_1 x_{i1} - \cdots - \hat{\beta}_p x_{ip}\right)^2 \quad \text{(ISLR: RSS)}$$

$$\hat{\beta}_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \qquad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

(the line passes through $(\bar{x}, \bar{y})$). We cannot get $\hat{\beta} = \beta$ exactly: the data are random, so $\hat{\beta}_0, \hat{\beta}_1$ are random variables (new sample → new line, S2.25). **Unbiased**: $E(\hat{\beta}_1) = \beta_1$, $E(\hat{\beta}_0) = \beta_0$, i.e. Bias $= 0$ (S2.27).**Variance** ($\sigma^2 = \mathrm{Var}(\varepsilon)$):

$$\mathrm{SE}(\hat{\beta}_1)^2 = \frac{\sigma^2}{\sum_i (x_i - \bar{x})^2}, \quad \mathrm{SE}(\hat{\beta}_0)^2 = \sigma^2 \left[\frac{1}{n} + \frac{\bar{x}^2}{\sum_i (x_i - \bar{x})^2}\right]$$

σ^2 unknown → $\hat{\sigma}^2 = \mathrm{SSR}/(n-2)$ and plug in ($\widehat{\mathrm{SE}}$). More spread in x and larger $n \Rightarrow$ smaller SE. Approx. 95% confidence interval: $\hat{\beta}_j \pm 2\,\mathrm{SE}(\hat{\beta}_j)$ (ISLR).

Model accuracy and tests (S2.30, S2.34–39)

$\mathrm{SST} = \sum_i (y_i - \bar{y})^2$, $\mathrm{SSR} = \sum_i (y_i - \hat{y}_i)^2$. **RSE**(residual standard error) $= \sqrt{\mathrm{SSR}/(n-p-1)}$ ($n-2$ in SLR): absolute lack of fit, in units of Y. $R^2 = 1 - \mathrm{SSR}/\mathrm{SST} \in [0, 1]$ = proportion of the variability in Y explained by X. R^2 **always increases** as predictors are added; RSE and R^2 favor flexible models, which may overfit. **adjusted R-squared** $= 1 - \dfrac{(1 - R^2)(n-1)}{n-p-1}$ (penalizes extra predictors). **F-statistic** for $H_0 : \beta_1 = \cdots = \beta_p = 0$ vs. H_a: at least one $\beta_j \neq 0$:

$$F = \frac{(\mathrm{SST} - \mathrm{SSR})/p}{\mathrm{SSR}/(n-p-1)}$$

Under H_0 both numerator and denominator $\approx \sigma^2 \Rightarrow F \approx 1$; **large** F **favors** H_a. **t-statistic** $t = \hat{\beta}_j / \mathrm{SE}(\hat{\beta}_j)$ tests $H_0 : \beta_j = 0$ *given the other predictors*; small p-value (< 0.05) $\Rightarrow$ reject.

Three questions (S2.34): Q1 is at least one predictor useful? (F-test). Q2 which subset matters? (selection). Q3 how well does the model fit? (RSE, R^2). **Variable selection** (S2.36–37): best subset selection fits all 2^p subsets ($\approx$1 billion for $p = 40$) and picks the "best" by Mallow's C_p, AIC, BIC or adjusted R^2; forward selection starts from the null model and adds the most significant variable one at a time (greedy) until a stopping rule; backward selection starts from the full model and removes the least significant one at a time (impossible when $p > n$); mixed selection combines both.

Reading lm / tidy() output

(Lab 2): `diamonds`, `price ~ .`, `step_dummy()` on all nominal predictors, `linear_reg()` + `set_engine("lm")`.

```
lm_fit %>% extract_fit_parsnip() %>% tidy()
  term         estimate std.error statistic   p.value
1 (Intercept)  5317.       496.      10.7    8.54e- 27
2 carat       11282.        54.3    208.     0
3 depth         -58.4        6.25    -9.33   1.07e- 20
5 x            -970.        49.0    -19.8    5.16e- 87
6 y               9.08      19.9      0.456  6.49e-  1
8 cut_1         596.        25.0     23.8    1.22e-124
# ... (18 more rows)
```

- `estimate` = $\hat{\beta}_j$ (coefficient); `std.error` = $\mathrm{SE}(\hat{\beta}_j)$; `statistic` = t = estimate/std.error ($11282/54.3 \approx 208$); `p.value` for $H_0 : \beta_j = 0$.
- **carat**: holding all other predictors fixed, +1 carat is *associated* with +\$11,282 in mean price; $p \approx 0 \Rightarrow$ significant. (Association, not causation.)
- **y**: $p = 0.649 \Rightarrow$ no evidence that y adds information *once carat, x, z are in the model* — not "y is unrelated to price" ($r = 0.87$). **x** is *negative* although $r(x, \text{price}) = +0.88$: coefficients are partial effects and collinearity can flip signs.
- **Intercept** = predicted price when every numeric predictor (and contrast) is 0 — an extrapolation (carat $= 0$), not meaningful here.
- `cut_1`… are **not** 0/1 dummies: `cut`, `color`, `clarity` are *ordered* factors (<ord>), so `step_dummy()` makes polynomial contrasts (`cut_1` linear, `cut_2` quadratic, …; values like 0.632, -0.267).
- Fit: train RMSE 1126 / $R^2$0.920, test RMSE 1146 / R^2 0.918 (no overfitting), but the predicted-vs-actual plot is curved and some predicted prices are negative $\Rightarrow$ a linear f is a poor approximation here.

2.5 Categorical predictors: dummy variables

Dummy (indicator) coding

(S2.41–44) categorical predictors (qualitative / factor variables) can enter a regression whose Y is quantitative via dummy variables. **2 levels** (Credit: balance vs. gender): $x_i = 1$ if female, 0 if male $\Rightarrow y_i = \beta_0 + \beta_1 + \varepsilon_i$ (female), $\beta_0 + \varepsilon_i$ (male). β_0 = mean balance of males (baseline); β_1 = average difference female $-$ male. **Coding is not unique**: 1=male/0=female flips the sign of β_1 and β_0 becomes the female mean; ± 1 coding (1 male, -1 female) gives β_0 = average of the two group means, β_1 = half the male$-$female difference. Same fitted values, different interpretation. m **levels** $\Rightarrow m-1$ **dummies.** Ethnicity {African American, Asian, Caucasian}: $x_{i1} = 1$ if Asian, $x_{i2} = 1$ if Caucasian; African American = **baseline**(no dummy).

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \varepsilon_i = \begin{cases} \beta_0 + \beta_1 + \varepsilon_i & \text{Asian} \\ \beta_0 + \beta_2 + \varepsilon_i & \text{Caucasian} \\ \beta_0 + \varepsilon_i & \text{Afr. American} \end{cases}$$

β_0 = mean of the baseline; β_j = difference between level j and the baseline. Coding and choice of baseline are arbitrary but change the interpretation of $\beta_0, \beta_1, \beta_2$. (PE Q11): only categorical variables (`Major`, `Dorm`) need dummy encoding; numeric ones (`HS_GPA`, `CreditsCompleted`) do not.

step_dummy() vs one-hot (S2.45–46)

```
recipe(age ~ type, data = abalone_train) %>%   # type: F, I, M
  step_dummy(type)             # -> type_I, type_M  (F =
    ↪ reference)
recipe(age ~ type, data = abalone_train) %>%
  step_dummy(type, one_hot = TRUE)  # -> type_F, type_I, type_M
    ↪
```

New columns are named `var_level`. Default = $m-1$ **dummy coding**; the reference level is the *first factor level* (alphabetical for characters): a row with `type_I = 0, type_M = 0` is F. Change it with `step_relevel(type, ref_level = "M")` or `fct_relevel()`. `one_hot = TRUE` = one-hot encoding: one 0/1 column per level. Usually `step_dummy()(all_nominal_predictors())` (Lab 2).

one_hot = TRUE + intercept = rank deficient (PE Q13)

`salary ~ experience + department` (Sales, IT, HR) with `step_dummy(department, one_hot = TRUE)` and `linear_reg()`: R reports the model matrix is **rank-deficient**, a coefficient cannot be identified (`NA`, "prediction from a rank-deficient fit may be misleading"). (a) Cause: one-hot makes 3 indicators, one per level. (b) For every row they sum to 1 = the intercept column (a column of ones) $\Rightarrow$ perfect linear dependency, design matrix not full rank ("dummy-variable trap"). (c) Fix: `one_hot = FALSE` (the default). (d) Then **2** indicators are created ($m-1$). One-hot is **fine for KNN** (no intercept, no linear dependency problem) (PE Q7).

2.6 Interactions and nonlinear terms

Relaxing additivity and linearity

(S2.48, S2.54) The plain model is **additive** — the effect of changing X_j on Y does not depend on the other predictors (relax with **interaction terms**) — and **linear** — a one-unit change in X_j changes Y by the same amount whatever the value of X_j (relax with **polynomial regression** / transformations).

Interaction (S2.49)

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1 X_2 + \varepsilon = \beta_0 + (\beta_1 + \beta_3 X_2) X_1 + \beta_2 X_2 + \varepsilon$$

Without X_1X_2: $+1$ in $X_1 \Rightarrow +\beta_1$ in Y (holding X_2 fixed). With it: $+1$ in $X_1 \Rightarrow +(\beta_1 + \beta_3 X_2)$ — the effect of X_1 is no longer constant; changing X_2 changes the impact of X_1 (and symmetrically the effect of X_2 is $\beta_2 + \beta_3 X_1$). **Definition**: an interaction term is needed when the relationship between X_1 and Y differs depending on the value of X_2. It says nothing about whether X_1 and X_2 are correlated (PE Q4b, Q9).

Worked examples

Advertising (S2.52): sales $= \beta_0 + \beta_1$TV $+ \beta_2$radio $+ \beta_3$TV $\times$ radio $+ \varepsilon$.

	Coef.	SE	t	p
Intercept	6.7502	0.248	27.23	$<$0.0001
TV	0.0191	0.002	12.70	$<$0.0001
radio	0.0289	0.009	3.24	0.0014
TV$\times$radio	0.0011	0.000	20.73	$<$0.0001

$+\$1{,}000$ TV $\Rightarrow (\hat\beta_1 + \hat\beta_3$ radio$) \times 1000 = 19.1 + 1.1 \times$ radio increase in sales. **Practice answer**: $+\$1{,}000$ radio $\Rightarrow (\hat\beta_2 + \hat\beta_3$ TV$) \times 1000 = 28.9 + 1.1 \times$TV. **Dummy** $\times$ **numeric** (Credit, S2.53): balance $= \beta_0 + \beta_1$income$+ \beta_2$student $+ \beta_3$ income $\times$ student $\Rightarrow$ students: $(\beta_0 + \beta_2) + (\beta_1 + \beta_3)$ income; non-students: $\beta_0 + \beta_1$ income. Without the interaction the two lines are **parallel** (same slope, different intercepts); with it the slopes differ. **Interaction plots** (S2.50–51): one line per level of A over levels of B. Parallel lines = no interaction (vertical gap = A main effect, common slope = B main effect); non-parallel or crossing lines = interaction. **hierarchical principle** (ISLR): if X_1X_2 is in the model, keep X_1 and X_2 too, even if their p-values are large.

```
step_dummy(all_nominal_predictors()) %>%     # FIRST make dummies
step_interact(terms = ~ starts_with("type_"):shucked_weight +
                longest_shell:diameter + shucked_weight:shell_
    ↪ weight)
# -> type_I_x_shucked_weight, type_M_x_shucked_weight, ...   (HW2
    ↪ Q3)
```

`step_interact()` needs numeric columns: after `step_dummy()` the factor `type` no longer exists, so refer to its dummies with `starts_with()("type_")`. Slope of `shucked_weight` for infants $= \hat\beta_{\text{shucked}} + \hat\beta_{\text{type_I_x_shucked}}$; for females (reference) $= \hat\beta_{\text{shucked}}$.

Polynomials and transformations (S2.55–56)

polynomial regression of degree d: $Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \cdots + \beta_d X^d + \varepsilon$. Y is no longer linear in X, **but it is still a linear model**: let $Z_k = X^k$, then $Y = \beta_0 + \beta_1 X + \beta_2 Z_2 + \cdots + \beta_d Z_d + \varepsilon$ is linear in $X, Z_2, \ldots, Z_d$ (linear in the parameters). Figure: linear vs. degree 2 vs. degree 5 fits to a curved cloud — degree 2 follows the curve, degree 5 wiggles at the edges (more flexible $\rightarrow$ overfitting risk). transformations: `step_poly()(x, degree = 2)`, `step_log()`; a log transformation tames right skew and funnel-shaped residuals. Transform the *outcome* with `mutate()` (e.g. `log(price)`) before splitting rather than in the recipe.

2.7 Regression diagnostics

Six potential problems (S2.58–65)

Problem	Detect $\rightarrow$ consequence / fix
1. Non-linearity	**residual plot**: $e_i = y_i - \hat y_i$ vs. fitted $\hat y_i$. Correct model $\Rightarrow$ *no discernible pattern*; a U-shape means missed curvature $\rightarrow$ add X^2, $\log X$, $\sqrt{X}$.
2. Correlated errors	e.g. time series; estimated SEs inaccurate $\rightarrow$ wrong CIs and tests.
3. Non-constant variance	heteroscedasticity: funnel shape in the residual plot $\rightarrow$ inaccurate CIs and tests; fix by transforming Y ($\log Y$, $\sqrt{Y}$): S2.61 funnel vanishes for $\log(Y)$.
4. outliers	y_i far from $\hat y_i$ (e.g. recording error). Use studentized residuals (residual / its SE; $\lvert\cdot\rvert > 3$ suspicious, ISLR).
5. high leverage points	unusual x_i values. Leverage (SLR) $h_i = \frac{1}{n} + \frac{(x_i - \bar x)^2}{\sum_k (x_k - \bar x)^2}$; $h_i \in [1/n, 1]$, average $(p+1)/n$.
6. collinearity	correlation matrix, VIF (below).

influential observation: removing it changes the fit a lot. It may or may not be an outlier or have high leverage, but tends to have at least one of the two. **Example** (S2.66–72) (fit without the red point: $y = 1.73 + 5.12x$): outlier only $(4, 40)$: $y = 2.96 + 5.04x$ — barely changes, **not** influential; high leverage on the trend $(14, 68)$: $y = 2.47 + 4.93x$ — **not** influential; outlier *and* high leverage $(13, 15)$: $y = 8.51 + 3.32x$ — **influential**. Status is model-specific (S2.65). What to do: 1. check for a data-entry error; 2. exclude the points; 3. try re-including them if the model changes.

Collinearity (S2.73–75)

collinearity: two or more predictors closely related. If they rise and fall together it is hard to separate how each is associated with $Y \Rightarrow$ **variances (SEs) of the estimates increase** $\rightarrow$ large p-values and unstable or flipped signs (diamonds x: -970). **Detect**: (1) correlation matrix of $X_1, \ldots, X_p$ (diamonds: carat–x 0.98, carat–y, z 0.95, x–y 0.97); (2) variance inflation factor (VIF) $\text{VIF}_j = 1/(1 - R^2_{X_j|X_{-j}})$, where R^2 comes from regressing X_j on the other predictors (ISLR: > 5–10 is a problem; also catches multicollinearity among 3+ variables that pairwise correlations miss). **structural collinearity** = created by us from other predictors (X and X^2, interaction terms, a ratio plus its components); **data-based** = present in the observed data (carat vs. x, y, z). **Handle** ("it depends"): 1. drop variables; 2. combine variables; 3. feature extraction (PCA); 4. regularization (ridge, lasso). Collinearity mainly hurts interpretation and inference; predictions can still be fine.

2.8 Feature engineering with recipes

What may go in the model

feature engineering = creating or reformatting predictors (features) so the model can use them: dummy coding, interactions, polynomial/log transformations, normalization, imputation, new combined variables (ratios).

- **target leakage**: a predictor that carries information about the outcome that would *not be available at prediction time*, or is derived from the outcome $\Rightarrow$ unrealistically good performance that collapses in practice. `default_date` when predicting `defaulted` (PE Q7); `GraduationDate`, `FinalGPA` when predicting `Graduated4Years` from end-of-first-year data (PE Q10); `rings` for `age = rings + 1.5` (age is a deterministic function of rings, so the model just learns the formula and the other predictors become useless) (HW2 Q3).
- **Rule**: use only information you would have at the moment the prediction is made.
- **Not leakage** (PE Q12): `CompletionRate` = CreditsCompleted/CreditsAttempted (first-year information, known at prediction time); including it together with both components is redundant (structural collinearity).
- **identifier variables** (`StudentID`, `application_id`): arbitrary labels with no meaning $\Rightarrow$ exclude (spurious patterns; they distort KNN distances). The outcome itself is obviously excluded.
- `recipe(y ~ ., data)`: "`.`" = *every* other column, including IDs and leaky variables $\rightarrow$ drop them first (`select(-id)`, `step_rm()`) or list the predictors explicitly.

Centering, scaling, normalization

centering $x - \bar{x}$ (`step_center()`); scaling x/s (`step_scale()`); both = normalization / standardization $z = (x - \bar{x}_{\text{train}})/s_{\text{train}}$ (`step_normalize()`): mean 0, SD 1.

- **Required for KNN** (distance-based): an unscaled large-variance variable (income) dominates the distance (PE Q6, Q7).
- **Linear regression**: fitted values, R^2, RMSE unchanged; coefficients become "per 1 SD". With all predictors centered the **intercept** $= \bar{y}_{\text{train}}$ (HW2: `(Intercept)` 11.5 $\approx$ mean age).
- `prep()` estimates means/SDs (and imputation models, factor levels) from the **training** data; `bake()` / `predict()` apply those same training values to new data. `bake(new_data = NULL)` returns the processed training set.
- **Step order** (recipes docs): impute $\rightarrow$ transform (log, poly) $\rightarrow$ `step_dummy` $\rightarrow$ `step_interact` $\rightarrow$ normalize.

HW2 pipeline: abalone age, LR + KNN

```
abalone <- read_csv("data/abalone.csv") %>%
  mutate(age = rings + 1.5, type = factor(type))
abalone_recipe <- recipe(age ~ type + longest_shell + diameter +
    height + whole_weight + shucked_weight + viscera_weight +
    shell_weight, data = abalone_train) %>%        # NO rings
  step_dummy(all_nominal_predictors()) %>%
  step_interact(terms = ~ starts_with("type_"):shucked_weight +
      longest_shell:diameter + shucked_weight:shell_weight) %>%
  step_center(all_predictors()) %>%
  step_scale(all_predictors())   # = step_normalize(all_
    ↪ predictors())
lm_model  <- linear_reg() %>% set_engine("lm")
knn_model <- nearest_neighbor(neighbors = 7) %>%
  set_engine("kknn") %>% set_mode("regression")
lm_wflow <- workflow() %>% add_model(lm_model) %>%
  add_recipe(abalone_recipe)
lm_fit <- fit(lm_wflow, data = abalone_train)  # preps recipe
predict(lm_fit, new_data = new_abalone)     # one row -> .pred
abalone_metrics <- metric_set(rsq, rmse, mae)
augment(lm_fit, new_data = abalone_test) %>%   # adds .pred
  abalone_metrics(truth = age, estimate = .pred)
```

Q7: the new observation must be a tibble with *every* predictor the recipe uses and the same types (`type = factor("F", levels = levels(abalone$type))`); the workflow applies the recipe automatically $\rightarrow$ `.pred` = 21.8. **Q8 results (test)**: LR R^2 0.585, RMSE 1.95, MAE 1.44; KNN ($k = 7$) R^2 0.483, RMSE 2.21, MAE 1.62. Training: KNN R^2 0.811 vs. LR 0.552 $\Rightarrow$ KNN overfits (each training point is its own nearest neighbour); LR generalizes better. Interpretation: $R^2 = 0.585$ $\Rightarrow$ 58.5% of the variability in test-set age is explained by the model. (`yardstick` `rsq()` = squared correlation of truth and `.pred`; `rsq_trad()` $= 1 - \text{SSR}/\text{SST}$.)

2.9 Linear regression vs. KNN regression

Parametric vs non-parametric (S2.76)

Linear regression (parametric model)	**KNN regression** (non-parametric model)
assume $f(X) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p$	$f(X)$ can have any functional form
no need to tune	tuning parameter K (`neighbors`)
performs well when the true f is close to linear	much more general-purpose
interpretable; statistical inference (SE, p-values)	not very interpretable
extends to very large p (ridge, lasso)	curse of dimensionality

KNN regression predicts $\hat{f}(x_0)$ = average of the y_i of the K training points closest to x_0 (ISLR); it needs normalized predictors and no interaction terms (non-parametric) (PE Q1). **Lab 2 (diamonds), RMSE train / test**: LR 1126 / 1146; KNN $k = 5$: 386 / 711; KNN $k = 2$: 152 / 851. KNN wins on test because the true f is nonlinear; $k = 2$ has the biggest train–test gap (most flexible $\rightarrow$ overfits more).

Bias–variance of least squares (S2.77)

$E\left[(y_0 - \hat{f}(\mathbf{x}_0))^2\right] = \text{Var}(\hat{f}(\mathbf{x}_0)) + [\text{Bias}(\hat{f}(\mathbf{x}_0))]^2 + \text{Var}(\varepsilon)$.If the true f is linear and $\hat{f} = \hat{\beta}_0 + \hat{\beta}_1 x_{01} + \cdots + \hat{\beta}_p x_{0p}$ (least squares): **Property 1**: unbiased, $\text{Bias}(\hat{f}(\mathbf{x}_0)) = E[\hat{f}(\mathbf{x}_0)] - f(\mathbf{x}_0) = 0$. **Property 2** (Gauss-Markov theorem): least squares has the **smallest** expected test error among all **unbiased linear** estimators. Modern methods (ridge, lasso) can beat least squares in expected test MSE by accepting a *small bias* for a *much smaller variance*.

3 Classification models: logistic regression, LDA, QDA, naive Bayes

3.1 Classification set-up; why not linear regression?

Classification in general

- Predicting a qualitative response for an observation = classification: assign the observation to a class. Usually we first predict the *probability* of each class (a quantitative number) and then classify (S3.4).
- Classifiers in this course: KNN, logistic regression, discriminant analysis (LDA, QDA), naive Bayes.
- Running example: `Default` data (ISLR): $Y =$ `default` (Yes/No); $X_1 =$ annual `income`, $X_2 =$ monthly credit-card `balance`, $X_3 =$ `student`. Defaulters have much higher balances; income barely differs (S3.8).

Why not linear regression for a categorical Y?

- **Binary** $Y \in \{0,1\}$: linear regression models $\Pr(Y=1 \mid X) = \beta_0+\beta_1X_1+\cdots+\beta_pX_p$, an unbounded line $\Rightarrow$ fitted "probabilities" can be <0 (Default data: **negative probability** for balance $\lesssim 500$) or >1. Logistic regression's S-curve stays in $[0,1]$ (S3.5).
- >2 **classes**: coding $Y=1$ (stroke), 2 (drug overdose), 3 (seizure) imposes an **ordering** and equal spacing (stroke→overdose gap = overdose→seizure gap); a different coding (1 stroke, 2 seizure, 3 overdose) gives a different model; {dog, cat, fish} has no order at all (S3.6).
- Course focuses on **binary** classification (multi-class logistic regression exists) (S3.7).

3.2 The logistic regression model

Logistic regression: three equivalent forms

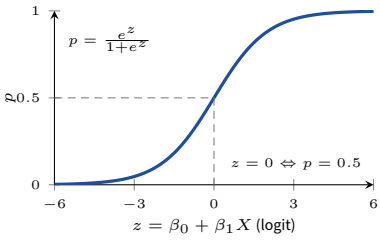
With $p(X) = \Pr(Y=1 \mid X)$ (S3.10–11, S3.24):

$$p(X) = \frac{e^{\beta_0+\beta_1X}}{1+e^{\beta_0+\beta_1X}} \in [0,1] \qquad \text{logistic function}$$

$$\frac{p(X)}{1-p(X)} = e^{\beta_0+\beta_1X} \in [0,\infty) \qquad \text{odds}$$

$$\log\left(\frac{p(X)}{1-p(X)}\right) = \beta_0+\beta_1X \in (-\infty,\infty) \qquad \text{log odds} = \text{logit}$$

The logit is linear in X (not p!). Multiple predictors: replace $\beta_0+\beta_1X$ by $\beta_0+\beta_1X_1+\cdots+\beta_pX_p$ everywhere (S3.24). $\log = \ln$ (natural log). Equivalent: $p = \frac{1}{1+e^{-z}}$ with $z = \beta_0+\beta_1X$ (the sigmoid).
Odds $= p/(1-p)$, *not* p: rolling a 3 on a fair die has $p = 1/6$, odds $= \frac{1/6}{5/6} = 1/5$ (S3.10). $p = 0.8 \Rightarrow$ odds $= 4$ ("4 to 1").



p	odds	log odds
0	0	$-\infty$
0.269	0.368	-1
0.5	1	0
0.731	2.72	1
0.88	7.33	2
0.95	19	3
1	∞	$+\infty$

p on $[0,1] \mapsto$ log odds on $(-\infty,\infty)$ (S3.16–21)

Interpreting logistic coefficients

(S3.12–13, S3.25)

- $\beta_1 =$ the **additive** change in the logit (log odds) for a 1-unit increase in X.
- e^{β_1} = the **multiplicative** change in the odds: odds$(X+1)$/odds$(X) = e^{\beta_1}$ (the odds ratio). A c-unit increase multiplies the odds by $e^{c\beta_1}$ (HW3 Q12: $+2$ units $\Rightarrow \times e^{2\beta_1}$). % change in odds $= 100(e^{\beta_1}-1)\%$.
- β_1 is **NOT** the change in $\Pr(Y=1 \mid X)$ per unit of X. Only the **sign** carries over: $\beta_1>0 \Rightarrow X \nearrow \Rightarrow p \nearrow$; $\beta_1<0 \Rightarrow X \nearrow \Rightarrow p \searrow$; $\beta_1=0 \Rightarrow$ no association.
- The rate of change of p per unit X **depends on the current value of** X — not linear (calculus: $dp/dX = \beta_1p(1-p)$, steepest, $= \beta_1/4$, at $p=0.5$).
- Multiple logistic regression: $\beta_j =$ change in logit per unit X_j **conditional on (holding fixed) the other predictors** (S3.25).
- $\beta_0 =$ log odds when all X's $= 0$. Regression on observational data shows **association, not causation**.

PE Q16: coefficient 0.04 is NOT "+4% probability"

Logistic regression for cancelling a subscription; coefficient of `MonthlyFee` $= 0.04$.
Wrong: "A \$1 increase in the monthly fee *causes* a customer's probability of cancelling to increase by 4%." Problems: (1) it treats the probability as linear in the fee — the relationship is linear in the log odds, and the change in p depends on the current fee; (2) "causes" — only **association** can be claimed.
Right: "A \$1 increase in the monthly fee is associated with a 0.04 increase in the log odds of cancelling" (equivalently, odds $\times e^{0.04} \approx 1.041$, about $+4.1\%$ in the *odds*), or "a higher fee is associated with a higher probability of cancelling" (sign is positive) (PE Q16).

HW3 Q11–Q12 (231): logit is the inverse of the logistic

$p = \frac{e^z}{1+e^z} \iff z = \ln\frac{p}{1-p}$ (logit and logistic are inverses) (HW3 Q11).
$z = \beta_0+\beta_1x_1$: increasing x_1 by 2 multiplies the odds by $e^{2\beta_1}$. If $\beta_1<0$: $x_1 \to \infty \Rightarrow p \to 0$; $x_1 \to -\infty \Rightarrow p \to 1$ (HW3 Q12).

3.3 Estimating the coefficients: maximum likelihood

Fitting with glm()

- Coefficients are estimated by maximum likelihood (MLE), *not* least squares (S3.26): choose $\hat\beta$'s so $\hat p(x_i)$ is near 1 when $y_i=1$ and near 0 when $y_i=0$, i.e. maximise $\ell(\beta_0,\beta_1) = \prod_{i:y_i=1}p(x_i)\prod_{i':y_{i'}=0}(1-p(x_{i'}))$ (ISLR 4.3.2).
- Base R: `glm()` with `family = binomial` ("generalized linear model"):

```
glm.fit <- glm(default ~ student + balance + income,
               data = Default, family = binomial)
summary(glm.fit)
```

```
             Estimate Std. Error z value Pr(>|z|)
(Intercept) -1.087e+01  4.923e-01 -22.080  < 2e-16 ***
studentYes  -6.468e-01  2.363e-01  -2.738  0.00619 **
balance      5.737e-03  2.319e-04  24.738  < 2e-16 ***
income       3.033e-06  8.203e-06   0.370  0.71152
    Null deviance: 2920.6  on 9999  degrees of freedom
Residual deviance: 1571.5  on 9996  degrees of freedom
AIC: 1579.5
```

- `z value` = Estimate/Std. Error (a z-statistic, not t); small p-value $\Rightarrow$ reject H_0: $\beta_j=0$ given the other predictors. Here income is not significant ($p=0.71$) once balance and student are in the model.
- `studentYes` is the dummy variable for student = Yes (reference level = No).
- Units: `income` coefficient is 3.033×10^{-6} per \$1; the slide tables show 0.0030 because there income is in \$1000s.
- deviance: residual deviance < null deviance $\Rightarrow$ predictors help; lower AIC is better.

Perfect separation makes logistic regression unstable

If $Y=0$ for $X \le 50$ and $Y=1$ for $X>50$ (**perfectly separable**), `glm(y~x, family = "binomial")` warns "`algorithm did not converge`" and "`fitted probabilities numerically 0 or 1 occurred`": the MLE $\hat\beta_1 \to \infty$, so the coefficient estimates are unstable. **LDA does not suffer from this problem** (S3.45).

3.4 Multiple logistic regression and confounding

The student paradox on the Default data

(S3.27–29)

	`default~student`	`~balance+income+student`
Intercept	-3.5041	-10.8690
balance	—	0.0057 ($p<.0001$)
income (\$1000s)	—	0.0030 ($p=.7115$)
student[Yes]	$\mathbf{0.4049}$ ($p=.0004$)	$\mathbf{-0.6468}$ ($p=.0062$)

- **Contradicting signs!** Simple model: a student is **riskier** than a non-student *if no other information* about the student is available. Multiple model: a student is **less risky** than a non-student *with the same credit-card balance* (and income).
- Why: **student and balance are correlated** — students are more likely to carry large balances, which are associated with high default rates. Balance confounding the student effect.
- Lesson: pay extra attention when regressing on a **single predictor** while other relevant predictors exist (confounding; Simpson's paradox: S3.30 shows two groups each with a positive y–x trend, but the pooled data trend is negative) (S3.29–31).

3.5 Predicted probabilities and the threshold

Worked prediction (Default data)

$$\widehat{\Pr}(\text{default} = \text{Yes} \mid \text{balance} = 1500, \text{income} = \$40\text{k}, \text{student} = \text{No})$$

$$= \frac{e^{-10.869+0.00574(1500)+0.0030(40)-0.6468(0)}}{1+e^{-10.869+0.00574(1500)+0.0030(40)-0.6468(0)}}$$

$$= \frac{e^{-2.14}}{1+e^{-2.14}} = 0.105$$

Same person but student = Yes: $z = -2.14 - 0.6468 = -2.79 \Rightarrow \hat{p} = 0.058$ (S3.32). Recipe: (1) $z = \hat{\beta}_0 + \sum \hat{\beta}_j x_j$; (2) $\hat{p} = e^z/(1+e^z)$. The slide writes "0.0030×40000", but with income in \$1000s you must plug in 40 (40000 would give $\hat{p} \approx 1$); fully rounded $\hat{\beta}$'s give ≈ 0.10.

From predicted probability to predicted class

(S3.33)

- Classify by **thresholding** the predicted probability: predict default = Yes if $\hat{p} > 0.5$ (the usual threshold; $\hat{p} > 0.5 \iff \text{logit} > 0$).
- A **conservative** credit company might predict default = Yes whenever $\hat{p} > 0.1 \Rightarrow$ more individuals flagged: more true positives *and* more false positives.
- Raising the threshold $\Rightarrow$ fewer positive classifications (PE Q15): $\hat{p} = .31, .48, .63, .81$ for borrowers A–D; threshold $0.5 \Rightarrow$ C, D; threshold $0.7 \Rightarrow$ only D.
- Changing the threshold changes the confusion matrix, sensitivity, specificity — but **not** the ROC curve or AUC (PE Q18). Details: see the model-evaluation section.

PE Q14: what a predicted probability means

Pneumonia model gives one patient $\hat{p} = 0.78$. Correct: **(c)** "the fitted model assigns this patient a probability of 0.78 of belonging to the pneumonia class." NOT "the model is 78% accurate for this patient", NOT "definitely has pneumonia", NOT "78% of the symptoms indicate pneumonia" (PE Q14).

Logistic regression in tidymodels (Lab 3)

(Lab 3) Data: `Smarket` (outcome `Direction`: Down/Up; predictors Lag1–Lag5, `Volume`) and `OJ` (outcome `Purchase`: CH/MM).

```
library(tidymodels); library(discrim); library(klaR)
set.seed(3435)
oj_split <- initial_split(OJ, prop = 0.70, strata = Purchase)
oj_train <- training(oj_split); oj_test <- testing(oj_split)
oj_recipe <- recipe(Purchase ~ PriceCH + PriceMM + DiscCH +
    ↪ DiscMM +
                      LoyalCH + PctDiscMM + PctDiscCH, data = oj
    ↪ _train)
log_reg <- logistic_reg() %>%
  set_engine("glm") %>%
  set_mode("classification")
ojlog_wkflow <- workflow() %>%
  add_model(log_reg) %>% add_recipe(oj_recipe)
ojlog_fit <- fit(ojlog_wkflow, oj_train)
ojlog_fit %>% tidy()
predict(ojlog_fit, new_data = oj_train, type = "prob")
augment(ojlog_fit, new_data = oj_train) %>%
  conf_mat(truth = Purchase, estimate = .pred_class)
```

```
   term         estimate std.error statistic  p.value
1 (Intercept)     4.47      2.21       2.02   4.35e- 2
6 LoyalCH        -6.07      0.450    -13.5    1.48e-41
...
# predict(type = "prob"):    .pred_CH .pred_MM
                          1     0.606    0.394
                          2     0.898    0.102
```

- `predict()` default (`type = "class"`) returns `.pred_class` (threshold 0.5); `type = "prob"` returns one column per level, `.pred_<level>` (rows sum to 1). `augment()` attaches *both* to the data. `tidy()` gives the coefficient table.
- Smarket: training probabilities all ≈ 0.4–$0.6 \Rightarrow$ the model cannot discriminate Up/Down (training accuracy 0.525); OJ: probabilities near 0/1, training accuracy 0.833.

Which level is "Y = 1"? (factor levels)

`glm` (and `logistic_reg()` with `"glm"`) models the probability of the **second** factor level; the first is the baseline. OJ levels are (CH, MM), so the coefficients are log odds of buying **MM**: `LoyalCH` $= -6.07$ means more Citrus-Hill loyalty $\Rightarrow$ lower odds of MM. **yardstick** metrics, in contrast, treat the **first** level as the event (`event_level = "first"`). HW3 therefore reorders `survived` so "Yes" is first (`factor(survived, levels = c("Yes","No"))`) and uses `.pred_Yes` in `roc_auc()` — but then the glm coefficients describe the log odds of *No* (flip the sign to talk about survival).

3.6 Discriminant analysis: Bayes' theorem and the idea

Why bother with LDA?

(S3.45–46) **(1)** logistic regression coefficient estimates can be **unstable** (e.g. well/perfectly separated classes); **(2)** n is **small** and the distribution of X is close to **normal** (in each class); **(3)** multi-class classification ($K > 2$) problems.

Bayes' theorem for classification

The Bayes classifier $\operatorname{argmax}_{k=1,\dots,K} \Pr(Y = k \mid X_1, \dots, X_p)$ is the "best" we can do (S3.48). Logistic regression models $\Pr(Y = 1 \mid X)$ directly; LDA instead uses Bayes' theorem and makes assumptions on $X_1, \dots, X_p \mid Y$ (S3.48–49):

- prior probability $\pi_k = \Pr(Y = k)$; density function of X in class k: $f_k(x) = \Pr(X = x \mid Y = k)$.

$$\underbrace{\Pr(Y = k \mid X = x)}_{\text{posterior probability}} = \frac{\pi_k f_k(x)}{\sum_{j=1}^{K} \pi_j f_j(x)}$$

The denominator is the same for every $k \Rightarrow$ classify to the k with the largest $\pi_k f_k(x)$.

Discriminant analysis in one sentence: assume f_k is the **normal** density, i.e. $(X_1, \dots, X_p) \mid Y$ is normally distributed (S3.68).

Example (mine): $\pi_1 = 0.3$, $\pi_2 = 0.7$, $f_1(x_0) = 0.4$, $f_2(x_0) = 0.1 \Rightarrow \Pr(Y = 1 \mid x_0) = \frac{0.12}{0.12+0.07} = 0.63 \Rightarrow$ class 1, despite the smaller prior.

3.7 LDA with one predictor ($p = 1$)

LDA, p = 1: model, discriminant function, estimates

Assume $X \mid Y = k \sim N(\mu_k, \sigma^2)$ — **different mean, same variance** (S3.69):

$$f_k(x) = \frac{1}{\sqrt{2\pi}\,\sigma} \exp\Big(-\frac{1}{2\sigma^2}(x - \mu_k)^2\Big).$$

Assign x to the class whose discriminant function

$$\delta_k(x) = x \cdot \frac{\mu_k}{\sigma^2} - \frac{\mu_k^2}{2\sigma^2} + \log(\pi_k) \quad \text{is largest}$$

(S3.70–76 print the middle term as μ_k^2/σ^2; the correct form, as in ISLR, has $2\sigma^2$.)

$\mu_1, \dots, \mu_K, \pi_1, \dots, \pi_K, \sigma^2$ are unknown $\Rightarrow$ **estimate them** (S3.71, S3.76):

$$\hat{\pi}_k = \frac{n_k}{n}, \quad \hat{\mu}_k = \frac{1}{n_k} \sum_{i:y_i=k} x_i, \quad \hat{\sigma}^2 = \frac{1}{n-K} \sum_{k=1}^{K} \sum_{i:y_i=k} (x_i - \hat{\mu}_k)^2$$

(n_k = # training obs. in class k; $\hat{\sigma}^2$ is the pooled variance.) The LDA classifier plugs these into $\hat{\delta}_k(x) = x\,\hat{\mu}_k/\hat{\sigma}^2 - \hat{\mu}_k^2/(2\hat{\sigma}^2) + \log \hat{\pi}_k$, which is **linear in** x — hence "linear" discriminant analysis.

Decision boundary ($p = 1, K = 2$)

(S3.77)

- decision boundary: $\delta_1(x) = \delta_2(x)$. With equal priors $\pi_1 = \pi_2$: $x^* = \frac{\mu_1+\mu_2}{2}$ (midpoint of the means). Unequal priors (ISLR): $x^* = \frac{\mu_1+\mu_2}{2} + \frac{\sigma^2 \log(\pi_2/\pi_1)}{\mu_1-\mu_2}$ — the boundary moves toward the *less* likely class, enlarging the region of the more likely one.
- S3.77: left, densities $N(\mu_1, \sigma^2)$ (green) and $N(\mu_2, \sigma^2)$ (purple) cross at 0 = Bayes decision boundary (dashed); right, histograms of training samples with the **LDA decision boundary** (solid, slightly left of 0) — LDA estimates the Bayes boundary from data.
- Intuition (S3.52–75, gene-transcription/drug example): LDA builds a **new axis** that **maximises** $\frac{d^2}{s_1^2 + s_2^2}$ — squared distance between class means over the within-class "scatter" — rather than ignoring a

predictor (projecting on Gene X alone loses separation). For 3 classes: $\frac{d_1^2+d_2^2+d_3^2}{s_1^2+s_2^2+s_3^2}$ with d_k = distance from class centroid to the overall centroid; data are projected onto LD1, LD2 (S3.82–85).

3.8 LDA with several predictors ($p > 1$)

LDA, p > 1

Assume $(X_1, \dots, X_p) \mid Y = k \sim N(\mu_k, \Sigma)$ — **different mean vector, same covariance matrix** Σ ($p \times p$, shared by all classes) (S3.78) (multivariate normal distribution):

$$f_k(\mathbf{x}) = \frac{1}{(2\pi)^{p/2}|\Sigma|^{1/2}} \exp\Big(-\tfrac{1}{2}(\mathbf{x}-\mu_k)^T\Sigma^{-1}(\mathbf{x}-\mu_k)\Big)$$

The Bayes classifier assigns $\mathbf{x}$ to the class with the largest

$$\delta_k(\mathbf{x}) = \mathbf{x}^T\Sigma^{-1}\mu_k - \tfrac{1}{2}\mu_k^T\Sigma^{-1}\mu_k + \log \pi_k.$$

LDA plugs in $\hat{\mu}_k$, $\hat{\Sigma}$ (pooled), $\hat{\pi}_k$: $\hat{\delta}_k(\mathbf{x})$ is **linear in** $\mathbf{x}$ (only the term $\mathbf{x}^T\hat{\Sigma}^{-1}\hat{\mu}_k$ involves $\mathbf{x}$) (S3.79). Posterior probabilities: $\widehat{\Pr}(Y = k \mid \mathbf{x}) = e^{\hat{\delta}_k(\mathbf{x})} / \sum_j e^{\hat{\delta}_j(\mathbf{x})}$.

Boundary (S3.81): Bayes $= \{\mathbf{x} : \delta_k(\mathbf{x}) = \delta_j(\mathbf{x})\}$, LDA $= \{\mathbf{x} : \hat{\delta}_k(\mathbf{x}) = \hat{\delta}_j(\mathbf{x})\}$, $k \neq j \Rightarrow$ **linear boundary, i.e. a hyperplane** (a line when $p = 2$; with $K = 3$, three line pieces meeting at a point). In S3.81 the LDA (solid) and Bayes (dashed) boundaries nearly coincide.

3.9 Quadratic discriminant analysis (QDA)

QDA

Key in discriminant analysis: $(X_1, \dots, X_p) \mid Y$ is normal (S3.86). **LDA:** $N(\mu_k, \Sigma)$ — different mean, **same** covariance. **QDA:** $N(\mu_k, \Sigma_k)$ — different mean, **different** covariance (each class its own Σ_k). Assign $\mathbf{x}$ to the class with the largest (S3.87)

$$\begin{aligned}\delta_k(\mathbf{x}) = &-\tfrac{1}{2}\mathbf{x}^T\Sigma_k^{-1}\mathbf{x} + \mathbf{x}^T\Sigma_k^{-1}\mu_k \\ &- \tfrac{1}{2}\mu_k^T\Sigma_k^{-1}\mu_k - \tfrac{1}{2}\log|\Sigma_k| + \log \pi_k\end{aligned}$$

(QDA: hats on μ_k, Σ_k, π_k.) The class-specific term $-\frac{1}{2}\mathbf{x}^T\hat{\Sigma}_k^{-1}\mathbf{x}$ makes it **quadratic in** $\mathbf{x} \Rightarrow$ curved (quadratic) decision boundary.
Parameter count (covariances): LDA one Σ: $p(p+1)/2$; QDA: $K \cdot p(p+1)/2$ (e.g. $p = 50$: 1275 vs. $1275K$) (S3.91).

S3.88: LDA vs QDA boundaries (purple dashed = Bayes, black dotted = LDA, green = QDA)

- $\Sigma_1 = \Sigma_2$: the Bayes boundary is **linear**; LDA is close to it, QDA's curve deviates (extra variance, no bias gain).
- $\Sigma_1 \neq \Sigma_2$: the Bayes boundary is **curved**; QDA tracks it, LDA's straight line cannot (bias).

3.10 Comparing classifiers

KNN vs logistic vs LDA vs QDA vs naive Bayes

Method	Param.?	Boundary	Models / assumes
KNN	no	nonlinear (any shape)	majority vote of the k nearest training points; no assumption on f
Logistic	yes	linear	$\Pr(Y = 1 \mid X)$ directly via the logit (discriminative model); fit by MLE
LDA	yes	linear	$X \mid Y = k \sim N(\mu_k, \Sigma)$, common Σ (generative model)
QDA	yes	quadratic	$X \mid Y = k \sim N(\mu_k, \Sigma_k)$, class-specific Σ_k
Naive Bayes	yes	quadratic, no cross terms	predictors independent within each class

Model flexibility (S3.89): KNN > QDA > logistic regression = LDA (choose via the bias-variance tradeoff); interpretability runs the other way. KNN is non-parametric model; logistic, LDA, QDA are parametric models.

Logistic regression vs LDA

(S3.90)

- **Both produce linear decision boundaries**; the coefficient-estimation procedures differ (logistic: maximum likelihood; LDA: plug in estimated normal means/variance/priors).
- LDA assumes a **normal** distribution of the predictors given the response class.
- Logistic regression learns $\Pr(Y = k \mid X)$ **directly**: discriminative model (discriminative learning). LDA learns $\Pr(Y = k \mid X)$ by modelling $\Pr(Y = k, X) = \pi_k f_k(x)$: generative model (generative learning).
- Rule of thumb: normality roughly holds / small n / well-separated classes / $K > 2 \Rightarrow$ LDA; normality doubtful $\Rightarrow$ logistic regression is more robust.

LDA vs QDA (bias–variance)

(S3.91)

- LDA is a **special case** of QDA (all $\Sigma_k = \Sigma$). QDA is **more flexible**, but has **more parameters** to estimate (the Σ_k's) $\Rightarrow$ lower bias, higher variance.
- Prefer **LDA**: few training observations (small n) relative to p, or class covariances similar — reducing variance is crucial. Prefer **QDA**: large n (variance less of a concern) or the common-covariance assumption is clearly untenable.
- In practice we often do not know which assumptions are reasonable — let the data decide via cross-validation (S3.93).

PE Q20 (LDA vs QDA)

50,000 observations, two predictors, clear evidence that the covariance structures differ substantially across outcome levels. Which has greater potential to capture the true decision boundary? (a) LDA (b) QDA (c) equal (d) neither (both parametric).

Answer. (b) QDA. LDA assumes a common covariance, so it would not do well; QDA estimates more parameters, but $p = 2$ is small and n is huge, so the extra variance is negligible (parametric is fine) (PE Q20).

LDA/QDA traps

- **QDA only helps if the class covariances really differ**; if $\Sigma_1 \approx \Sigma_2$ it just adds variance (more parameters, no bias reduction) and LDA usually wins on test data.
- "Linear" in LDA refers to the **decision boundary** (linear $\hat{\delta}_k$), not to linear regression. LDA and logistic regression share the boundary shape but are estimated differently.
- LDA/QDA assume **normal** predictors within each class — categorical/-dummy predictors violate this (they still run, but the assumption is wrong).
- Higher training error rate is not the criterion: the most flexible model (QDA/KNN) tends to win on *training* data and can lose on testing data (overfitting).

3.11 Naive Bayes (Lab 3)

Naive Bayes

- Same Bayes'-theorem idea, $\Pr(Y = k \mid x) \propto \pi_k f_k(x)$, but instead of a multivariate normal, assume the p predictors are **independent within each class**: $f_k(x) = f_{k1}(x_1) \times f_{k2}(x_2) \times \cdots \times f_{kp}(x_p)$.
- With `usekernel = FALSE` each f_{kj} is a **normal (Gaussian)** density (`TRUE` $\Rightarrow$ a kernel density estimate) (Lab 3). Gaussian naive Bayes $\approx$ QDA with *diagonal* Σ_k (no within-class correlations).
- "Naive" = the independence assumption (usually false); it slashes the number of parameters $\Rightarrow$ higher bias, much lower variance; useful when p is large relative to n.

LDA / QDA / naive Bayes in tidymodels (Lab 3)

Needs `library(discrim)` (and `klaR` for naive Bayes). The recipe/workflow is reused; only the model spec changes:

```
lda_mod <- discrim_linear() %>%
  set_mode("classification") %>% set_engine("MASS")
qda_mod <- discrim_quad() %>%
  set_mode("classification") %>% set_engine("MASS")
nb_mod <- naive_Bayes() %>%
  set_mode("classification") %>% set_engine("klaR") %>%
  set_args(usekernel = FALSE)
smarketqda_wkflow <- workflow() %>% add_model(qda_mod) %>%
```

```
  add_recipe(smarket_recipe)
smarketqda_fit <- fit(smarketqda_wkflow, smarket_train)
augment(smarketqda_fit, new_data = smarket_train) %>%
  accuracy(truth = Direction, estimate = .pred_class)
```

Spelling: `naive_Bayes()` (capital B), `discrim_linear()`, `discrim_quad()`; engines `"MASS"` (LDA, QDA), `"klaR"` (naive Bayes).

Lab 3 training accuracies (picked by training accuracy only because CV had not been taught):

Model	Smarket	OJ
Logistic regression	0.525	**0.833**
LDA	0.524	0.828
QDA	**0.548**	0.797
Naive Bayes	0.529	0.778

Test set: Smarket QDA accuracy **0.487** (sensitivity 0.254, specificity 0.703) — worse than a coin flip despite the best training accuracy; OJ logistic accuracy 0.832 (sens. 0.908, spec. 0.714). Lesson: choosing a model by training error is **not** a good idea in general; use cross-validation. Test metrics via `metric_set()(accuracy, sensitivity, specificity)` and `roc_curve()(Direction, .pred_Down)` (see model evaluation).

3.12 HW3: four classifiers on the Titanic data

HW3 pipeline (Q1, Q4–Q10)

(HW3) `titanic` (891 passengers; `survived` Yes $\approx$ 38%; `age` has 177 NAs). Make `survived` and `pclass` factors ("Yes" first), split with `strata = survived` (stratified sampling keeps the ≈38/62 class ratio in both sets — class imbalance).

```
titanic <- read_csv("data/titanic.csv") %>%
  mutate(survived = factor(survived, levels = c("Yes", "No")),
         pclass = factor(pclass))
set.seed(131)
t_split <- initial_split(titanic, prop = 0.7, strata = survived)
t_train <- training(t_split); t_test <- testing(t_split)
t_recipe <- recipe(survived ~ pclass + sex + age + sib_sp +
    ↪ parch + fare,
                   data = t_train) %>%
  step_impute_linear(age, impute_with = imp_vars(sib_sp, parch,
    ↪ fare)) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_interact(terms = ~ starts_with("sex"):fare + age:fare)
log_spec <- logistic_reg() %>% set_engine("glm") %>% set_mode("
    ↪ classification")
lda_spec <- discrim_linear() %>% set_engine("MASS") %>% set_mode
    ↪ ("classification")
qda_spec <- discrim_quad() %>% set_engine("MASS") %>% set_mode("
    ↪ classification")
knn_spec <- nearest_neighbor(neighbors = 10) %>%
  set_engine("kknn") %>% set_mode("classification")
log_fit <- workflow() %>% add_model(log_spec) %>% add_recipe(t_
    ↪ recipe) %>%
  fit(t_train)                                  # repeat for lda
    ↪ /qda/knn
predict(log_fit, new_data = t_train, type = "prob") %>%
  bind_cols(t_train %>% select(survived)) %>%
  roc_auc(truth = survived, .pred_Yes)
augment(log_fit, new_data = t_test) %>%
  conf_mat(truth = survived, estimate = .pred_class) %>%
    ↪ autoplot(type = "heatmap")
augment(log_fit, new_data = t_test) %>% roc_curve(survived, .
    ↪ pred_Yes) %>% autoplot()
```

- Order of steps: **impute** → **dummy** → **interact**. After `step_dummy()` sex becomes `sex_male`, so the interaction term must use `starts_with()("sex")`. (`imp_vars()` lists the predictors of the linear imputation model; use complete columns.)
- AUC needs **probabilities** (`type = "prob"`, column `.pred_Yes`), not `.pred_class`. Compare **training vs testing** AUC: a big drop (typical for KNN with small k, and for the more flexible QDA) signals overfitting.
- HW3 Q11–12 (231): see the logit-inverse box above.

4 Model evaluation: metrics, thresholds, ROC/AUC

4.1 Train vs. test: where performance is measured

Training vs. test error (S1.57–60, S1.89–91)

- Regression. Training MSE$= \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \hat{f}(\mathbf{x}_i)\right)^2$ on the n training points; test MSE$= \frac{1}{m}\sum_{i=1}^{m}\left(\tilde{y}_i - \hat{f}(\tilde{\mathbf{x}}_i)\right)^2$ on m points *not seen or used* to fit. In both, $\hat{f}$ is obtained on the training set.
- Classification. error rate $= \frac{1}{n}\sum_{i=1}^{n} I(y_i \neq \hat{y}_i)$ = fraction misclassified; accuracy $= 1-$ error rate. Toy example (S1.90): 2 of 5 wrong $\Rightarrow$ error rate 0.40, accuracy 0.60.
- Goal: the $\hat{f}$ with the smallest *test* MSE / error rate. "Can we just select $\hat{f}$ that minimizes training MSE? NO!" Lowest training MSE $\neq$ lowest test MSE (S1.60).

Why training error is optimistic. $\hat{f}$ was chosen to fit exactly those training points (noise included), so it looks better on them than on new data. As flexibility (complexity) grows, training MSE/error rate keeps falling, but test error is U-shaped (S1.62, S1.101): higher complexity $\rightarrow$ lower training error, but higher complexity $\not\rightarrow$ lower test error (see bias-variance tradeoff).

Overfitting / underfitting signatures

- overfitting: training performance $\gg$ test performance (train accuracy $\gg$ test accuracy; train RMSE $\ll$ test RMSE). Extreme case: 100 % training accuracy.
- underfitting (too rigid, high bias): train $\approx$ test, but both worse than a more flexible model.
- Compare models on **test** performance (or, later, cross-validation). Lab 3 picks a model by *training* accuracy only because CV had not been covered: "not a good idea in general" (Lab 3).
- A model with lower training error is *not* always preferable (PE Q4c).

Worked: PE Q3 — which model?

Model	Train acc.	Test acc.	Verdict
A	71%	70%	no overfit, but weaker
B	86%	**82%**	**choose** (best test)
C	100%	68%	overfitting (gap 32 pts)

Official solution. Prefer B (highest *testing* accuracy). C overfits; B shows some overfitting too, but less. (PE Q3)

Worked: Lab 2 `diamonds` (predict `price`)

Model	train RMSE	test RMSE	test R^2
linear reg. (`lm`)	1126	1146	0.918
KNN, $k = 5$	386	**711**	**0.968**
KNN, $k = 2$	152	851	0.955

Best on *test*: KNN $k = 5$. $k = 2$ has the best training RMSE ($R^2_{\text{train}} = 0.999$) but the largest train–test gap $\Rightarrow$ overfitting. The linear model has train $\approx$ test but both high: a linear form is too rigid for f here (Lab 2).

4.2 Regression metrics

Formulas and direction (S1.57, S2.30, Lab 2)

Metric	Definition	Better
MSE	$\frac{1}{n}\sum(y_i - \hat{y}_i)^2$ (units of Y^2)	lower
RMSE	$\sqrt{\text{MSE}}$ (units of Y)	lower
MAE	$\frac{1}{n}\sum\lvert y_i - \hat{y}_i\rvert$ (units of Y)	lower
R^2	$1 - \frac{\text{SSR}}{\text{SST}} = 1 - \frac{\sum(y_i-\hat{y}_i)^2}{\sum(y_i-\bar{y})^2}$	higher
RSE	$\sqrt{\frac{1}{n-2}\text{SSR}}$ (simple LR)	lower

- RMSE $\approx$ size of a typical prediction error in Y's units (e.g. \$711 for `price`). Squaring makes RMSE punish large errors more than MAE; always MAE $\leq$ RMSE.
- R-squared $\in [0, 1]$ = proportion of the variability in Y that can be explained using X (S2.30); RSE = an absolute measure of lack of fit.
- `rsq()` in `yardstick` is the squared correlation of `truth` and `.pred`, so it stays in $[0, 1]$ on test data too.

RSE and R^2 favor flexible models (S3.30)

"Both RSE and R^2 favor **flexible** methods, which may **overfit** the data!" Training R^2 never decreases when you add predictors ⇒ judge models on *test* RMSE/R^2.

```
lm_test <- predict(lm_fit, new_data = diamonds_test %>% select(-
    ↪ price))
lm_test <- bind_cols(lm_test, diamonds_test %>% select(price)) # .
    ↪ pred + truth
rmse(lm_test, truth = price, estimate = .pred)
diamond_metrics <- metric_set(rmse, rsq, mae)
diamond_metrics(lm_test, truth = price, estimate = .pred)
```

```
# A tibble: 3 x 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 rmse    standard    1146.
2 rsq     standard       0.918
3 mae     standard     751.
```

Output is a tibble: the number is in `.estimate`; `.estimator` is `standard` for regression, `binary` for a 2-class outcome. Extract with `acc$.estimate` (Lab 3).

4.3 From probabilities to classes: the threshold

Thresholding. logistic regression gives $\widehat{\Pr}(Y = 1 \mid X)$, e.g. $\widehat{\Pr}(\text{default} = \text{TRUE} \mid \text{balance} = 1500, \text{income} = 40000, \text{student} = \text{FALSE}) = 0.105$ (S3.32). Predict the class by *thresholding the predicted probability*: default = TRUE if $\hat{p} > 0.5$; a "super conservative" credit-card company might use $\hat{p} > 0.1$ (S3.33). A predicted probability of 0.78 means "the model assigns probability 0.78 of belonging to that class", *not* "78% accurate" (PE Q14).

Moving the threshold t (predict positive if $\hat{p} \geq t$)

	Lower t	Raise t
# predicted positives	more	fewer
TP, FP	↑ (or same)	↓ (or same)
FN, TN	↓ (or same)	↑ (or same)
TPR (sensitivity)	↑	↓
FPR ($1-$specificity)	↑	↓
specificity	↓	↑
ROC point moves	up/right	down/left

TPR and FPR move *together*: you trade false negatives for false positives. $t = 0$ (or $-\infty$): everything positive, TPR = FPR = 1. $t = 1$ (or ∞): nothing positive, TPR = FPR = 0.

Worked: PE Q15 — four borrowers, $\hat{p}$ = A .31, B .48, C .63, D .81

(a) $t = 0.50$: predicted default = **C, D**. **(b)** $t = 0.70$: only **D**. **(c)** Raising the threshold ⇒ the number of positive classifications tends to *decrease*. (PE Q15)

4.4 The confusion matrix and its rates

For a binary classifier, use a confusion matrix. Course layout (S3.34) (rows = *true* class, columns = *predicted* class, class 0 first):

		Predicted class $\hat{Y} = 0$	$\hat{Y} = 1$	row total
True class	$Y = 0$	TN	FP (Type I)	N = TN+FP
	$Y = 1$	FN (Type II)	TP	P = FN+TP

true positive (TP): truly 1, predicted 1. false positive (FP): truly 0, predicted 1 = Type I error. false negative (FN): truly 1, predicted 0 = Type II error. true negative (TN): truly 0, predicted 0. Column totals = number *predicted* 0 / 1; row totals = number *actually* 0 / 1.

Rates (S3.34, S3.36–37)

- Accuracy = $\dfrac{TP+TN}{TP+TN+FP+FN}$; error rate = $\dfrac{FP+FN}{\text{total}}$ = $1-$ accuracy.
- TPR = $\dfrac{TP}{TP+FN} = \dfrac{TP}{P}$ = sensitivity = recall = power = $1-$ Type II error ("probability of detection").
- FPR = $\dfrac{FP}{FP+TN} = \dfrac{FP}{N}$ = Type I error = $1-$ specificity ("fall-out", "false alarm").
- specificity = TNR = $\dfrac{TN}{TN+FP}$ = $1-$ FPR. FNR (miss rate) = $\dfrac{FN}{TP+FN}$ = $1-$ TPR.
- precision = PPV = $\dfrac{TP}{TP+FP}$ (of those *predicted* positive, fraction truly positive); negative predictive value (NPV) = $\dfrac{TN}{TN+FN}$.
- F1 score = $\dfrac{2}{1/\text{recall} + 1/\text{precision}}$; prevalence = P/total (S3.36).

Memory hook: TPR, FNR divide by the *actual-positive* total; FPR, specificity divide by the *actual-negative* total; precision, NPV divide by a *predicted* total.

Venn picture (S3.38): sensitivity = "how many relevant items are selected?" (sick people correctly identified as having the condition); specificity = how many truly negative elements are identified as negative (healthy people identified as not having the condition).

Multiclass (S3.35): a $K \times K$ table; correct predictions on the diagonal. Slide example (rows = predicted, cols = true; Apple/Orange/Mango): diagonal $7 + 2 + 1 = 10$ of 36 ⇒ accuracy = $10/36 \approx 0.28$.

Worked: PE Q17

	Actually Pos.	Actually Neg.
Predicted Pos.	TP = 72	FP = 18
Predicted Neg.	FN = 28	TN = 182

(Rows are *predicted* here — the opposite of S3.34. Find cells by meaning, not position.)
Accuracy = $\frac{72+182}{300} = \frac{254}{300} = 0.8467$. TPR = $\frac{72}{72+28} = \frac{72}{100} = 0.72$. FPR = $\frac{18}{18+182} = \frac{18}{200} = 0.09$.
Extra: specificity = $182/200 = 0.91$; precision = $72/90 = 0.80$; error rate = $46/300 = 0.153$. (PE Q17)

Worked: reading yardstick output (Lab 3, `Smarket` QDA, test set)

```
          Truth
Prediction Down  Up
      Down   46  58
      Up    135 137
```

`Direction` levels are `Down`, `Up` ⇒ **Down is the event**: TP = 46, FP = 58, FN = 135, TN = 137. Sensitivity = $46/181 = 0.254$, specificity = $137/195 = 0.703$, accuracy = $183/376 = 0.487$ — exactly the printed `metric_set(accuracy, sensitivity, specificity)` output. Training accuracy was 0.548, so the test accuracy (below 0.5) is the honest number (Lab 3).

4.5 Class imbalance: why accuracy can mislead

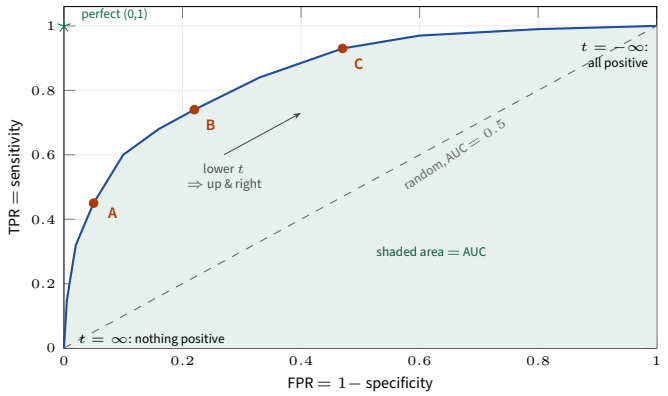
- With class imbalance, the trivial "always predict the majority class" rule already gets accuracy = majority proportion. Compare accuracy to that baseline.
- **Titanic Model B** (`survived` ~ `sib_sp` only) *never predicts Yes*: test matrix TP = 0, FP = 0, FN = 103, TN = 165 ⇒ accuracy $165/268 \approx 0.61$ "relatively high", yet TPR = 0 and AUC = 0.496 (a coin flip) (ROC).
- Check sensitivity and specificity (or AUC), not just accuracy. Lab 3: `Smarket` is fairly balanced (602 Down / 648 Up); `OJ` is unbalanced (653 CH / 417 MM) (Lab 3).
- Split with stratified sampling on the outcome, `initial_split(data, prop = 0.7, strata = survived)`, so training and test sets keep the same class proportions (ROC, Lab 3). (Up-/down-sampling: a later lab.)

4.6 ROC curve and AUC

ROC curve (S3.39–40, ROC)

- A popular graphic for displaying the **2 types of error** for **all threshold values**: TPR (sensitivity, y-axis) vs. FPR ($1-$specificity, x-axis).
- **Construction:** for every threshold t (in practice each distinct $\hat{p}$, plus $\pm\infty$): classify "Yes" if $\hat{p} \geq t$, build the confusion matrix, compute (FPR, TPR), plot the point; connect the points. Each point on the curve = one threshold.

- Ends: $t = \infty \to (0,0)$; $t = -\infty \to (1,1)$. Lowering t moves up and to the right.
- **Perfect classifier** = point (0, 1), top-left corner. **Random classifier** = the diagonal TPR = FPR. Curves toward the top-left are better (S3.40).
- Useful for *comparing* classifiers (e.g. logistic regression vs. KNN); methods perform differently in different parts of the curve, i.e. at different thresholds, so curves can cross (S3.39–40).



Sketch modelled on PE Q19: A ≈ (0.05, 0.45), B ≈ (0.22, 0.74), C ≈ (0.47, 0.93). A = high threshold, C = low threshold.

AUC (S3.39–40, ROC)

- AUC = area under the ROC curve; measures the *overall* performance of a classifier, **summarized over all possible thresholds**. Higher AUC, the better.
- 1 = perfect; 0.5 = random guessing / coin flip (the diagonal); < 0.5 = worse than random. Slide example: Model A 0.923 > B 0.794 > C 0.588 (S3.40).
- It measures the model's **ability to discriminate between classes**: high when positives get higher $\hat{p}$ than negatives (ROC).
- **Threshold-free:** the curve is built from *all* thresholds, so changing the cutoff used for `.pred_class` (0.5 → 0.3) does *not* change AUC; it only picks a different point on the same curve (PE Q18).
- AUC = 1 is suspicious: usually the outcome (or a linear combination of it) slipped in as a predictor (target leakage) (ROC).

Two uses of ROC curves (ROC): (1) evaluating and comparing models (prefer AUC closer to 1; overlay curves); (2) selecting the optimal threshold for a chosen model.

Worked: PE Q18 and Q19

Q18. "I changed my threshold from 0.50 to 0.30, so the ROC AUC will change." **False**: the ROC curve plots TPR and FPR *across* thresholds, so the chosen threshold has no impact on AUC.

Q19. Screening for a very dangerous disease; a false negative is much worse than a false positive. Choose **Point C**: maximize TPR (minimize FNR) regardless of FPR — even though FPR ≈ 50%. In general: FN costly ⇒ *lower* the threshold (point up/right); FP costly ⇒ raise it (down/left). (PE Q18–19)

4.7 Reading "ROC Curves Deep Dive": Titanic Model A vs. B

- Data `titanic` (Kaggle; 891 rows): `survived = factor(survived, levels = c("Yes", "No"))` ⇒ **"Yes" is the first level = the event**; also adds a dummy `coin_flip` column (445 zeros then 446 ones). Split 70/30 with `strata = survived` because the outcome is imbalanced (more died than survived).
- **Recipe A**: `pclass + sex + age + sib_sp + parch + fare`, `step_impute_linear(age)`, `step_dummy()`, `step_interact()`. **Recipe B**: `survived ~ sib_sp` only (intentionally bad). Both: `logistic_reg()` with engine `"glm"`.
- Training accuracy: A 0.807, B 0.616.

Model A (test set: 103 Yes, 165 No).

- Default $t = 0.5$: TP 80, FP 20, FN 23, TN 145 ⇒ TPR $= 80/103 = 0.777$, FPR $= 20/165 = 0.121$: one point near the top-left of the curve.
- $t \approx 0.0013$ (smallest $\hat{p}$): essentially everything predicted Yes: TP 103, FP 163, FN 0, TN 2 ⇒ TPR $= 1$, FPR $= 163/165 \approx 0.99$: the (1, 1) corner.
- `roc_curve()` table has 254 rows (one per threshold); predicted-probability histograms are well separated (No near 0, Yes mostly > 0.5) ⇒ good discrimination.

Model B.

- ROC curve hugs the diagonal; `roc_auc()` $= 0.496$: "just about as bad as …a coin flip".
- At $t = 0.5$ it never predicts Yes ⇒ point (0, 0).
- Only 9 thresholds: `sib_sp` takes few values, so there are few distinct $\hat{p}$; max $\hat{p} = 0.408$. At $t \approx 0.408$: TP 71, FP 118, FN 32, TN 47 ⇒ TPR $= 71/103 = 0.689$, FPR $= 118/165 = 0.715$ (a point just under the diagonal).
- $\hat{p}$ distributions of Yes and No overlap almost completely: the model cannot discriminate.

4.8 yardstick code and output

```
log_fit_a <- fit(log_wkflow_a, titanic_train)      # fit on
    ↪ TRAINING
res <- augment(log_fit_a, new_data = titanic_test)
# adds .pred_class, .pred_Yes, .pred_No to the test data
predict(log_fit_a, new_data = titanic_test, type = "prob") # .pred
    ↪ _Yes .pred_No
predict(log_fit_a, new_data = titanic_test)        # .pred_class
    ↪ only
res %>% conf_mat(truth = survived, estimate = .pred_class)
res %>% conf_mat(truth = survived, estimate = .pred_class) %>%
  autoplot(type = "heatmap")
res %>% accuracy(truth = survived, estimate = .pred_class)
m <- metric_set(accuracy, sensitivity, specificity)
res %>% m(truth = survived, estimate = .pred_class)
res %>% roc_auc(survived, .pred_Yes)    # prob. column of FIRST
    ↪ level
res %>% roc_curve(survived, .pred_Yes) %>% autoplot()
```

Real outputs (Lab 3 OJ logistic regression on test, event = CH; Titanic Model B):

```
          Truth                              # conf_mat: rows =
    ↪ Prediction,
Prediction  CH  MM                           #            cols =
    ↪ Truth
        CH 178  36      <- TP  FP
        MM  18  90      <- FN  TN
# A tibble: 3 x 3                            # metric_set(
    ↪ accuracy,
  .metric     .estimator .estimate           #   sensitivity,
    ↪ specificity)
1 accuracy    binary         0.832           # 268/322
2 sensitivity binary         0.908           # 178/196
3 specificity binary         0.714           # 90/126
# A tibble: 1 x 3                            # roc_auc(survived,
    ↪ .pred_Yes)
  .metric .estimator .estimate
1 roc_auc binary         0.496
# A tibble: 9 x 3                            # roc_curve(survived
    ↪ , .pred_Yes)
  .threshold specificity sensitivity
1    -Inf           0           1            # everything "Yes":
    ↪ (1,1)
2      0.113        0           1
...
8      0.408        0.285       0.689        # FPR = 1 - 0.285 =
    ↪ 0.715
9     Inf           1           0            # nothing "Yes":
    ↪ (0,0)
```

`autoplot()` on a `roc_curve()` tibble plots `sensitivity` vs. `1 - specificity` with a dotted diagonal.

yardstick traps / spot-the-bug

- **event level = FIRST factor level.** `sensitivity()`, `specificity()`, `roc_auc()`, `roc_curve()` treat `levels(y)[1]` as "positive". `factor()` orders levels alphabetically by default ("No" before "Yes"), hence Titanic's `levels = c("Yes", "No")`. `Smarket`: first level Down ⇒ `roc_curve(Direction, .pred_Down)`; OJ: `.pred_CH` (Lab 3).
- Pass the probability column *of the first level*. Using the other class's column (e.g. `.pred_No` when Yes is first) mirrors the curve: AUC becomes 1−AUC.

- `roc_curve()` / `roc_auc()` need **probabilities** (`.pred_Yes`), not `.pred_class`; given classes they throw an error (ROC). `accuracy()` and `conf_mat()` use `truth` = the factor outcome and `estimate` = `.pred_class`.
- yardstick's `conf_mat()` prints **Prediction as rows, Truth as columns**, event level first — the transpose (and reorder) of the slide layout S3.34. Always read the labels.
- Evaluate on the test set with the model fit on the training set: `augment(fit_on_train, new_data = test)`. (The reading's code re-fits the workflow on `titanic_test` before `augment()`; do not copy that.) Training-set metrics are optimistic.
- `augment()` adds `.pred_class` and one `.pred_<level>` per class (classification) or `.pred` (regression); `predict()` returns only predictions, so you `bind_cols()` the truth.

5 R code reference: tidyverse + tidymodels

Packages and set-up (Lab 1–3, S1.27–28)

`tidyverse` = data handling + plots (`dplyr`, `ggplot2`, `readr`, `tibble`, `forcats`, `tidyr`). `tidymodels` = modelling (`rsample` split, `recipes` preprocessing, `parsnip` model specs, `workflows`, `yardstick` metrics, `broom` `tidy()`, `tune`). Workflow picture (S1.28): Import → Tidy → Transform ↔ Visualize ↔ Model (tidymodels) → Share.

```
library(tidyverse); library(tidymodels)
library(corrplot); library(ggthemes)     # EDA extras
library(kknn)                  # engine for nearest_neighbor()
library(discrim); library(klaR) # LDA/QDA/naive Bayes specs
library(naniar); library(corrr) # vis_miss(); correlate()
tidymodels_prefer()  # tidymodels wins name conflicts
```

- `tidymodels_prefer()`: when two loaded packages export the same name (e.g. `MASS::select()` vs `dplyr::select()`, `MASS` is loaded with `klaR`), use the tidymodels/tidyverse version.
- Install once *outside* the .Rmd (`install.packages("kknn")`); an `install.packages()` line inside the .Rmd stops it knitting (Lab 1). `?fun` / `help(fun)` = documentation.
- Pipes: `%>%` (magrittr, Ctrl/Cmd+Shift+M) and native `|>` do the same here: `x %>% f(y)` = `f(x, y)`. `<-` = assignment ("gets").

5.1 The pipeline: split → recipe → spec → workflow → fit → predict → metrics

The 7 steps (memorise the order)

1. **Seed + split**: `set.seed()`, `initial_split()(data, prop, strata)`, `training()`, `testing()`.
2. **Recipe** (on *training* data): `recipe()(y ~ ., data = train)` + `step_*()`. Only a *plan*; nothing is computed.
3. **Model spec**: model function + `set_engine()` + `set_mode()`. No data yet.
4. **Workflow**: `workflow()` %>% `add_model()` %>% `add_recipe()`.
5. **Fit**: `fit()(wflow, train)` — preps the recipe on train, bakes train, fits the engine.
6. **Predict**: `predict()(fit, new_data = test)` (only predictions) or `augment()(fit, new_data = test)` (data + predictions).
7. **Evaluate**: `yardstick` `metric(data, truth = y, estimate = .pred_class / .pred)` or `metric_set()`.

Regression template (`diamonds` Lab 2, `abalone` HW2):

```
set.seed(3435)
d_split <- initial_split(diamonds, prop = 0.80, strata = price)
d_train <- training(d_split); d_test <- testing(d_split)

d_rec <- recipe(price ~ ., data = d_train) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_normalize(all_numeric_predictors())

lm_mod <- linear_reg() %>% set_engine("lm")
knn_mod <- nearest_neighbor(neighbors = 7) %>%
  set_engine("kknn") %>% set_mode("regression")

lm_wf <- workflow() %>% add_model(lm_mod) %>%
  add_recipe(d_rec)

lm_fit <- fit(lm_wf, data = d_train)

lm_fit %>% extract_fit_parsnip() %>% tidy()  # coef table
res <- augment(lm_fit, new_data = d_test)  # test + .pred
reg_metrics <- metric_set(rmse, rsq, mae)
reg_metrics(res, truth = price, estimate = .pred)
```

Classification template (`titanic` HW3, OJ/`Smarket` Lab 3):

```
titanic <- read_csv("data/titanic.csv") %>%
  mutate(survived = factor(survived, levels = c("Yes", "No")),
         pclass = factor(pclass))   # outcome MUST be a factor
set.seed(131)
t_split <- initial_split(titanic, prop = 0.7, strata = survived)
t_train <- training(t_split); t_test <- testing(t_split)
t_rec <- recipe(survived ~ pclass + sex + age + sib_sp +
                parch + fare, data = t_train) %>%
  step_impute_linear(age, impute_with = imp_vars(sib_sp, parch,
    ↪ fare)) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_interact(terms = ~ starts_with("sex"):fare + age:fare)
log_mod <- logistic_reg() %>% set_engine("glm") %>%
  set_mode("classification")
log_fit <- workflow() %>% add_model(log_mod) %>%
  add_recipe(t_rec) %>% fit(t_train)
res <- augment(log_fit, new_data = t_test)
# res = t_test + .pred_class + .pred_Yes + .pred_No
res %>% conf_mat(truth = survived, estimate = .pred_class)
res %>% accuracy(truth = survived, estimate = .pred_class)
res %>% roc_auc(truth = survived, .pred_Yes)
res %>% roc_curve(truth = survived, .pred_Yes) %>% autoplot()
```

To swap models, change only the spec (step 3) and `add_model()`; the recipe is reused (Lab 3). Without a recipe use `add_formula(y ~ x1 + x2)`.

5.2 dplyr / tidyverse verbs

Every verb takes a data frame first and returns a new data frame (tibble); nothing changes unless you assign with `<-` (Lab 1).

Function	Does / example
`filter()`	keep *rows*: `filter(class == "compact" & manufacturer == "audi")`; `==`, `!=`, `>`, `&` and, `|` or, `!` not, `%in%c("IAH","HOU")`
`select()`	keep *columns*: `select(year, hwy, class)`; ranges `year:class`; drop `-(year:class)`, `-price`; helpers `starts_with()`, `everything()`, `where(is.numeric)`
`mutate()`	add/modify columns (added at the end): `mutate(age = rings + 1.5)`, `mutate(after_2k = if_else(year <= 2000, 0, 1))`
`transmute()`	like `mutate()` but keeps only the new columns
`summarise()`	collapse to one row (per group): `summarise(avg_hwy = mean(hwy), count = n())`
`group_by()`	following `summarise`/`mutate`/`filter` work per group; `ungroup()` undoes
`arrange()`	sort rows ascending; `arrange(desc(x))` or `arrange(-x)` descending
`count()`	`count(manufacturer, sort = TRUE)` = `group_by` + `summarise(n = n())`
`drop_na()`	delete rows with any NA (`airquality`: 153 → 111 rows)
`head()`, `glimpse()`, `dim()`, `summary()`	first 6 rows; column types; (rows, cols); summary stats
`factor()`, `levels()`	make categorical; `factor(x, levels = c("Yes","No"))` sets level order
`bind_cols()` / `bind_rows()`	glue side by side (predictions + truth) / stack tables
`table()`	class counts: `Smarket %>% select(Direction) %>% table()` → Down 602, Up 648

```
mpg %>%
  group_by(manufacturer) %>%
  summarise(avg_hwy = mean(hwy), count = n()) %>%
  filter(count >= 9) %>%
  arrange(avg_hwy)          # ascending; top 10 makers
airquality %>% summarise(mean(Ozone))              # NA!
airquality %>% summarise(mean(Ozone, na.rm = TRUE)) # number
abalone %>% summarise(n = n(), mean = mean(age), median = median(
    ↪ age),
  sd = sd(age), q1 = quantile(age, 0.25))   # HW2 Q1
```

5.3 ggplot2 for EDA

Grammar: `ggplot()(data, aes()(x, y, fill, color))` + `geom_*()` + `labs()` + `theme_bw()`. Layers are joined with +, *not* %>% (but you may pipe the data *into* `ggplot()`). Map a variable inside `aes(fill = sex)`; a constant goes outside (`geom_point(alpha = 0.2)`). Which plot answers

which question: see the EDA table in *Linear models*.

```
ggplot(mpg, aes(x = hwy)) + geom_histogram(binwidth = 2)
ggplot(mpg, aes(x = hwy, y = cty)) + geom_point() +
  geom_smooth(method = "lm")          # fitted line
ggplot(mpg, aes(x = factor(cyl), y = hwy)) +
  geom_boxplot() + geom_jitter(alpha = 0.3)
ggplot(Smarket, aes(x = Direction)) + geom_bar() # class balance
# HW3 Q2: percent stacked bar (each bar sums to 100%)
ggplot(t_train, aes(x = survived, fill = sex)) +
  geom_bar(position = "fill") + labs(y = "proportion")
# correlation matrix of numeric columns
t_train %>% select(where(is.numeric)) %>%
  cor(use = "complete.obs") %>%   # NAs (age) else NA
  corrplot(type = "lower", diag = FALSE, method = "number")
Smarket %>% select(-Direction) %>% correlate() %>% rplot()
# predicted vs observed (Lab 2, HW2): points on dashed 45-degree
    ↪ line = perfect
ggplot(res, aes(x = .pred, y = price)) + geom_point(alpha = 0.2) +
  geom_abline(lty = 2) + coord_obs_pred()
ggplot(res, aes(.pred, price)) + geom_point() + facet_wrap(~ model
    ↪ )
```

Other helpers seen: `reorder(cut, price)` orders categories by mean price; `fct_infreq()` by frequency, `fct_rev()` reverses; `geom_tile()` heatmap; `xlab()`; `theme_bw()`. `vis_miss()(airquality)` plots missingness (`naniar`).

5.4 rsample: splitting

`set.seed()(3435)`	before every random step (split; later CV) ⇒ reproducibility; any number
`initial_split()(df, prop, strata)`	returns an `rsplit` object (prints `<Training/Testing/Total>`), not data
`prop`	fraction to **training** (default 0.75); 0.80 Lab 1–2, 0.70 Lab 3, 0.85 HW2 (3548/629)
`strata`	stratified sampling on the outcome (numeric: binned into quartiles) so train/test have the same outcome distribution
`training()`, `testing()`	extract the two data frames from the split

Details and why stratify: see *Data splitting* in Linear models.

5.5 recipes: steps, selectors, prep/bake

`recipe(outcome ~ predictors, data = train)`: the formula lists roles (" . " = all other columns, *including* IDs and leaky columns); `data` supplies column names/types and is what `prep()` estimates from. Steps run **in the order written**.

Step	What it does
`step_dummy()`	usually on `all_nominal_predictors()`: factor/character with K levels → $K-1$ 0/1 columns `var_level`; first level = reference level (no column) (S2.45)
`step_dummy(type, one_hot = TRUE)`	K columns (one-hot encoding) (S2.46): rank-deficient with `lm` (has intercept), fine for KNN (PE Q7, Q13)
`step_normalize()`	$(x-\bar{x})/s$ with train mean/SD = `step_center()` + `step_scale()`
`step_interact()(terms = ~ a:b)`	adds product column `a_x_b`; for factors use the dummy names: `starts_with("type_")`:`shucked_weight`
`step_impute_linear()`	`(x, impute_with = imp_vars()(a, b))`: fill NAs of x by a linear regression on the listed (complete) columns
`step_impute_mean()` / `_median` / `step_impute_mode()` / `step_impute_knn()` / `step_impute_bag()`	other imputations (S2.7); mean/median/mode shrink variance
`step_poly()(x, degree = 2)`	orthogonal polynomial columns `x_poly_1`, `x_poly_2`
`step_log()(x)`	$\log(x)$ (natural; `base` = to change)
`step_mutate(new = expr)`	any new column, e.g. `square_meters = square_feet * 0.092903` (Sai Q9)
`step_rm(id)`	remove a column (IDs, leaky vars)

Selectors (used inside steps)

`all_predictors()` (every predictor) · `all_numeric_predictors()` · `all_nominal_predictors()` (factor/character) · `all_outcomes()` · by name (type, age) · `starts_with()("type_")` · `contains()`, `ends_with()` · exclude with `-`: `all_predictors()`, `-age`. `all_predictors()` never touches the outcome, so the outcome is not normalized.

Step order that works

impute → transform (log, poly, mutate) → step_dummy → step_interact → step_normalize/center/scale.

- Impute first: dummies/products/means cannot be computed from NAs.
- Dummy before interact: after `step_dummy()` the factor `sex` no longer exists; it is `sex_male`, hence `starts_with("sex")` (HW3 Q4).
- Dummy before normalize when normalizing `all_predictors()`: centering/scaling needs numeric columns (a factor would error). With `all_numeric_predictors()` written *after* `step_dummy`, the 0/1 dummies get normalized too (HW2 centers/scales every predictor, dummies and interactions included).

```
abalone_recipe <- recipe(age ~ type, data = abalone_train) %>%
  step_dummy(type)                                  # S2.45
abalone_recipe %>% prep() %>%
  bake(new_data = abalone_train)  # or new_data = NULL
```

```
# A tibble: 3,131 x 3
    age type_I type_M      # type has levels F, I, M
  <dbl>  <dbl>  <dbl>      # F = reference (both 0)
1   8.5      1      0      # an infant
3   8.5      0      1      # a male
5   9.5      0      0      # a female
```

- `prep()` = *estimate* everything from the training data (means/SDs, imputation models, factor levels); `bake()(new_data = X)` = *apply* those fixed values to `X`; `new_data = NULL` returns the processed training set. Used only to *look* at the result — `fit()` on a workflow does prep+bake itself, and `predict()`/`augment()` bake new data with the **training** estimates.
- Baked columns: original numerics, then outcome, then new dummies/interactions at the end (HW2: `type_I`, `type_M`, `type_I_x_shucked_weight`, `longest_shell_x_diameter`, …).
- **Ordered factors** (`diamonds`: `cut`, `color`, `clarity`) get *polynomial* contrasts, not 0/1: columns `cut_1`…`cut_4` with values like 0.632, -0.267 (Lab 2).

Printed recipe (HW2):

```
-- Recipe ------------------------------------
-- Inputs
Number of variables by role
outcome:   1
predictor: 8
-- Operations
* Dummy variables from: all_nominal_predictors()
* Interactions with: ...
* Centering for: all_predictors()
* Scaling for: all_predictors()
```

5.6 parsnip: model specs, engines, modes

Model	Spec function	Engine	Mode / package / main args
linear regression	`linear_reg()`	`"lm"`	regression only (default); base R `lm`
logistic regression	`logistic_reg()`	`"glm"`	classification only; base R `glm` (binomial)
KNN	`nearest_neighbor()`	`"kknn"`	**must** set mode `"regression"` or `"classification"`; `library(kknn)`; `neighbors` = k (default 5)
LDA	`discrim_linear()`	`"MASS"`	classification; `library(discrim)`, engine pkg `MASS`
QDA	`discrim_quad()`	`"MASS"`	classification; `library(discrim)`
naive Bayes	`naive_Bayes()`	`"klaR"`	classification; `library(discrim)` + `library(klaR)`; `set_args(usekernel = FALSE)` = Gaussian

- Spec = model type + `engine` (which R package computes it) + `mode` (regression/classification). Same thing as arguments: `nearest_neighbor(mode = "classification", engine = "kknn", neighbors = 7)`.
- Hyperparameters go *in the model function* (`nearest_neighbor(neighbors = 2)`) or via `set_args()`; never in the recipe. Spelling: `naive_Bayes` (capital B), `nearest_neighbor` (no s).
- The spec function for LDA/QDA/naive Bayes only exists after `library(discrim)`; "could not find function" = package not loaded.

```
> knn_model             # printing a spec (HW2)
K-Nearest Neighbor Model Specification (regression)
Main Arguments:
```

```
  neighbors = 7
Computational engine: kknn
> knn_fit %>% extract_fit_parsnip()   # Lab 2
parsnip model object
Call: kknn::train.kknn(formula = ..y ~ ., data = data,
                       ks = min_rows(5, data, 5))
Type of response variable: continuous
minimal mean absolute error: 373.8765
Minimal mean squared error: 515091.1
Best kernel: optimal
Best k: 5
```

("Best k: 5" only because 5 was the only (default) value tried (Lab 2).)

5.7 fit, predict, augment, tidy

predict() / augment() output columns

Call	Returns (a tibble, same rows as new_data)
`predict(fit, new_data = d)` regression	`.pred` (type `"numeric"`)
`predict(fit, new_data = d)` classification	`.pred_class` (type `"class"`: most probable class, i.e. $\hat{p} > 0.5$ for 2 classes)
`predict(fit, d, type = "prob")`	one column per level `.pred_<level>`: `.pred_Yes` `.pred_No`; `.pred_Down` `.pred_Up`; `.pred_CH` `.pred_MM` (rows sum to 1)
`predict(lm_fit, d, type = "conf_int")` / `"pred_int"`	`.pred_lower`, `.pred_upper` (CI / PI, linear reg.)
`augment()(fit, new_data = d)`	all columns of d **plus** `.pred` (regr.) or `.pred_class` + all `.pred_<level>` (classif.)

```
pr <- predict(lm_fit, new_data = d_train %>% select(-price))
pr <- bind_cols(pr, d_train %>% select(price))  # add truth
predict(smarketlog_fit, new_data = smarket_train, type = "prob")
```

```
# A tibble: 874 x 2
   .pred_Down .pred_Up          # Lab 3: all near 0.5 =>
1       0.521    0.479         # model can't discriminate
```

- **Predicting one new case** (HW2 Q7): build a `tibble()` with *every* predictor the recipe uses, same names and types (factor with the training levels): `tibble(type = factor("F", levels = levels(abalone$type)), longest_shell = 0.5, ...)`, then `predict(lm_fit, new_data = new_abalone)` (recipe applied automatically; → `.pred` 21.8).
- `predict()` gives *only* the prediction columns ⇒ `bind_cols()` the truth before metrics; `augment()` already keeps it.

Reading tidy() output

`tidy()` on a fitted workflow (directly, or after `extract_fit_parsnip()`) = coefficient table:

```
# A tibble: 24 x 5                      # Lab 2 diamonds, lm
   term        estimate std.error statistic   p.value
 1 (Intercept)  5317.       496.      10.7   8.54e- 27
 2 carat       11282.        54.3    208.    0
 6 y               9.08      19.9      0.456 6.49e-  1
 8 cut_1         596.        25.0     23.8   1.22e-124
```

- `term` = predictor column after the recipe (dummy names); `estimate` = $\hat{\beta}_j$; `std.error` = SE($\hat{\beta}_j$); `statistic` = estimate/std.error (t for `lm`, z for `glm`); `p.value` tests H_0: $\beta_j = 0$ given the other predictors (here y is not significant, $p = 0.65$).
- Interpretation: `carat` $+1$ ⇒ price $+\$11{,}282$ holding the others fixed. If the recipe normalized predictors, an estimate is the change per 1 SD and `(Intercept)` $= \bar{y}_{\text{train}}$ (HW2: 11.5).
- Logistic (`glm`): estimates are changes in log odds of the **second** factor level; $e^{\hat{\beta}}$ = odds ratio (see *Classification models*).
- KNN has no coefficients: `tidy()` is not informative; print `extract_fit_parsnip()` instead.

Base R equivalents (slides, ISLR)

```
fit <- lm(price ~ carat + cut, data = d_train); summary(fit)
glm.fit <- glm(default ~ student + balance + income,
               data = Default, family = binomial)   # S3.26
predict(glm.fit, newdata = new, type = "response")  #
    ↪ probabilities
predict(glm.fit, newdata = new)  # default type = "link": LOG
    ↪ ODDS
```

Base R uses `newdata=` and `summary()`; tidymodels uses `new_data=` and `tidy()`. Perfectly separable data ⇒ `glm` warns "algorithm did not converge" / "fitted probabilities numerically 0 or 1 occurred" (S3.45).

5.8 yardstick: metrics

Every metric: `metric(data, truth = <true col>, estimate = <prediction col>)`, `truth` **first**; returns a tibble `.metric` / `.estimator` (`standard` = regression, `binary`) / `.estimate`; pull the number with `$.estimate`. Full outputs and interpretation: see *Model evaluation*.

Call	Needs
`rmse()`, `rsq()`, `mae()` (`rsq_trad()`)	numeric `truth`, `estimate = .pred`. No `mse()` in yardstick: MSE = RMSE^2
`accuracy()`, `sensitivity()`, `specificity()`, `precision()`	factor `truth`, `estimate = .pred_class` (same levels)
`conf_mat()(truth =, estimate = .pred_class)`	rows = Prediction, cols = Truth; `%>% autoplot(type = "heatmap")` (or `"mosaic"`)
`roc_auc()(truth, .pred_Yes)`	factor truth + **probability** column of the **event** (first level); 3rd argument is unnamed
`roc_curve()(truth, .pred_Yes)`	returns `.threshold`, `specificity`, `sensitivity`; `%>% autoplot()` draws the ROC curve
`metric_set()(accuracy, sensitivity, specificity)`	makes a new function; list metric names *without* `()`; call it like any metric

- **Event level** = **first** factor level (`event_level = "first"` default). Smarket: `Down` ⇒ `roc_curve(Direction, .pred_Down)`; OJ: `.pred_CH`; Titanic: relevel so "Yes" is first. Alternative: keep levels and pass `event_level = "second"`.
- Lab 3 Smarket QDA test: sensitivity 0.254 $= 46/(46 + 135)$ is the rate for `Down`, the first level.

5.9 Common bugs / exam traps

Spot-the-bug checklist

- **No `set.seed()`** before `initial_split()` ⇒ a new random split (and new results) every run (Lab 1, HW2).
- **Recipe/model on the wrong data**: `recipe(..., data = test)`, `fit(wf, test)`, or recipe built on the full data before splitting ⇒ test information leaks into training (means/SDs/imputation from test). Recipe and `fit()` use **training**; test is only for the final `predict()`/`augment()`.
- **Metrics on training data** reported as performance: optimistic (KNN: each training point is its own neighbour; Lab 2 $k = 2$ train R^2 0.999). Use `augment(fit, new_data = test)`.
- **Recipe defined but never used** (PE Q7): `loan_model |> fit(defaulted ~ ., data = loans)` fits raw data; the `step_dummy` never runs. Fix: `workflow() %>% add_model() %>% add_recipe()` then `fit(wf, train)`.
- **`y ~ .`** pulls in identifier variables (`application_id`) and target leakage (`default_date`, `rings` for age) (PE Q7, HW2 Q3).
- **KNN without `step_normalize()`** ⇒ large-scale variables dominate distances (PE Q6–7).
- **`one_hot = TRUE` with `lm`** ⇒ "rank-deficient" fit (dummies sum to the intercept column); use default `one_hot = FALSE` → $K - 1$ dummies (3 departments → 2) (PE Q13). Fine for KNN.
- **Redundant predictors**: `step_mutate(square_meters = square_feet * 0.092903) + step_normalize()` still leaves two perfectly collinear columns; normalizing does not fix collinearity — drop one (Sai Q9).
- **Step order**: `step_interact(~ sex:fare)` after `step_dummy` must use `starts_with("sex")`; `step_normalize(all_predictors())` before `step_dummy` hits factor columns; impute before anything that uses the column.
- **Outcome not a factor** for classification (0/1 numbers, character): `logistic_reg()` / `conf_mat()` need a factor ⇒ `mutate(y = factor(y))`.
- **Wrong event level**: `factor()` sorts levels alphabetically (`"No"` before `"Yes"`), so sensitivity/AUC would be for "No". Relevel (`levels = c("Yes","No")`) or `event_level = "second"`. Passing `.pred_No` when Yes is the event gives AUC $= 1 -$ AUC.
- **glm vs yardstick**: glm coefficients are for the **second** level; yardstick's event is the **first**.
- **Missing `type = "prob"`**: `predict()` default gives only `.pred_class`;

`roc_auc()`/`roc_curve()` need `.pred_<event>` (`augment()` has both).

- **`roc_auc(truth, .pred_class)`** = error (needs numeric probabilities); `accuracy(truth, .pred_Yes)` = error (needs classes).
- **Argument names**: `new_data =` (tidymodels) vs `newdata =` (base R); `truth` before `estimate`.
- **Missing `set_mode()`** for `nearest_neighbor()` (mode unknown) ⇒ error at fit.
- **Package not loaded**: `discrim_linear()`/`naive_Bayes()` need `library(discrim)`; `"kknn"` needs kknn installed.
- **`filter(x = 5)`** instead of ==; `%>%` between ggplot layers instead of +; `mean(x)` with NAs returns NA (`na.rm = TRUE`); forgetting <- so the result is not saved.
- **Picking a model by training accuracy** (Lab 3 did, only because CV was not yet taught): Smarket QDA best on train (0.548) but 0.487 on test (Lab 3).
- Lab 2 text says "metric set of RMSE, MSE, and R^2" but the code `metric_set(rmse, rsq, mae)` computes **MAE**.

6 Exam prep: past questions, patterns, quick answers

Format (Canvas): ≤12 multiple choice + 4 short answer, paper notes only; R code / R output may appear. Sai's midterm (Summer 2026, 26 pts) = 7 MC × 2 pts + 2 free-response (8 + 4 pts). MC rule from the cover sheet: *more than one bubble = zero*; "Both X and Y" options are traps unless *both* parts are right. Round to 3–4 decimals.

6.1 Practice exam: every question, one line

Practice exam (Fall 2026) answer key, condensed (PE, official solutions)

Q1 KNN great on train, worse on new data ⇒ **(b) increase** k. Small k = overfitting (high variance); decreasing k makes it worse; weighting high-variance variables hurts (standardization); interactions irrelevant for non-parametric KNN. (PE Q1)

Q2 "Most flexible is always best" — wrong: flexible models lose interpretability (bad if goal is inference) and can fit noise ⇒ low training error, high testing error. (PE Q2)

Q3 Train/test acc. A 71/70, B 86/82, C 100/68 ⇒ **B** (highest *test* accuracy); C overfits (B a little). (PE Q3)

Q4 All three claims **false**: (a) zero Pearson correlation = no *linear* relation, can still help nonlinear/empirical models; (b) correlated ≠ interacting (interaction term = effect of X_1 depends on X_2); (c) lower training error not better. (PE Q4)

Q5 Neighbours A B B A A B B: $k = 1$ **A**, $k = 3$ **B** (2B:1A), $k = 5$ **A** (3A:2B), $k = 7$ **B** (4B:3A). (PE Q5)

Q6 Age standardized, income not: KNN is distance-based; income's huge variance dominates the Euclidean distance. Normalize *all* numeric predictors. (PE Q6)

Q7 Loan KNN code, ≥4 bugs: (1) `defaulted ~ .` pulls in `application_id` (identifier variable) and `default_date` (target leakage) — drop them; (2) no `step_normalize()`; (3) recipe never used (`fit()` on raw formula, no `workflow()`/`add_recipe()`); (4) so `employment_type` never dummy-coded. `one_hot = TRUE` is *fine* for KNN (no intercept). (PE Q7)

Q8 Strong U-shape, $r = 0.03$ ⇒ **(c)** little linear correlation despite strong nonlinear relationship; X can still predict Y. (PE Q8)

Q9 Evidence of interaction = **(b)** X_1–Y slope positive for one level of X_2, negative for the other. Correlation/skew are not evidence. (PE Q9)

Q10 Graduation model (info at end of year 1): exclude **StudentID** (ID), **GraduationDate**, **FinalGPA** (unknown at prediction time = leakage), **Graduated4Years** (the outcome). Keep HS_GPA, FirstYearGPA, Major, Credits*. (PE Q10)

Q11 Need dummies for logistic regression: **Major, Dorm** (categorical). (PE Q11)

Q12 CompletionRate = Completed/Attempted (year-1 info) is **not** leakage; but with both components it is redundant (collinearity). (PE Q12)

Q13 Rank-deficient `lm`: `step_dummy()` with `one_hot = TRUE` gives 3 indicators that sum to the intercept column ⇒ design matrix not full rank. Fix `one_hot = FALSE` ⇒ **2** indicators ($K-1$, one reference level). (PE Q13)

Q14 $\hat{p} = 0.78$ ⇒ **(c)** model assigns probability 0.78 of the pneumonia class (not "78% accurate", not certain). (PE Q14)

Q15 $\hat{p}$ = .31,.48,.63,.81: threshold 0.5 ⇒ **C, D**; 0.7 ⇒ **D**; raising t ⇒ fewer positives. (PE Q15)

Q16 $\beta = 0.04$ for MonthlyFee: not "causes +4% probability". Linear in log odds, and only association: "$1 more is *associated* with +0.04 log odds" (odds $\times e^{0.04} = 1.041$). (PE Q16)

Q17 TP 72, FP 18, FN 28, TN 182: accuracy $254/300 = 0.8467$, TPR $72/100 = 0.72$, FPR $18/200 = 0.09$. (PE Q17)

Q18 Threshold 0.5 → 0.3 does **not** change AUC (ROC built over all thresholds). (PE Q18)

Q19 Dangerous disease, FN ≫ FP cost ⇒ **Point C** (highest TPR, accept FPR ≈ 0.5) on the ROC curve. (PE Q19)

Q20 $n = 50{,}000$, $p = 2$, covariances clearly differ ⇒ **(b)** QDA (LDA assumes a common covariance matrix; extra variance negligible at this n). (PE Q20)

6.2 Sai's graded midterm: every question, where points were lost

Q	Topic (correct answer, why)	Sai	Key	Pts
1	More flexible? Add polynomial/interaction terms. (↑ k, fewer predictors, LDA vs QDA = *less* flexible)	C	C	2/2
2	Why KNN makes nonlinear boundaries: classification decided *locally* by neighbours	C	C	2/2
3	ROC point A→C = *decreasing* the threshold (TPR and FPR both rise)	B	B	2/2
4	Readmission predictor: `HospitalID` (15 hospitals, populations differ = real group info); others are unique IDs	E	B	**1/2**
5	Covariances truly equal: QDA gains nothing, only adds estimation variance	B	B	2/2
6	Interaction: dosage effect $+10$ under A vs. $+25$ under B	F	B	**0/2**
7	$\beta_{\text{age}} > 0$ but class unchanged: $\hat{p}$ rose without crossing the threshold	A	B	**0/2**
8a	Threshold $0.40 \to 0.70$: sensitivity decreases	✓		2/2
8b	specificity increases (= 1−FPR, FPR falls)	✓		2/2
8c	ROC curve unchanged (model-level; only the operating point moves)	✓		2/2
8d	Number flagged decreases (fewer $\hat{p}$ above the bar)	✓		2/2
9	Keep `square_feet` & `square_meters` after normalizing? **No**: exact collinearity	ok		**3/4**

Total 20/26. Verified against the scanned booklet and Gradescope rubric ((Sai pp. 1–8)).

Sai's lost points and the correct reasoning

- **Q4 (−1).** Sai: (E) "None of the above". Key: (B) `HospitalID`. A grouping ID with few levels that *carries information* (patient populations differ) is a legitimate categorical predictor (dummy-encode it); `PatientID`, `AdmissionID`, `RecordID` are unique per row (identifier variable: no pattern can generalize). *Unclear grading:* the rubric item applied reads "partial credit for answering all variables", which does not literally match (E); it appears the grader gave half credit for rejecting the three unique IDs. Rule: unique-per-row ID ⇒ drop; shared group label ⇒ keep. (Sai Q4)
- **Q6 (−2).** Sai: (F) "Both B and D". D ("dosage increases the fitted value under both treatments") is a main effect, not evidence of interaction; so F is wrong even though B is right. Interaction = the *size* (or sign) of one variable's effect differs across levels of the other: $30 - 20 = 10$ vs. $50 - 25 = 25$. (Sai Q6)
- **Q7 (−2).** Sai: (A) "a positive coefficient does not affect predicted probabilities" — false; a positive β raises the log odds and hence $\hat{p}$. Key (B): $\hat{p}$ can rise (e.g. 0.31→0.32) without crossing $t = 0.5$, so the class is unchanged. (Sai Q7)
- **Q9 (−1).** Sai said "not appropriate" (right) but argued via normal curves. Required word: **collinearity**. `square_meters` $= 0.092903 \times$ `square_feet` exactly, so after `step_normalize()` the two columns are *identical* ($r = 1$); OLS cannot separate their coefficients (one is NA/huge SEs). Normalizing never removes collinearity — drop one. (Sai Q9)

6.3 Question patterns and how to answer them

P1. Confusion matrix → metrics (PE Q17; HW3 Q10; S3.34–37)

Recipe. (1) Find the orientation: are *rows* predicted or true? (PE Q17: rows = predicted; S3.34: rows = true; `conf_mat()` prints `Prediction` rows × `Truth` columns.) (2) Which class is "positive"? (yardstick: the *first* factor level.) (3) Label TP, FP, FN, TN by *meaning*. (4) Divide by the right total: actual positives (TPR, FNR), actual negatives (FPR, spec.), predicted positives (precision).

Example. Truth Yes: 40 predicted Yes, 10 predicted No; truth No: 20 predicted Yes, 130 No. So TP 40, FN 10, FP 20, TN 130, $n = 200$. accuracy $= 170/200 = 0.85$; sensitivity $= 40/50 = 0.80$; specificity $= 130/150 = 0.867$; FPR $= 20/150 = 0.133$; precision $= 40/60 = 0.667$; error rate $= 0.15$.

P2. Change the threshold: what happens? (Sai Q7–8; PE Q15)

Recipe. Predict positive if $\hat{p} \geq t$. Raise $t \Rightarrow$ fewer positives $\Rightarrow$ TP, FP ↓; sensitivity/TPR ↓, FPR ↓, specificity ↑; the ROC curve and AUC are **unchanged** (only the operating point slides down-left). Lower t: the reverse. Coefficients and $\hat{p}$ never change with t.

Example. $\hat{p}$ / truth: 0.15 No, 0.40 Yes, 0.62 No, 0.85 Yes.

t	TP	FP	FN	TN	sens	spec
0.3	2	1	0	1	1.0	0.5
0.5	1	1	1	1	0.5	0.5
0.7	1	0	1	2	0.5	1.0

P3. Read an ROC curve / compare AUCs (PE Q18–19; Sai Q3, Q8c; ROC)

Recipe. x = FPR $(1-$spec.$)$, y = TPR (sens.); each point = one threshold. Up/right = lower t. Pick the point by costs: FN costly (disease, fraud) $\Rightarrow$ high-TPR point (up/right); FP costly $\Rightarrow$ low-FPR point (down/left). Compare models by AUC (higher better; 0.5 = coin flip, 1 = perfect, AUC $= 1 \Rightarrow$ suspect target leakage). AUC does not depend on the chosen threshold.

Example. Same 4 cases as P2: positives score 0.40, 0.85; negatives 0.15, 0.62. Of the 2×2 positive–negative pairs, the positive scores higher in 3 $\Rightarrow$ AUC $= 3/4 = 0.75$ (standard "probability a random positive outranks a random negative" reading). Model A AUC 0.88 vs. B 0.79 $\Rightarrow$ A discriminates better at almost every threshold.

P4. Flexibility, bias–variance, k in KNN (PE Q1–2; Sai Q1–2, Q5; S1.113–115)

Recipe. More flexible $\Rightarrow$ lower bias, higher variance, training error ↓, test error U-shaped. *More* flexible: smaller k, add predictors / polynomial / interaction terms, QDA instead of LDA, KNN instead of a linear model. Overfit symptom: train $\ll$ test $\Rightarrow$ make it *less* flexible (increase k, drop terms).

Example. $k = 1$: each training point is its own neighbour, training error $= 0$ (maximal variance). $k = n$: always predicts the overall majority class (maximal bias). With 400 training points, $k = 1$ train/test error 0.00/0.30 vs. $k = 15$ 0.18/0.21 $\Rightarrow$ choose $k = 15$.

P5. Pick a model from a train/test table (PE Q3; HW2 Q9; Lab 2)

Recipe. Choose by the **test** column (highest accuracy/AUC/R^2, lowest RMSE/MAE). Then diagnose: big train–test gap = overfitting; train ≈ test but both poor = underfitting.

Example. RMSE train/test: linear 2.18/1.95, KNN 1.46/2.21 (HW2 abalone) $\Rightarrow$ linear model (lower test RMSE); KNN overfits.

P6. KNN by hand (PE Q5–6; S1.103–109)

Recipe. Standardize first; compute distances $d = \sqrt{\sum_j (x_j - x_{0j})^2}$; sort; classification = majority vote of the k closest; regression = *average* y of the k closest.

Example. $x_0 = (0, 0)$; $(1, 1)$ A: $d = 1.41$; $(2, 0)$ B: 2; $(0, 3)$ B: 3; $(3, 4)$ A: 5. $k = 1 \Rightarrow$ A; $k = 3 \Rightarrow$ B (A,B,B). Regression: nearest 3 have $y = 10, 12, 17$ $\Rightarrow \hat{y} = 13$.

P7. Interpret a logistic coefficient / compute $\hat{p}$ (PE Q14, Q16; Sai Q7; S3.12–25)

Recipe. β_j = change in log odds per unit X_j, others held fixed; e^{β_j} = odds ratio (multiplicative); only the *sign* transfers to p; say "associated with", never "causes". To predict: $z = \beta_0 + \sum \beta_j x_j$, $\hat{p} = e^z/(1+e^z)$, then compare with t.

Example. $\beta_0 = -3$, $\beta_1 = 0.05$ (age). Age 40: $z = -1$, $\hat{p} = e^{-1}/(1+e^{-1}) = 0.269 \Rightarrow$ "No" at $t = 0.5$. Age $+10$: odds $\times e^{0.5} = 1.65$. $\beta = 0.7$: odds $\times e^{0.7} = 2.01$ (about double). Template: "A one-unit increase in X is associated with a β increase in the log odds of Y=Yes (odds multiplied by e^{β}), holding other predictors fixed."

P8. Read regression output; dummy and interaction coefficients (HW2 Q6; PE Q13; S2)

Recipe. `tidy()` columns: `term`, `estimate` ($\hat{\beta}$), `std.error`, `statistic` = estimate/SE (t-statistic), `p.value` ($< 0.05 \Rightarrow$ significant). Dummy coefficient = difference from the reference level (the level with no column), others fixed. Interaction $X_1{:}X_2$ coefficient = how much the X_1 slope changes per unit of X_2.

Example. `diameter` 2.20, SE 0.308 $\Rightarrow t = 7.14$, $p \approx 10^{-12}$. Salary $= 40 + 2$ exp $+ 5$ IT $- 3$ HR (Sales = reference): IT with 10 yrs $\Rightarrow 40 + 20 + 5 = 65$; HR earns 8 less than IT at equal experience.

P9. Spot an interaction in a table / plot (PE Q9; Sai Q6)

Recipe. Compute the effect of X_1 at each level of X_2 (differences, or slopes). Different $\Rightarrow$ interaction; equal (parallel lines) $\Rightarrow$ additive. Correlation between predictors is irrelevant.

Example. Low/High dose: A 20/30, B 25/50 $\Rightarrow$ +10 vs +25: interaction. A 20/30, B 25/35 $\Rightarrow$ +10 both: none (B just shifted up 5).

P10. Which variables may be predictors? (PE Q7, Q10, Q12; Sai Q4; HW2 Q3)

Recipe — ask four questions of each variable: (1) Is it the outcome or computed from it? (2) Is it known *at prediction time*? (no $\Rightarrow$ target leakage). (3) Is it unique per row (identifier variable)? (4) Is it an exact function of another predictor (collinearity)? Group labels with a few shared levels (hospital, dorm, major) are fine $\Rightarrow$ dummy-encode.

Example. Predict abalone age: `rings` is out (age = rings $+1.5$, leakage). Loan default: `default_date` out (only exists after default), `application_id` out (ID).

P11. Choose recipe steps (and their order) (S2; Lab 2–3; HW2 Q3; HW3 Q4)

Situation	Step
missing numeric values	`step_impute_linear()` (`imp_vars()`) or `step_impute_mean()`
categorical predictor, lm/glm/LDA	`step_dummy()` (default $K - 1$ columns)
categorical, KNN	`step_dummy(..., one_hot = TRUE)` OK (no intercept)
KNN (distance)	`step_normalize()` = `step_center()` + `step_scale()`
effect of X_1 depends on X_2	`step_interact()`, after dummying: `~ starts_with("sex"):fare`
curved relationship	`step_poly()`; skewed X `step_log()`

Order: impute → dummy → interact → normalize. Recipe is *trained on the training data* (`recipe(y ~ ., data = train)`).

P12. Spot the bug in tidymodels code (PE Q7, Q13; Sai Q9; Lab 3)

Checklist: **(1)** recipe built but model fit with `fit(y ~ ., data)` $\Rightarrow$ recipe ignored; use `workflow()` %>% `add_recipe()` %>% `add_model()`. **(2)** `y ~ .` includes IDs / leakage. **(3)** KNN without `step_normalize()`. **(4)** `one_hot = TRUE` with an intercept model (rank deficient). **(5)** `nearest_neighbor()` without `set_mode()` (mode unknown). **(6)** metrics on *training* data / `predict()` on training instead of `testing()`. **(7)** `roc_auc()` given `.pred_class` instead of the probability column (`.pred_Yes`, from `type = "prob"`). **(8)** wrong event level (first factor level is the event). **(9)** no `set.seed()` before `initial_split()`; no `strata` for an imbalanced outcome. **(10)** `step_interact()` naming a variable that `step_dummy()` already renamed (`sex` → `sex_male`).

P13. LDA vs. QDA vs. logistic vs. KNN (PE Q20; Sai Q2, Q5; S3.89–93)

Recipe. Boundary linear? → logistic or LDA (LDA if predictors roughly normal within classes / small n; logistic if normality doubtful). Class covariances clearly differ *and* n large relative to $p \rightarrow$ QDA. Covariances equal → LDA (QDA only adds variance). Very irregular boundary, large n, small $p \rightarrow$ KNN. Flexibility: KNN > QDA > LDA = logistic.

Example. $n = 150, p = 8$, similar spreads ⇒ LDA. $n = 50{,}000, p = 2$, different covariances ⇒ QDA. Binary outcome, skewed/dummy predictors ⇒ logistic regression.

P14. Correlation, collinearity, class imbalance (PE Q4, Q8; Sai Q9; HW3 Q1–2)

Recipe. r measures *linear* association only ($r \approx 0$ with a U-shape is possible). Two predictors with $|r| \approx 1$ (or one an exact rescaling of the other) ⇒ collinearity: keep one. Imbalanced outcome ⇒ stratify the split (`strata`) and judge accuracy against the majority-class baseline.
Example. 90% negatives: "always negative" scores accuracy 0.90 with sensitivity 0 and AUC 0.5. Titanic: 61.6% died ⇒ any useful model must beat 0.616.

P15. Read other R output (Lab 2–3; HW2–3)

`predict()` default `type = "class"` → `.pred_class`; `type = "prob"` → `.pred_Yes`/`.pred_No`; regression → `.pred`. `augment()` = data + all prediction columns. Metric tibbles: `.metric`, `.estimator`, `.estimate`. LDA print: *Prior probabilities* = training class proportions ($\hat{\pi}_k$), *Group means* = $\hat{\mu}_k$, *LD1* = discriminant direction. kknn print: "Minimal misclassification" = leave-one-out training error. `initial_split()` print `<Training/Testing/Total>`.

6.4 60-second facts

High-yield one-liners

- Expected test MSE $= \mathrm{Var}(\hat{f}) + \mathrm{Bias}^2 + \mathrm{Var}(\epsilon)$; last term = irreducible error, always $\geq$ it.
- supervised learning has a Y; unsupervised learning does not. regression = numeric Y, classification = categorical Y.
- Parametric (linear, logistic, LDA, QDA) = assume a form for f; non-parametric model (KNN) = no form, needs more data.
- KNN: small k = flexible = low bias / high variance; $k = 1$ training error 0. Always normalize.
- curse of dimensionality: with many predictors, the "nearest" neighbours are far away ⇒ KNN degrades.
- Lower training error is never by itself a reason to prefer a model; compare on testing data.
- Bayes classifier: assign the class with the largest $\Pr(Y = k \mid X = x_0)$; its error = Bayes error rate (lowest possible).
- R^2 = fraction of variance in Y explained; RMSE/MAE in units of Y, lower is better.
- K levels ⇒ $K - 1$ dummies with an intercept; K (one-hot) only without an intercept / for KNN.
- Interaction = effect of one predictor depends on another; hierarchical principle: keep main effects if the interaction is in.
- Correlation $\neq$ interaction; correlation $\approx 0 \neq$ useless; correlation $\neq$ causation.
- logistic regression: logit linear in X; e^{β} = odds ratio; $p = 0.5 \Leftrightarrow$ log odds $= 0$; odds $= p/(1-p)$.
- Logistic fit by maximum likelihood; LDA by plugging in $\hat{\pi}_k, \hat{\mu}_k, \hat{\Sigma}$.
- LDA: common Σ, linear decision boundary; QDA: Σ_k per class, quadratic boundary, more parameters.
- confounding: a coefficient's sign can flip once another predictor is added (Default `student`).
- sensitivity = TPR = recall = $TP/(TP + FN)$; specificity = $TN/(TN + FP) = 1 -$FPR; precision $= TP/(TP + FP)$.
- Raise threshold: sens ↓, spec ↑, positives ↓; ROC curve and AUC unchanged.
- ROC curve: x = FPR, y = TPR; diagonal = random (AUC 0.5); top-left corner = perfect.
- AUC is threshold-free and summarizes all thresholds; AUC = 1 on real data ⇒ check for target leakage.
- `yardstick`: first factor level = event ⇒ `factor(survived, levels = c("Yes","No"))`.
- `glm` (logistic) models the *second* level: with Yes first, coefficients are for "No".
- Stratify on the outcome so train/test keep the class proportions; `set.seed()` for reproducibility.
- Recipe + model → `workflow()` → `fit()` on training → `augment()`/`predict()` on testing → metrics.
- `step_normalize()` cannot fix collinearity; it only puts predictors on a common scale.
- A predicted probability is *not* an accuracy; a model has accuracy, one patient has a probability.

6.5 HW answers at a glance

HW1 — ML ideas; EDA on `mpg` (HW1; submission)

Q1 supervised = labelled Y to learn f; unsupervised = no Y, find structure (clusters). **Q2** regression: quantitative Y; classification: qualitative Y (class or class probabilities). **Q3** regression: RMSE, R^2 (MAE); classification: accuracy, sensitivity/recall (AUC). **Q4** descriptive model: visually emphasize a trend; inferential model: which features are significant, test theories/relationships; predictive model: best prediction of Y, minimal reducible error (S1.50). **Q5** mechanistic model = parametric model (assumed form; easier to understand; higher bias, lower variance); empirically-driven model = non-parametric model (few assumptions, more data, more flexible: lower bias, higher variance); both estimate f and both can overfit. (Correction to my submission: empirical models are *lower*-bias, not "unbiased".) **Q6** (a) chance a voter supports the candidate = **predictive**; (b) effect of personal contact = **inferential**. **EDA:** `hwy` histogram bimodal (peaks ≈16 and ≈26), slightly right-skewed; `hwy–cty` scatterplot strong positive linear ($r = 0.96$); bar plot: most cars dodge (37), fewest lincoln (3); boxplot: `hwy` falls with `cyl` (medians 29/24/17 for 4/6/8); `corrplot()` correlation matrix: `displ–cyl` 0.93, mpg vs. `displ`/`cyl` ≈ -0.8, `year` ≈ 0.

HW2 — abalone, linear regression vs. KNN (HW2; submission)

Q1 `age = rings + 1.5` (`mutate()`); mean 11.4, median 10.5, SD 3.22, range 2.5–30.5: unimodal, right-skewed, ≈6.7% boxplot outliers. **Q2** `initial_split()` (`prop = 0.85`, `strata = age`) → 3548 / 629. **Q3** recipe: `step_dummy()` → `step_interact()` (type×shucked, longest×diameter, shucked×shell) → `step_center()` → `step_scale()`; exclude `rings`: age is an exact function of it (target leakage). **Q4–6** `linear_reg()` `"lm"`, `nearest_neighbor()` (`neighbors = 7`) `"kknn"`, two workflows, fit on training. **Q7** hypothetical female (`predict()`): $\hat{y} = 21.8$ years. **Q8** `metric_set(rsq, rmse, mae)` + `augment()` on test: LM R^2 0.585, RMSE 1.95, MAE 1.44; KNN 0.483 / 2.21 / 1.62. $R^2 = 0.585$: 58.5% of the variance in test-set age explained. **Q9** LM wins on test; KNN training R^2 0.811 → test 0.483 = overfitting (flexible, high variance); LM 0.552 → 0.585 (rigid, stable). **Q10–12 (231)** reducible error $= \mathrm{Var} + \mathrm{Bias}^2$, irreducible error $= \mathrm{Var}(\epsilon)$; both nonnegative ⇒ test error $\geq \mathrm{Var}(\epsilon)$.

HW3 — Titanic classifiers (MY OWN answers; not submitted/graded) (HW3; raw HW section)

Self-worked solutions (R, seed 3435); exact numbers depend on the seed/split — know the reasoning. **Q1** 70/30, `strata = survived`: 623 / 268. missing data: `age` 128 (impute), `cabin` 481 (drop), `embarked` 2; IDs (`passenger_id`, `name`, `ticket`) not predictors. Stratify because the outcome is imbalanced (38.4% Yes) — both sets keep the same proportion. **Q2** sex is a strong predictor (survival 72.9% female vs. 20.3% male); class too (63.0 / 47.7 / 23.8% for 1st/2nd/3rd). Percent stacked bars (`geom_bar()`, `position = "fill"`) compare *proportions* when the bar totals differ. **Q3** `sib_sp–parch` $+0.43$, `age–sib_sp` -0.33, `fare` weakly $+$ with the family counts; all $|r| < 0.45$, no collinearity problem (needs `use = "pairwise.complete.obs"`). **Q4** `step_impute_linear(age, impute_with = imp_vars(...))` → `step_dummy(all_nominal_predictors())` → `step_interact(~ starts_with("sex"):fare + age:fare)`. **Q5–8** `logistic_reg()` `"glm"`, `discrim_linear()` / `discrim_quad()` `"MASS"` (`discrim`), `nearest_neighbor(neighbors = 10)` `"kknn"`; same recipe, four workflows. **Q9** `predict()` `type = "prob"` + `bind_cols()` → `roc_auc()`; training AUC: logistic 0.845, LDA 0.844, QDA 0.852, KNN 0.964 (inflated:

own neighbour). **Q10** testing AUC: logistic 0.880, **LDA 0.881**, QDA 0.859, KNN 0.879 $\Rightarrow$ LDA (near-tie). LDA test confusion matrix (`conf_mat()`) TP 73, FP 22, FN 30, TN 143: accuracy 0.806, sensitivity 0.709, specificity 0.867; `roc_curve()` + `autoplot()` for the plot. Train 0.844 vs. test 0.881: rigid low-variance model, the difference is split-to-split sampling noise (small test set); KNN's 0.964 $\rightarrow$ 0.879 is the overfitting pattern. **Q11–12 (231)** logit $=$ inverse logistic function; $x_1 + 2$ multiplies odds by $e^{2\beta_1}$ (odds ratio); $\beta_1 < 0$: $p \rightarrow 0$ as $x_1 \rightarrow \infty$, $p \rightarrow 1$ as $x_1 \rightarrow -\infty$.

PART II RAW TRANSCRIBED MATERIAL (only what Part I does not cover)

7 Slide Set 1 — Intro to ML, bias–variance tradeoff, KNN (pp. 1–58)

[Slides/parts already covered in Part I are omitted: S1.1–5, S1.7, S1.10, S1.14, S1.25, S1.29, S1.33, S1.36–37, S1.39–44, S1.46–47, S1.49–50, S1.52–53, S1.55–58.]

7.1 Course overview and introduction

S1.6 **Required reading:**

- For **undergraduates**, ch. 1 of ISLR
- For **graduate students**, ch. 1 of ESL

S1.8 **A silly introduction**

How a cat decides whether to scratch on something: *[Figure: a hand-drawn decision tree (flowchart) next to a photo of a black-and-white cat scratching a sofa arm. Root node (blue box): "Am I supposed to scratch on it?". Branch **Yes** → red leaf "Don't scratch on it.". Branch **No** → blue node "Will my owners be mad if I scratch on it?"; from there **Yes** → green leaf "Definitely scratch on it!" and **No** → red leaf "Don't scratch on it.". Blue = questions (splits), green/red = final decisions (classes).]*

S1.9 **[untitled: Speed vs. amount of yams consumed]**

Link (top right): Jamaica's sprinting glory: Is it the yams or the genes? *[Figure: a toy scatterplot with x-axis "**Amount of Yams Consumed**" and y-axis "**Speed**" (three tick marks on each axis, no numbers). Five blue dots trend upward; a black dashed straight line (a fitted regression line) runs through them from lower left to upper right. Annotations: "This is me:" points to the lowest-left dot (few yams, slow); "These are some other people:" points to three middle dots; "This is Usain Bolt:" points to the top-right dot (most yams, fastest). A red **X** on the x-axis marks a new yam amount; a green dashed arrow goes straight up from it to the dashed line and then a green dashed line goes left to a red **X** on the y-axis, i.e. reading off the predicted speed for a new person from the line. Right side: a photo of Usain Bolt (Jamaica, "BOLT", London 2012) crossing the finish line and a photo of a baked yam/sweet potato with butter.]*

S1.11 **[untitled: xkcd "Machine Learning" comic]**

[Figure: xkcd comic #1838 (`https://xkcd.com/1838/`). One stick figure stands on a big pile (labelled with matrices, λ, f, numbers) holding a paddle; a funnel labelled "DATA" feeds into the pile and a box labelled "ANSWERS" comes out the other side. Paraphrased dialogue: the other figure asks whether this is your machine learning system; the answer is yes — pour the data into this big pile of linear algebra and collect the answers on the other side; asked what happens if the answers are wrong, he says to just stir the pile until they start looking right. (Joke: ML as an opaque, trial-and-error "black box".)]

S1.12 **[untitled: xkcd "Tasks" comic]**

[Figure: xkcd comic #1425 (`https://xkcd.com/1425/`). A manager asks a programmer at a desk for an app that, when a user takes a photo, checks whether they are in a national park (programmer: easy GIS lookup, a few hours) and also checks whether the photo is of a bird (programmer: that needs a research team and five years). Caption, paraphrased: in CS it can be hard to explain the difference between the easy and the virtually impossible. (Point: image recognition, a classic ML task, is much harder than rule-based lookups.)]

S1.13 **Machine learning vs. statistical learning**

[Figure: four-panel meme. Panel 1: a man looks at a crack in a wall labelled "statistics". Panel 2: he hangs a picture frame around the crack. Panel 3: the framed crack is now labelled "Machine Learning" and he stands proudly with arms crossed. Panel 4: the framed crack, now labelled "Artificial intelligence", is displayed to a large admiring audience. (Joke: ML/AI are statistics re-branded.)]

S1.15 **[untitled: AI ⊃ machine learning ⊃ deep learning timeline]**

[Figure: Nvidia infographic ("Source: Nvidia"): a timeline from the 1950's to the 2010's with three nested green blocks. ARTIFICIAL INTELLIGENCE (from the 1950's; early AI stirs excitement; icon of a checkers board), MACHINE LEARNING (from the 1980's; ML begins to flourish; icon of e-mails going into a trash can = spam filtering), and DEEP LEARNING (from the 2010's; deep learning breakthroughs drive the AI boom; icon of a cat picture made of binary digits under a magnifying glass = image recognition). Caption, paraphrased: since the 1950s, successively smaller subsets of AI — first machine learning, then deep learning (a subset of ML) — have created ever larger disruptions.]

S1.16 **AI has a long history of being "the next big thing"…**

[Figure: line chart of AI "Popularity" (y-axis) against "Time" (x-axis, ticks 1950, 1956, 1974, 1980, 1987, 1993). The curve rises from 1950 ("Birth" at 1956) to a local peak "Inflated Hype" (1974), dips to "AI winter I" (1980), rises sharply to a peak "New Hopes" (1987), dips to "AI winter II" (1993), then climbs steeply to "Explosive Growth".] **Timeline of AI Development**

- **1950s–1960s**: First AI boom — the age of reasoning, prototype AI developed
- **1970s**: AI winter I
- **1980s–1990s**: Second AI boom: the age of Knowledge representation (appearance of expert systems capable of reproducing human decision-making)
- **1990s**: AI winter II
- **1997**: Deep Blue beats Gary Kasparov
- **2006**: University of Toronto develops Deep Learning
- **2011**: IBM's Watson won Jeopardy
- **2016**: Go software based on Deep Learning beats world's champions

S1.17 **[untitled: AI vs. ML vs. deep learning]**

*[Figure: (left) nested-ellipse Venn diagram: the outer ellipse **Artificial Intelligence** ("Rule-based techniques, expert systems…") contains **Machine Learning** ("CART, SVM…"), which contains **Deep Learning** ("Neural networks…").]*

S1.18 **[untitled: New Yorker quote — how a child vs. a program learns "dog"]**

[long magazine quotation, paraphrased here:] An old-fashioned program to identify a dog would be written by a software engineer as thousands of if-then-else rules (*if* it has ears, a snout, *and* hair, and *is not* a rat…). A child instead learns by seeing dogs and being told they are dogs, makes mistakes (thinks a wolf is a dog), is corrected, and gradually shifts her understanding. The machine-learning algorithm, like the child, pulls information from a training setthat has been classified ("here's a dog, and here's not a dog"), extracts featuresfrom one set versus the other, and by testing itself against hundreds and thousands of classified images it creates its own way to recognize a dog. It just knows *how* to do it.
Link: `https://www.newyorker.com/magazine/2017/04/03/ai-versus-md` *[Figure: photo of a white wolf next to a dog with similar face markings (hard to tell apart), and a grid of brain MRI scans (the article is about AI in medical diagnosis).]*

S1.19 **[untitled: generative AI examples]**

*[Figure: collage of generative-AI products: **LENSA** (AI-generated stylised portrait avatars); **DALL·E** (OpenAI; an AI-generated photo of two teddy bears working on a computer underwater); an AI anime-filter app (a photo of a cat turned into an anime girl); the ChatGPT login screen ("Welcome to ChatGPT", Log in / Sign up); and OpenAI's **Jukebox** page (April 30, 2020), a neural net that generates music, including rudimentary singing, as raw audio in a variety of genres and artist styles (Read paper / View code buttons).]*

S1.20 **[untitled: Lee Sedol vs. AlphaGo]**

[Figure: photo of Go champion Lee Sedol, head in hand, studying a Go board during his 2016 match against AlphaGo.] Caption: Lee Sedol vs. AlphaGo in 2016, Source: The New Yorker.
`https://www.wired.com/2016/03/two-moves-alphago-lee-sedol-redefined-future/`

S1.21 **[untitled: Google self-driving car]**

[Figure: photo of Google's small white two-seat self-driving car (roof-mounted sensor dome) in a parking lot at sunset; "Image courtesy of Google".] Headline: **Google's artificial intelligence becomes first non-human to qualify as a driver**

S1.22 **Machine learning competitions**

- Netflix Challenge
- 100,000,000+ ratings from 400,000+ users of 17,000+ movies
- Predict user rating for a film, winner got $1,000,000!

[Figure: screenshot of the Netflix web interface: a row of movie posters (Due Date [highlighted with a red box], Kiss Kiss Bang Bang, Surfer, Dude, The Wedding Planner, Texas Chainsaw Massacre: Next Gen), each with an "Add" button, a 1–5 star rating (about 2–3.5 red stars) and a "Not Interested" button — the star ratings are what the competition asked teams to predict.]

S1.23 **[untitled: Kaggle competitions]**

[Figure: screenshot of the Kaggle "Competitions" web page (`https://www.kaggle.com/competitions`): left menu Create / Home / Competitions / Datasets / Code / Discussions / Courses / More; main text "Grow your data science skills by competing in our exciting competitions. Find help in the documentation or learn about Community competitions.", a "Host a Competition" button, a "Search competitions" box, filter chips All competitions / Entered / Hosted / Featured / Research, and a "Get Started" heading.]

S1.24 **Machine learning in research: Examples**

- High-energy physics experiments
- Microarrays for gene expression (20,000+ features)
- Genome-wide association studies (millions of features)

Unprecedented large scale of data
+
Highly complicated underlying structure

S1.26 **[untitled: data science workflow diagram]**

[Figure: the R for Data Science workflow diagram inside a box labelled "Program": Import → Tidy → [grey box "Understand": a cycle Transform → Visualise → Model → back to Transform] → Communicate.]

S1.27 **[untitled: tidymodels website]**

[Figure: screenshot of the tidymodels home page (`https://www.tidymodels.org/`) with package hexagon logos.] **TIDYMODELS**

The `tidymodels` framework is a collection of packages for modeling and machine learning using `tidyverse` principles.

Install tidymodels with:

```
install.packages("tidymodels")
```

S1.28 **[untitled: tidyverse/tidymodels workflow]**

*[Figure: workflow diagram with package hex logos: a top box with the **tidyverse** logo containing Import → Tidy → Transform → Visualize; an arrow from Visualize goes down to a yellow box with the **tidymodels** logo containing **Model**, which loops back up to Transform; an arrow from Model goes down to a bottom box with the **rmarkdown** and **Shiny** logos labelled **Share**.]* Links: `https://hansjoerg.me/2020/02/09/tidymodels-for-machine-learning/` `https://rviews.rstudio.com/2019/06/19/a-gentle-intro-to-tidymodels/`

7.2 The learning set-up: response, predictors, f and error

S1.30 **Example data:**

- At the observation level:

$$\underset{\text{wage}}{\begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_9 \end{pmatrix}} \qquad \underset{\text{age, education, year}}{\begin{pmatrix} x_{11}, x_{12}, x_{13} \\ x_{21}, x_{22}, x_{23} \\ x_{31}, x_{32}, x_{33} \\ \vdots, \vdots, \vdots \\ x_{91}, x_{92}, x_{93} \end{pmatrix}}$$

[note: on the slide "wage" is the label above the y vector (orange) and "age, education, year" the label above the x matrix (blue).]

S1.31 **Example data: (one observation)**

- At the observation level:

$$\text{John} \underset{\text{wage}}{\begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_9 \end{pmatrix}} \qquad \underset{\text{age, education, year}}{\begin{pmatrix} x_{11}, x_{12}, x_{13} \\ x_{21}, x_{22}, x_{23} \\ x_{31}, x_{32}, x_{33} \\ \vdots, \vdots, \vdots \\ x_{91}, x_{92}, x_{93} \end{pmatrix}}$$

[Figure: same layout as S1.30, but the label "John" is placed next to row 3: y_3 is highlighted red and the whole row x_{31}, x_{32}, x_{33} is highlighted blue — one row = one observation (one employee's wage and his age, education, year).]

S1.32 **Example data: (one predictor)**

- At the observation level:

$$\underset{\text{wage}}{\begin{pmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_9 \end{pmatrix}} \qquad \underset{\text{age, education, year}}{\begin{pmatrix} x_{11}, & x_{12}, & x_{13} \\ x_{21}, & x_{22}, & x_{23} \\ x_{31}, & x_{32}, & x_{33} \\ & \vdots, \vdots, \vdots & \\ x_{91}, & x_{92}, & x_{93} \end{pmatrix}}$$

*[Figure: the whole 2nd column $x_{12}, x_{22}, \ldots, x_{92}$ is highlighted blue, with the caption below it "**all 9 employees' education levels**" — one column = one predictormeasured on all observations; x_{ij} = observation i, predictor j.]*

S1.34 **[untitled: Regression vs. Machine Learning cartoon]**

[Figure: cartoon (credit: Chelsea Parlett Pelleriti). Two identical walking scatterplots with a fitted upward line through the dots: the left one is labelled "REGRESSION"; the right one wears a top hat, a monocle and carries a cane and is labelled "MACHINE LEARNING". Joke: ML is often just regression dressed up fancily.]

S1.35 **Evaluate: How well did the machine learn?**

- Need to give the model observed output and input

Learner	Supervisor
College student	Answer key
Machine	Raw observed output

S1.38 **[untitled: error of one test prediction]**

*[Figure: yam/speed plot with only the black dashed line and the four red **Testing Data** points. For the left-most test point: a red X on the x-axis marks its yam amount; a dashed line goes up to the line and left (purple) to a red X on the y-axis = the predicted speed $\hat{f}(\tilde{x})$, while a green dashed line from the actual point goes left to a higher red X = the observed speed $\tilde{y}$. A black vertical bar (error bar) between the point and the line marks the prediction error $\tilde{y} - \hat{f}(\tilde{x})$ (the residual for that test point).]*

S1.45 **[untitled: map of supervised and unsupervised methods]**

*[Figure: hand-lettered blob diagram (credit: Chelsea Parlett Pelleriti). A large peach region **SUPERVISED** contains: a purple blob "CONTINUOUS OUTCOME" with **Linear regression**; a pink blob "CATEGORICAL* OUTCOME" with **Logistic regression**, **Decision tree**, **Naive bayes**, **K nearest neighbors**; an orange blob "REGULARIZATION" with **Ridge** and **LASSO**. A large grey-blue region **UNSUPERVISED** contains a blue blob "CLUSTERING" with **DBSCAN**, **Hierarchical clustering**, **K-means**, **Gaussian Mixtures (EM)**. A grey "DIMENSION REDUCTION" blob straddles both regions, overlapping **LASSO** (supervised side) and containing **PCA** (unsupervised side).]*

S1.48 **[untitled: Machine Learning mind map]**

*[Figure: mind map with a central grey bubble **Machine Learning** and three branches. **Unsupervised Learning** (teal) →* Dimensionality Reduction *(Meaningful Compression, Structure Discovery, Big data Visualisation, Feature Elicitation) and* Clustering *(Recommender Systems, Targetted Marketing, Customer Segmentation). **Supervised Learning** (red) →* Classification *(Image Classification, Customer Retention, Identity Fraud Detection, Diagnostics) and* Regression *(Advertising Popularity Prediction, Weather Forecasting, Market Forecasting, Estimating life expectancy, Population Growth Prediction). **Reinforcement Learning** (orange) → Real-time decisions, Game AI, Robot Navigation, Skill Acquisition, Learning Tasks.]*

7.3 Parametric vs. non-parametric; uses of models

S1.51 **General comments**

- It's important to:
- understand the ideas behind these methods
- understand the simpler methods first
- accurately evaluate the performance of a method

7.4 Bias–variance tradeoff: flexibility, training MSE and test MSE

S1.54 **[untitled: model flexibility/complexity]**

*[Figure: four 3-D perspective plots (the ISLR ch. 2 `Income` example) with axes **Years of Education**, **Seniority** (horizontal) and **Income** (vertical); red dots are the observations, with short black vertical segments joining each dot to the surface (the errors). Top: the blue surface = the true f (a smooth, gently wavy sheet rising with education and seniority). Bottom row, from left to right under a long arrow labelled "**model flexibility/complexity**": yellow fitted surfaces $\hat{f}$ — (1) a flat plane (linear model; misses the curvature, many visible error segments); (2) a moderately smooth surface close to the true blue f; (3) a very rough, wiggly surface that bends to pass through (almost) every red point, i.e. zero training error but far from the true f (overfitting).]*

8 Slide Set 1 (cont.) — pp. 59–116

[Slides/parts already covered in Part I are omitted: S1.59–60, S1.66, S1.68–80, S1.82, S1.85–96, S1.99–102.]

8.1 Training MSE vs. test MSE; overfitting

S1.61 **Lowest training MSE $\neq$ lowest test MSE**

*[Figure: Scatterplot (ISLR Fig. 2.9 left): about 50 open black circles, X from 0 to 100 (horizontal), Y from about 3 to 11 (vertical axis ticks 2–12). Four fitted curves: **black** = the true f (rises from ≈4.5 at $X = 0$ to a peak ≈9.4 near $X = 70$, then falls to ≈6.3 at $X = 100$); **orange** = linear regression straight line (from ≈4.5 to ≈9.9); **light blue** = a simple smoothing spline, close to the black curve; **green** = a complicated smoothing spline that wiggles up and down to chase individual points.] [Figure: Colour key below the plot, placed along a long right-pointing arrow labelled "**level of model flexibility**": Linear regression (least flexible), True f (black), smoothing spline (simple), smoothing spline (complicated) (most flexible).]*

S1.62 **[untitled — training vs. test MSE as flexibility grows]**

*[Figure: Left: the same scatterplot and four fits as S1.61 (smaller), with the same colour key and "level of model flexibility" arrow underneath (Linear regression, True f, smoothing spline (simple), smoothing spline (complicated)). Right: plot of **Mean Squared Error** (vertical, 0.0–2.5) vs. **Flexibility** (horizontal, log scale, ticks 2, 5, 10, 20). **Red** curve labelled "Test MSE" is U-shaped: ≈2.12 at flexibility 2 (orange square = linear regression), minimum ≈1.13 at flexibility ≈5.5 (blue square = simple spline), ≈1.53 at flexibility ≈25 (green square = complicated spline), then shoots up to ≈2.4. **Grey** curve labelled "Training MSE" decreases monotonically: ≈1.73 (orange square), ≈0.87 (blue square), ≈0.38 (green square), ≈0.12 at the right end. A horizontal*

dashed line at MSE $= 1.0$ (the irreducible error $\mathrm{Var}(\epsilon)$; unlabelled) — test MSE never goes below it, training MSE does.]

S1.63 **[untitled — example 2: true f nearly linear]**

*[Figure: Left: scatterplot of Y (2–13) vs. X (0–100); the true f (black) is almost linear, rising from ≈2.7 to ≈12.7. Orange = linear regression line (from ≈2 to ≈12.5) lies almost on top of it; light blue = simple smoothing spline, also close; green = complicated smoothing spline, very wiggly. Right: Mean Squared Error (0.0–2.5) vs. Flexibility (log scale 2, 5, 10, 20). Red test-MSE curve: ≈1.09 at flexibility 2 (orange square), minimum ≈1.06 at ≈3 (blue square), ≈1.52 at ≈25 (green square), rising to ≈2.35. Grey training-MSE curve: ≈0.99 (orange), ≈0.91 (blue), ≈0.39 (green), ≈0.13 at the end. Dashed line at 1.0. Here the inflexible linear fit is already nearly optimal.] [Figure: Colour key under the plots with the arrow "**level of model flexibility**": Linear regression (orange), True f (black), smoothing spline (simple) (blue), smoothing spline (complicated) (green).]*

S1.64 **[untitled — example 3: true f highly nonlinear]**

[Figure: Left: scatterplot of Y (-10 to 25) vs. X (0–100). True f (black) is strongly nonlinear: ≈27 at $X = 0$, drops to ≈11 near $X = 30$, small bump ≈13.8 near $X = 65$, then plunges to ≈ -11 at $X = 100$. The orange linear-regression line (from ≈20 down to ≈3) fits badly; the light-blue and green splines both follow the data closely (green slightly wiggly). Right: Mean Squared Error (0–20) vs. Flexibility (log scale 2, 5, 10, 20). Test MSE (red) ≈18.8 and training MSE (grey) ≈14.8 at flexibility 2 (orange squares); both fall steeply between flexibility 3 and 6; at flexibility ≈10 (blue square) test ≈1.2, training ≈0.8; at ≈25 (green squares) test ≈1.5, training ≈0.4; test MSE turns up slightly at the right end (≈2.5). Dashed line at 1.0. Here a flexible method is needed: the linear fit has huge error (high bias).]

S1.65 **Overfitting = Small train MSE, large test MSE**

[Figure: 3×3 grid of cartoons; columns "Under-fitting" / "Optimal-fitting" / "Over-fitting", rows "Regression" / "Classification" / "Deep learning". Regression row: blue points following a U-shape; under-fitting = a straight downward-sloping line missing the U; optimal = a smooth parabola through the cloud; over-fitting = a jagged black polyline that zig-zags through every point. Classification row: blue and red points (red cluster on the right, a few red points among the blue); under-fitting = a straight line boundary that misclassifies many; optimal = a smooth parabola-shaped boundary enclosing the red cluster; over-fitting = a tortuous, snaking boundary that loops around individual points. Deep learning row: Error (vertical) vs. Epochs (horizontal), blue "Validation" and red "Training" curves; under-fitting = both curves decrease and level off together at a fairly high error; optimal = both curves decrease, validation slightly above training, both low; over-fitting = training error keeps falling to near 0 while validation error levels off much higher (large gap).]

S1.67 **[untitled — overfitting memes]**

[Figure: Two memes. Left: a photo of a wooden bed frame with a mattress bent into the curvy shape of a body lying on it; caption "THE BEST WAY TO EXPLAIN OVERFITTING". Right: the Futurama "Fry squinting" meme, captioned "NOT SURE IF 99% ACCURACY… OR OVERFITTING" (memegenerator.net).]

8.2 Bias-variance decomposition

S1.81 **Bias-variance tradeoff**

[Figure: ISLR Fig. 2.12 (left) style plot: horizontal axis Flexibility (log scale 2, 5, 10, 20), vertical axis 0.0–2.5. Dark-red curve "Test MSE" is U-shaped (≈2.12 at flexibility 2, minimum ≈1.13 near flexibility 5.5, up to ≈2.43 at the right end). Orange curve "Variance" starts near 0.05 and increases, slowly at first then sharply, to ≈1.42. Cyan curve "Bias^2" starts at ≈1.08 and drops quickly to ≈0 by flexibility ≈5.5, then stays flat near 0. Horizontal dashed line at 1.0 (= $\mathrm{Var}(\varepsilon)$); vertical dotted line at flexibility ≈5.5 where the test MSE is smallest. Test MSE = Variance + Bias^2 + 1.0 at every flexibility.]

S1.83 **[untitled — example 2: data, MSE, and its decomposition]**

[Figure: Three panels for the second example (true f nearly linear, S1.63). Right: decomposition — dark-red test MSE starts ≈1.09, reaches its minimum ≈1.06 at flexibility ≈3 (vertical dotted line), then rises to ≈2.35; cyan squared bias is tiny everywhere (≈0.05 at flexibility 2, ≈0 afterwards); orange variance rises from ≈0.04 to ≈1.33; dashed line at 1.0. With little bias to remove, the optimum is at low flexibility and variance dominates.]

S1.84 **[untitled — example 3: data, MSE, and its decomposition]**

[Figure: Three panels for the third example (true f highly nonlinear, S1.64). Right: decomposition with legend "MSE" (dark red), "Bias" (cyan), "Var" (orange): MSE falls from ≈18.8 to ≈1.2 between flexibility 2 and 6 and stays low, turning up to ≈2.5 at the end; squared bias falls from ≈17.7 to ≈0 by flexibility ≈8; variance is ≈0 until it rises to ≈1.5 at the right end; vertical dotted line at flexibility ≈10 (the minimum); dashed line at 1.0. Here bias dominates, so the optimum is at higher flexibility.]

8.3 Classification setting: error rates and the Bayes classifier

S1.97 **[untitled — Bayes classifier, two classes]**

[Figure: ISLR Fig. 2.13 (simulated two-class data). Axes X_1 (horizontal) and X_2 (vertical), no tick values. About 100 orange circles (class 1) mostly in the upper left and about 100 blue circles (class 2) mostly in the lower and right part, with considerable overlap in the middle. A purple dashed curve — the Bayes decision boundary — winds from the left edge (about 1/3 up from the bottom), dips down around the lower middle, rises up through the centre, bulges left, and exits through the top edge right of centre. The background is a grid of small dots coloured orange on the side where the Bayes classifier predicts orange (upper left) and blue where it predicts blue (lower part and right side). Several orange circles lie in the blue region and vice versa.]

S1.98 **[untitled — reading the Bayes classifier plot]**

Each circle is an observation
location: predictor X
color: class label Y, $Y = 1$ vs. $Y = 2$

Areas:

$$\mathrm{Prob}(Y = 1 \mid X = \mathbf{x}_0) > 50\% \qquad \mathrm{Prob}(Y = 2 \mid X = \mathbf{x}_0) > 50\%$$

9 Slide Set 2 — Linear models, feature engineering, EDA

[Slides/parts already covered in Part I are omitted: S2.1, S2.3–5, S2.7–10, S2.12–14, S2.16–17, S2.20–24, S2.26–31, S2.33–36, S2.38–44, S2.46–49, S2.51–52, S2.54, S2.56–58; S2.59–78 (diagnostics, collinearity, LR vs. KNN) are covered in Part I.]

S2.2 **Required reading:**

- For **undergraduates**, ch. 2 of ISLR
- For **graduate students**, ch. 2 of ESL

9.1 Exploratory data analysis (EDA)

S2.6 **[Missing-data patterns and missingness mechanisms]**

D: Dog (missingness mechanism)

[Figure: left: four schematic data matrices of variables Y_1, Y_2, Y_3, Y_4 drawn as tall bars, grey shading = missing values. (a) Univariate Pattern*: Y_1–Y_3 complete, only Y_4 has a missing block at the bottom. (b)* Monotone Pattern*: Y_1, Y_2 complete, Y_3 missing at the bottom third, Y_4 missing at the bottom half (each later variable missing for a superset of the rows). (c)* General Pattern*: small missing bands scattered at different rows in every variable. (d)* Planned Missingness*: Y_1 complete, Y_2 missing at the top, Y_3 in the middle, Y_4 at the bottom (missing blocks staggered by design).] [Figure: right: three hand-drawn causal diagrams (R. McElreath), each with A → H, H → H*, D → H*. (1) MCAR: no arrow into D. (2) MAR: extra arrow A → D. (3) MNAR: extra arrow H → D.]*

9.2 Linear regression

S2.11 **Linear regression**

PSTAT 126
Carl Friedrich Gauss in 1795
Simple, interpretable, and often very useful
Many nonlinear methods are direct generalization of linear regression
(1809): Method of least squares, MLE, Gaussian distribution…

S2.15 **Simple linear regression**

[Figure: base-R scatterplot of 9 counties (open circles): x-axis "population" (ticks 1000000–3000000), y-axis "confirmed" (ticks 50000–200000). Points approx. (0.89M, 73k), (0.98M, 70k), (1.52M, 70k), (1.66M, 56k), (1.93M, 75k), (2.15M, 212k), (2.41M, 200k), (3.17M, 182k), (3.31M, 171k). A blue increasing line $y = \beta_0 + \beta_1 x$ runs from about 71k at the left edge to about 210k at the right edge.]

S2.18 **Estimating parameters**

Which line should we choose? *[Figure: Riverside County (population ≈2.41M, observed confirmed y_{RS} ≈200k) vs. its predicted response $\beta_0 + \beta_1 x_{\mathrm{RS}}$ ≈157k on the line; the vertical gap is the residual.]*

S2.19 **Simple linear regression**

*[Figure: flow diagram of three light-blue boxes joined by arrows: **Data** → **Learn** → **Evaluate**. Under Data: "Training set $\{(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)\}$". Under Learn: $\hat{\beta}_0, \hat{\beta}_1$. Under Evaluate: "1. Accuracy of the coefficient estimates $\hat{\beta}_0, \hat{\beta}_1$" and "2. Accuracy of the linear model".]*

S2.25 **Accuracy of the coefficient estimates**

*[Figure: two panels (ISLR Fig. 3.3 style), both with x-axis X (-2 to 2) and y-axis Y (-10 to 10). Left: about 100 simulated points (open circles) with a positive trend and large scatter; a **red** line $Y = \beta_0 + \beta_1 X$ (the true population line, ≈ $2 + 3X$) and a **dark blue** line $Y = \hat{\beta}_0 + \hat{\beta}_1 X$ (the least-squares fit to these data, slightly steeper). Right: the same red and dark-blue lines plus about ten **light-blue** lines $Y = \hat{\beta}_0 + \hat{\beta}_1 X$,*

"each computed on the basis of a separate random set of observations"; the light-blue lines scatter around the red line, with slopes varying.]

9.3 Multiple linear regression

S2.32 **Coefficient estimates**

ith residual: $y_i - \hat{y}_i = y_i - \hat{\beta}_0 - \sum_{j=1}^{p} \hat{\beta}_j x_{ij}$ *[Figure: 3-D plot (ISLR Fig. 3.4) with axes X_1, X_2 (floor) and Y (vertical): a tilted blue-to-green least-squares regression plane fitted to about 30 red points; each red point is joined to the plane by a vertical black segment (its residual), some points above and some below the plane.]*

S2.37 **[Forward / backward stepwise selection example]**

Forward stepwise selection example with 5 variables: **Backward stepwise selection example with 5 variables:**

[Figure: forward: Null Model (no variables) → add the most significant, X_2 (Model with 1 variable) → add X_5 (Model with 2 variables $\{X_2, X_5\}$), "until reaching the stopping rule or running out of variables". Backward: Full Model $\{X_1, \ldots, X_5\}$ → remove the least significant, X_4 (Model with 4 variables) → remove X_1 (Model with 3 variables $\{X_2, X_3, X_5\}$).]

9.4 Practical considerations: qualitative predictors

S2.45 **tidymodels: Create traditional dummy variables**

`tidymodels` — screenshot of the `recipes` reference page **"Create traditional dummy variables"** (Source: `R/dummy.R`):

"`step_dummy()` creates a *specification* of a recipe step that will convert nominal data (e.g. character or factors) into one or more numeric binary model terms for the levels of the original data."

9.5 Extensions of the linear model: interactions, polynomial regression

S2.50 **Interactions vs. main effects**

*[Figure: three schematic interaction plots, "Cleanliness" vs. "B: Water Temperature" (Cold, Warm, Hot), one line per "A: Detergent" #1/#2. (1) **A Significant, B Not**: two horizontal flat lines, Detergent #1 clearly above #2. (2) **A and B Significant**: two parallel rising lines, constant gap. (3) **B Significant, A Not**: two almost coincident rising lines.]*

S2.53 **Interaction: Credit example**

[Figure: two panels (ISLR Fig. 3.7), x-axis Income (0–150), y-axis Balance (200–1400); red line = student, black line = non-student. Left (no interaction): student from ≈600 at income 0 to ≈1490 at 150; non-student from ≈215 to ≈1105. Right (with interaction): student line from ≈680 to ≈1310 (flatter), non-student from ≈205 to ≈1130 (steeper).]

S2.55 **Non-linear relationships**

*[Figure: ISLR Fig. 3.8, Auto data: mpg (Y, 10–50) vs. horsepower (X, 50–230), ≈400 points with a curved, decreasing, convex trend. **Orange** = Linear (from ≈34 at $X = 40$ down to ≈6 at $X = 215$ — too high in the middle, too low at the right); **light blue** = Degree 2 (from ≈41 down to a minimum ≈13 near $X = 190$, slightly rising after); **green** = Degree 5 (bump ≈35 near $X = 50$, dips to ≈11.5 near $X = 205$, then shoots up to ≈22 at the right edge).]*

10 Slide Set 3 — Logistic regression & model evaluation (pp. 1–42, the tested part)

[Slides/parts already covered in Part I are omitted: S3.1–4, S3.6–7, S3.9–25, S3.28–29, S3.32–34, S3.37–38, S3.41–42; on the kept slides only the details missing from Part I remain.]

10.1 Overview: classification

S3.5 **Why not Linear Regression?**

[Figure: Probability of Default *vs.* Balance *(0–2500). **Left, "Linear Regression"**: the fitted blue straight line starts below zero (about -0.1 at balance 0), crosses 0 near balance 500–600 and only reaches about 0.28 at balance 2700. **Right, "Logistic Regression"**: the blue fitted curve is S-shaped (sigmoid): essentially 0 up to balance ≈ 1000, rising steeply between ≈ 1500 and ≈ 2200 (0.5 at about 1950–2000), and levelling off near 0.98 at 2650.]*

S3.8 **A data example**

*[Figure: three panels (the ISLR `Default` data). **Left**: scatterplot of* Income *(y, 0 to ≈ 70000) vs.* Balance *(x, 0 to ≈ 2700); the defaulters (orange "+") sit almost entirely at balance $\gtrsim 1200$, most of them 1500–2500, at every income level. **Middle**: boxplots of* Balance *by* Default*: No has median ≈ 800, box ≈ 470–1130, whiskers 0 to ≈ 2120 and high outliers up to ≈ 2390; Yes has median ≈ 1790, box ≈ 1510–1990, whiskers ≈ 960–2650 and low outliers at ≈ 650–780. **Right**: boxplots of* Income *by* Default*: No median ≈ 34500 (box ≈ 21500–44000, whiskers ≈ 800–73500), Yes median ≈ 31500 (box ≈ 19000–43000, whiskers ≈ 10000–66500).]*

10.2 Estimating the coefficients

S3.26 **Estimating coefficients**

Maximum Likelihood Estimation (MLE)

```
Call:
glm(formula = default ~ student + balance + income,
    family = binomial, data = Default)

Deviance Residuals:
    Min       1Q   Median       3Q      Max
-2.4691  -0.1418  -0.0557  -0.0203   3.7383

Coefficients:
...
---
Signif. codes:
0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)
...
Number of Fisher Scoring iterations: 8
```

[note: the elided rows (coefficient table, null/residual deviance, AIC) are in Part I.]

10.3 Confounding

S3.27 **What is wrong?**

A simple logistic regression of **default** on **student**

	Coefficient	Std. error	Z-statistic	P-value
`Intercept`	-3.5041	0.0707	-49.55	< 0.0001
`student[Yes]`	0.4049	0.1150	3.52	0.0004

A multiple logistic regression of **default** on **balance**, **income**, and **student**

	Coefficient	Std. error	Z-statistic	P-value
`Intercept`	-10.8690	0.4923	-22.08	< 0.0001
`balance`	0.0057	0.0002	24.74	< 0.0001
`income`	0.0030	0.0082	0.37	0.7115
`student[Yes]`	-0.6468	0.2362	-2.74	0.0062

S3.30 **[Simpson's paradox example]**

[Figure: Blue group: points $(1,6), (2,7), (3,8), (4,9)$ with a solid blue line of slope $+1$ through them. Red group: points $(8,1), (9,2), (10,3), (11,4)$ with a solid red line of slope $+1$. A black dashed line fitted to all eight points together slopes downward*, from $y \approx 9.2$ at the left edge ($x \approx -1.5$) to $y \approx 0.8$ at the right edge ($x \approx 13.5$), i.e. slope ≈ -0.56.]*

S3.31 **[Simpson's paradox: scatter with clusters]**

Korrelation: *[Figure: a ggplot-style scatterplot (title "Korrelation:" with no value shown — a frame of an animation) of about 500 black points, Y (vertical, ticks 0, 5, 10, 15) vs. X (horizontal, ticks 0, 5, 10). The points form about five overlapping clusters arranged along a diagonal from the upper left ($X \approx 1$–5, $Y \approx 7$–13) to the lower right ($X \approx 9$–13, $Y \approx 1$–6), so the overall correlation of X and Y is clearly negative, while inside each cluster the points tilt upward (positive within-cluster relationship).]*

10.4 Classification performance: confusion matrix

S3.35 **Confusion matrices (multiclass)**

		True Class: Apple	Orange	Mango
Predicted	Apple	7	8	9
	Orange	1	2	3
	Mango	3	2	1

S3.36 **Many definitions and synonyms**

A large table (Wikipedia "Receiver operating characteristic"): `https://en.wikipedia.org/wiki/Receiver_operating_characteristic`

Row ratios (right of the block):

- **False discovery rate (FDR)** $= \dfrac{\Sigma \text{ False positive}}{\Sigma \text{ Predicted condition positive}}$
- **False omission rate (FOR)** $= \dfrac{\Sigma \text{ False negative}}{\Sigma \text{ Predicted condition negative}}$

Ratios of ratios (bottom right):

- **Positive likelihood ratio (LR+)** $= \dfrac{\text{TPR}}{\text{FPR}}$; **Negative likelihood ratio (LR−)** $= \dfrac{\text{FNR}}{\text{TNR}}$
- **Diagnostic odds ratio (DOR)** $= \dfrac{\text{LR+}}{\text{LR−}}$

10.5 ROC curve and AUC

S3.39 **ROC**

[Figure: plot titled "ROC Curve": the blue step-like ROC curve rises almost vertically from $(0, 0)$ to TPR ≈ 0.5 at FPR ≈ 0.01, reaches ≈ 0.8 at FPR ≈ 0.07, ≈ 0.9 at FPR ≈ 0.15, ≈ 0.95 at FPR ≈ 0.25, ≈ 0.99 by FPR ≈ 0.4 and 1.0 from FPR ≈ 0.6 on: a curve hugging the top-left corner (an excellent classifier, AUC close to 1).]

S3.40 **[ROC curve examples]**

*[Figure: **Left** (hand-drawn "xkcd" style): three ROC curves — green just above the diagonal (weak), orange in between, blue highest (jumps to TPR ≈ 0.45 almost immediately, ≈ 0.7 at FPR 0.2, ≈ 0.85 at FPR 0.5). **Right**: A (purple, AUC 0.923) rises steeply to TPR ≈ 0.85 by FPR ≈ 0.12 then flattens towards 1; B (blue, AUC 0.794) starts with a jump to ≈ 0.17 at FPR 0 (so it is above A for FPR $\lesssim 0.03$), then rises more gradually (≈ 0.6 at FPR 0.2, ≈ 0.8 at 0.5); C (red, AUC 0.588) rises slowly and almost linearly (≈ 0.4 at FPR 0.2, ≈ 0.55 at 0.4, ≈ 0.87 at 0.8).]*

11 Slide Set 3 (cont.) — Discriminant analysis: LDA & QDA (pp. 43–93; past the Slide-42 cutoff)

[Slides/parts already covered in Part I are omitted: S3.43–44, S3.46, S3.48–51, S3.53, S3.56, S3.58–60, S3.62, S3.65–74, S3.76–79, S3.81–84, S3.86–93; on the kept slides only the details missing from Part I remain.] [note: these pages lie past the official "up until Slide 42" midterm cutoff, but are included because practice exam Q20, Lab 3 and HW 3 test LDA/QDA.]

11.1 Motivation: why LDA?

S3.45 **Why bother considering LDA?**

```
> x <- seq(100)
> y <- c(rep(0, 50), rep(1, 50))
> fit <- glm(y~x, family = "binomial")
```

S3.47 **[untitled — feature extraction vs feature selection]**

[Figure: Two-panel graphic with no other text. Left panel (dark slate background), titled "feature extraction": a row of 5 diamonds, a red down-arrow, a row of 8 circles, a red arrow, a row of 6 circles, a red arrow, and finally a row of 3 glowing circles — the original features are transformed/combined into a small number of new *features. Right panel (light teal background), titled "feature selection": a single row of 7 dark circles, 3 of which (the 2nd, 5th and 6th) are crossed out with red X's — a subset of the* original *features is kept and the rest are dropped.][note: LDA is a feature-extraction method — it builds new axes (LD1, LD2, …) from combinations of the predictors; see S3.64–67, S3.80–85.]*

11.2 The geometric picture: the gene-expression example

S3.52 **LDA: A motivating example**

[Figure: Number line titled "Transcription for Gene X" ("Fewer Transcripts" ← / → "More transcripts"). Twelve dots from left to right: green, green, green, green, red, green, red, green, red, red, red, red (6 green, 6 red). Legend: green dot "= The drug works!"; red dot "= The drug does not work :(".]

`https://youtu.be/azXCzI57Yfc` *(link at the bottom of the slide)*

S3.54 **LDA: A motivating example**

[Figure: Same Gene X number line; the four leftmost green dots are circled in red (left callout) and the four rightmost red dots are circled in red (right callout).]

- Left callout: "For the most part, the drug works in people with low transcription of gene X."
- Right callout: "And, for the most part, the drug does not work in people with high transcription of gene X."

S3.55 **LDA: A motivating example**

[Figure: Same Gene X number line; now the middle section (red, green, red, green) is circled in red, with an arrow from the callout text:]

> "However, there is overlapand no obvious 'cutoff' for who to give the drug to."

S3.57 **LDA: A motivating example**

> In summary, Gene X does an OK job at telling us who should take the drug (and who shouldn't).
> Can we do better? What if we used more than one gene?

S3.61 **LDA: A motivating example**

[Figure: 2D scatterplot: "Gene X" (horizontal) vs. "Gene Y" (vertical). 6 green points (drug works) lie mostly toward the lower-left; 6 red points (drug does not work) lie mostly along the top/upper-right, plus one red at the lower right. A black dashed line running diagonally from the top down to the lower right separates the green points (below-left) from the red points (above-right) — a linear decision boundary.]

> Two genes do a better job separating the two categories.
> Would three genes be even better?

S3.63 **LDA: A motivating example**

[Figure: 3D plot (Gene X, Gene Y, Gene Z; point size suggests depth) with a translucent light-blue plane cutting through the cloud; an arrow points to it from the callout text:]

> With 3 dimensions, we use a plane to try and separate the two categories.

S3.64 **LDA: A motivating example**

Reducing a 2-D graph to a 1-D graph

[Figure: Left, "A 2-D graph (aka X/Y Graph)": the Gene X / Gene Y scatterplot. Arrow → "A 1-D graph (aka number line)": points (left to right) green, green, green, green, green, then red, red, green, red, then red, red, red.]

S3.75 **[untitled — distance between means alone vs means and scatter]**

[Figure: Left: a 2D scatterplot: 6 red points in an upper band and 6 green points in a lower band, both bands rising gently to the right (separated vertically, each spread out horizontally). Top right: the points are projected onto a line rising with the bands; the class means are far apart, but in the middle of the line green and red points are interleaved. Bottom right: the points are projected onto a steep, nearly vertical line; the red points cluster tightly near the top and the green points tightly near the bottom, with no overlap.]

- If we only maximize the distance between means… The separation isn't great…
- However, if we optimize the distance between means and scatter… We get nice separation!

S3.80 **[untitled — LDA with three genes]**

[Figure: Left: the 3D Gene X / Gene Y / Gene Z plot with a solid black line drawn diagonally through the cloud. Right: all points lie on the diagonal line: 6 green points on the lower-left part and 6 red points on the upper-right part, completely separated.]

- Here's the new axis that LDA created.
- The data are projected onto the new axis.

S3.85 **[untitled — LDA plot: LD1 vs LD2]**

[Figure: R scatterplot titled "LDA": horizontal axis "LD1" (ticks -4 to 2), vertical axis "LD2" (ticks -2 to 2), three classes: red points on the left (LD1 from about -4.2 to -0.8, LD2 around -1 to 1), black points at upper right (LD1 0 to 2, LD2 -0.7 to 2.6), blue points at lower right (LD1 -0.2 to 2.2, LD2 -2.7 to 0), with a few blue points mixed in among the black ones — LD1 separates red from the rest; LD2 separates black from blue.]

12 Reading — ROC Curves Deep Dive

[Parts already covered in Part I are omitted: data loading / factor / `coin_flip` / 70–30 stratified split code, Recipe B, both workflows, training-accuracy code and output (0.807 / 0.616), re-fit on test, all confusion-matrix heatmaps (Model A at 0.5 and 0.00131; Model B at 0.5 and 0.4076525) with their TPR/FPR arithmetic and plotted points, TPR/FPR definitions, Model B AUC output (0.496) and accuracy discussion, `roc_curve()` needing probabilities, AUC = discrimination, the two uses of ROC curves, AUC $= 1 \Rightarrow$ leakage.]

ROC Curves: An Overview PSTAT 131/231 *[note: HTML page title; the Canvas name of this reading is "ROC Curves Deep Dive".]*

12.1 Data

Recipe A:

```
titanic_recipe_a <- recipe(survived ~ pclass + sex + age +
                           sib_sp + parch + fare, titanic_train)
    ↪ %>%
  step_impute_linear(age, impute_with = imp_vars(sib_sp)) %>%
  # choice of predictors to impute with is up to you
  step_dummy(all_nominal_predictors()) %>%
  step_interact(~ starts_with("sex"):age + age:fare)
```

12.2 Model A: ROC Curve

[Figure: Model A ROC curve (`autoplot()` of `roc_curve()`): square panel, x-axis "1 - specificity" 0.00–1.00, y-axis "sensitivity" 0.00–1.00, dotted 45° diagonal reference line from (0,0) to (1,1). The black step curve starts at (0,0), shoots straight up to sensitivity ≈0.47 at FPR 0, then ≈0.58 by FPR ≈0.04, ≈0.73 by FPR ≈0.07, ≈0.80 by FPR ≈0.14, ≈0.85 by FPR ≈0.26, then flattens: ≈0.875 around FPR 0.43–0.49, ≈0.93 at 0.55, ≈0.95 at 0.59–0.74, ≈0.97 at 0.79, reaching 1.00 at FPR ≈0.85 and running flat to (1,1). The curve bows far toward the top-left corner, well above the diagonal (good discrimination; Model A's AUC is not printed in the reading).]

Here is a table of the threshold values, sensitivity, and specificity:

```
# A tibble: 254 x 3
   .threshold specificity sensitivity
        <dbl>       <dbl>       <dbl>
 1 -Inf             0               1
 2    0.00131       0               1
 3    0.00474       0.0121          1
 4    0.00563       0.0182          1
 5    0.00635       0.0242          1
 6    0.0116        0.0303          1
 7    0.0117        0.0364          1
 8    0.0122        0.0424          1
 9    0.0147        0.0485          1
10    0.0186        0.0545          1
# ... with 244 more rows
```

Notice that, by default, the `roc_curve()` function tries values for `threshold` in a grid from `-Inf`, or basically 0, to `Inf`, or basically 1. Let's look at the first few predicted class probabilities Model A has generated:

```
log_a_results %>%
  select(.pred_Yes, survived) %>%
  head()
```

```
# A tibble: 6 x 2
  .pred_Yes survived
      <dbl> <fct>
1   0.0672  No
2   0.268   No
3   0.792   Yes
4   0.00474 No
5   0.247   Yes
6   0.693   Yes
```

Let's look at the default threshold value for a binary classification problem, and see where that falls on the curve. Generally, the default threshold value is at 0.50. We'll specifically classify the predictions as No if $p < .50$ and Yes otherwise:

```
augment(log_test_a, new_data = titanic_test) %>%
  mutate(.pred_class = factor(if_else(.pred_Yes < 0.5, "No", "Yes"
    ↪ ), levels = c("Yes", "No"))) %>%
  conf_mat(truth = survived, estimate = .pred_class) %>%
  autoplot(type = "heatmap")
```

On the ROC curve, that is this point:

```
log_a_results %>%
  roc_curve(survived, .pred_Yes) %>%
  autoplot() +
  geom_point(aes(x = 0.1212121, y = 0.776699), colour = spec_
    ↪ colors[3], size = 2.5)
```

And we could do the same exact thing for threshold values of 0.25, 0.75, etc. – for basically any value between 0 and 1. Our goal is to develop a model that is able to maximize sensitivity while minimizing 1 - specificity, or that can maximize the true positive rate while minimizing the false positive rate.

Here is a distribution of the predicted probabilities:

```
log_a_results %>%
  ggplot(aes(x = .pred_Yes, fill = survived)) +
  geom_histogram(position = "dodge") +
 theme_bw() +
  xlab("Probability of Yes") +
  scale_fill_manual(values = c("blue", "orange"))
```

[Figure: Model A histogram of `.pred_Yes` on the test set, bars dodged side by side; x "Probability of Yes" 0.00–1.00, y "count" 0–35; fill legend "survived": Yes = blue, No = orange. Orange (truly No) bars pile up at low probabilities (tallest ≈35 near 0.08; others 9, 18, 19, 13, 15, 13, 7 between 0 and 0.3) and are small (≤4) above 0.5; blue (truly Yes) bars are mostly at high probabilities (11 near 0.68; 9–11 each at 0.88–0.98), with only a few low ones (max 7 near 0.1). Clear separation with some overlap in the middle.]

```
log_a_results %>%
  ggplot(aes(x = .pred_Yes, fill = survived)) +
  geom_boxplot() +
theme_bw() +
  xlab("Probability of Yes") +
  scale_fill_manual(values = c("blue", "orange"))
```

[Figure: Model A horizontal boxplots of `.pred_Yes` by `survived` (x "Probability of Yes" 0–1; the y-axis, -0.2 to 0.2, is meaningless because no y aesthetic was given). No (orange, upper box): box ≈0.08–0.25, median ≈0.12, whiskers ≈0 to ≈0.49, outlier dots ≈0.51–0.81. Yes (blue, lower box): box ≈0.54–0.93, median ≈0.79, whiskers from ≈0.04 to ≈1.0. The two boxes do not overlap.]

12.3 Model B: ROC Curve:

For models that do not predict the outcome as well, it basically doesn't matter where you set the threshold value – that model will NEVER achieve a true positive rate of 1 and a false positive rate of 0 (which a "perfect" classifier can do). *[Figure: Model B ROC curve (axes as before: x "1 - specificity", y "sensitivity", dotted diagonal). Only a few corner points, so it is piecewise-linear: a straight segment from (0,0) to ≈(0.715, 0.689) lying just* below *the diagonal, then up to ≈(0.909, 0.951) crossing slightly above it, a short jump to ≈(0.921, 1.00), then flat to (1,1).]*

Let's look at the ROC table of threshold values:

```
# A tibble: 9 x 3
  .threshold specificity sensitivity
       <dbl>       <dbl>       <dbl>
1   -Inf           0           1
2      0.113       0           1
3      0.193       0.0182      1
4      0.228       0.0364      1
5      0.267       0.0545      1
6      0.311       0.0788      1
7      0.358       0.0909      0.951
8      0.408       0.285       0.689
9    Inf           1           0
```

[Figure: Model B histogram of `.pred_Yes` (dodged; x "Probability of Yes" 0.1–0.42, y "count" 0–120; Yes = blue, No = orange). Only a handful of distinct predicted values (one per value of `sib_sp`), approx. counts: ≈0.113: No 3; ≈0.193: No 3; ≈0.228: No 3; ≈0.267: No 4; ≈0.311: Yes 5, No 2; ≈0.358: Yes 27, No 32; ≈0.408: Yes 71, No 118 (consistent with the ROC table). Both classes sit at the same few values — no separation; nothing above ≈0.41.]

The threshold values are selected as a grid over the range of observed predicted probabilities. This model has not predicted any observation's probability of Yes as greater than $0.4076525\ldots$ which, coincidentally, is the largest threshold value considered in its ROC curve. That's because considering any larger values for the threshold would give the same results as `Inf`. We can also view this information as a boxplot, which really serves to illustrate the overlap: *[Figure: Model B horizontal boxplots of `.pred_Yes` (x "Probability of Yes" 0.1–0.4; meaningless y-axis -0.2 to 0.2). No (orange, top) and Yes (blue, bottom) boxes are essentially identical: box ≈0.358–0.408 with the median at the upper edge (≈0.408), left whisker to ≈0.31; the No group also has outlier dots at ≈0.113, 0.193, 0.228, 0.267. Complete overlap of the two classes.]*

12.4 Conclusions

Once you've settled on a model, you can also use the ROC curve to determine the optimal threshold(s). This is arguably a less common use, though, because the default threshold(s) typically works well and provides a good balance between sensitivity and specificity. But if you are working on a project where you want to be extra careful to accurately predict **every** 0 or, vice versa, every 1, it can be advisable to adjust the threshold in the desired direction.

If you are working with a multiclass outcome, where there are more than two possible values, the ROC curve idea is easily extended! You can generate a curve to represent how well the model distinguishes between **each** class (we'll see examples of this in the homework and labs).

13 Lab 1 — R basics, dplyr, EDA (diamonds), data splitting

[Parts already covered in Part I are omitted: package loading / `tidymodels_prefer()` / `install.packages()` note, `?mean`/`help()`, pipe shortcut and operators, `select()`/`mutate()`/`transmute()` code and outputs, `ungroup()`, `corrplot`/boxplot/histogram code, seed rationale and split code.]
Lab 1 PSTAT 131/231. *[Rmd header:* `html_document` *with* `toc: true`, `toc_float: true`, `code_folding: show`. *Setup chunk (not shown in output):* `knitr::opts_chunk$set(echo = TRUE, message = FALSE, warning = FALSE)`*.]*

13.1 Setup

R and RStudio. All work in this course, including homework, labs, and the final project, will be conducted using *R* and *RStudio*. I understand that not all students will already be familiar with *R*, so I don't expect that everyone starts out at the same coding level. The instructional team, myself included, are here to help!

First, go to `https://www.r-project.org/` and click *Download R*. Select a CRAN mirror link. Do this **EVEN IF** you already have R installed on your machine. If you have a previous R installation, re-downloading R will *update* your copy of R to the most recent version, which often fixes many small problems.

Next, go to `https://www.rstudio.com/products/rstudio/download/` and download the **free** version of RStudio Desktop. We will almost always open and use RStudio to interact with R.

You will be working with RStudio a lot, and you'll have time to learn most of the bells and whistles RStudio provides. Think about RStudio as your "workbench". Keep in mind that RStudio is MORE than plain R. RStudio is an environment that makes it easier to work with R, while handling many of the little tasks than can be a hassle.

At this point, your TA will give a brief overview of the RStudio default four-pane layout and demonstrate how to change fonts, settings, etc.

Getting Help with R. Much of the time we spend using R involves interacting with functions. For example, to find the average of three numbers, we can call the `mean()` function:

```
mean(c(1, 2, 3))
```

```
[1] 2
```

Each function in R has its own set of arguments and possible values that these arguments accept. You will often need to look up a specific function or one of its arguments — very often! The good news is, there is a lot of R documentation out there, and it's fairly easy to get help. Or simply open your Web browser and do a search for something like `R function mean help`.

Update an .Rmd. Markdown is the language R uses to create and update documents. If you write in a Markdown file within a **code chunk**, as shown below, that text will be processed and run like R code. If you write *outside* of the code chunks, like here, that text will not be run and will appear as text. You can format it as usual, include headings, etc.!

.Rmd files are a special type of file, referred to as a dynamic document, that allows users to combine narrative (text) with R code. Because you will be turning in all your work for this course as .Rmd files and their knitted .html or .pdf file(s), it is important that you quickly become familiar with this resource.

Try updating the code in the following code chunk. Assign `2+2` to another object, called `y`. `<-` is the assignment operator in R, commonly read as "gets."

```
# This is a code chunk!
# Any uncommented text in here will be run as R code.
# For example:
x <- seq(1, 10, 1)
x
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
```

Take some time and work through the Markdown tutorial here: `www.markdown-tutorial.com`.

In Markdown, code chunks can have specific options set for them; you can also set the options for chunks in the entire document. At the top of this .Rmd, you'll see a code chunk with `opts_chunk$set()`. Any options you set inside that function will apply to all code chunks in the document. I recommend you set the options used in this file for all your assignments, along with the options at the **very** top of the document — `toc: true`, `toc_float: true`, and `code_folding: show`. You can go further and customize Markdown files as much as you like, but that's not required.

Creating an R Project. I **strongly** recommend working in R within the context of an R project (`https://bookdown.org/ndphillips/YaRrr/projects-in-rstudio.html`). It sounds complicated or unnecessary at first, but an R project — which is essentially a special working directory, designated with an (automatically created) `.Rproj` file — can make your life **much** easier, especially when working on your final project.

Your TA can now go over how to create a new project.

Working in an R project automatically sets your working directory to that project folder, rather than whatever your computer's default working directory is. That means you can readily access other .R scripts, photos, data files, etc. simply by putting them in your project folder, without having to write out lengthy file paths.

13.2 Basics of Data Processing

Now we'll take some time to go over some of the basic tools for managing data via the *tidyverse*. There are many more functions that you might find useful, and you can read more about them in *R for Data Science* (`https://r4ds.had.co.nz/`), if you're interested.

First, you'll need to install and load some packages. These include, but are not limited to: `tidyverse`, `tidymodels`, and `ISLR`. Make sure to install each of these using the `install.packages()` function and load them with `library()`.

```
library(ISLR)
```

Some packages include datasets when they are loaded. Set `eval = TRUE`and knit your .Rmd to run the following code chunk:

```
mpg
```

[note: this chunk has `eval = FALSE`, so no output is printed in the knitted lab.] Run `?mpg` to learn more about this data set.

Select observations by their value: `filter()`. Say that you are interested in selecting only those rows in `mpg` that represent Audi compact cars. The easiest way to select them is:

```
mpg %>%
  filter(class == "compact" & manufacturer == "audi")
```

```
# A tibble: 15 x 11
   manufacturer model      displ  year   cyl
   <chr>        <chr>      <dbl> <int> <int>
 1 audi         a4           1.8  1999     4
 2 audi         a4           1.8  1999     4
 3 audi         a4           2    2008     4
 4 audi         a4           2    2008     4
 5 audi         a4           2.8  1999     6
 6 audi         a4           2.8  1999     6
 7 audi         a4           3.1  2008     6
 8 audi         a4 quattro   1.8  1999     4
 9 audi         a4 quattro   1.8  1999     4
10 audi         a4 quattro   2    2008     4
11 audi         a4 quattro   2    2008     4
12 audi         a4 quattro   2.8  1999     6
13 audi         a4 quattro   2.8  1999     6
14 audi         a4 quattro   3.1  2008     6
15 audi         a4 quattro   3.1  2008     6
# (same 15 rows, columns 6-11:)
   trans    drv     cty   hwy fl    class
   <chr>    <chr> <int> <int> <chr> <chr>
 1 auto...  f        18    29 p     comp...
 2 manu...  f        21    29 p     comp...
 3 manu...  f        20    31 p     comp...
 4 auto...  f        21    30 p     comp...
 5 auto...  f        16    26 p     comp...
 6 manu...  f        18    26 p     comp...
 7 auto...  f        18    27 p     comp...
 8 manu...  4        18    26 p     comp...
 9 auto...  4        16    25 p     comp...
10 manu...  4        20    28 p     comp...
11 auto...  4        19    27 p     comp...
12 auto...  4        15    25 p     comp...
13 manu...  4        17    25 p     comp...
14 auto...  4        17    25 p     comp...
15 manu...  4        15    25 p     comp...
```

[note: the printed tibble is one 11-column table; it is split in two here only to fit the column width.]

Activities. On your own, find ways to filter the `flights` data set from the `nycflights13` package to achieve each of the following:

- Had an arrival delay of two or more hours
- Flew to Houston (IAH or HOU)
- Were operated by United, American, or Delta
- Departed in summer (July, August, and September)
- Arrived more than two hours late, but didn't leave late
- Were delayed by at least an hour, but made up over 30 minutes in flight
- Departed between midnight and 6am (inclusive)

Select specific variables or columns by their names: `select()`. Often in machine learning, we end up working with very large data sets that have a lot of columns. The `mpg` data set is pretty small, but we can still practice with it.

Activities. On your own, working with the `flights` data:

- Find as many ways as you can to select `dep_time`, `dep_delay`, `arr_time`, and `arr_delay`.
- What happens if you include the name of a variable multiple times in a `select()` call?

Create or add new variables: `mutate()`. Besides selecting existing columns, it's often useful to add new columns that are functions of existing columns. That's the job of `mutate()`.

You can see an overview of a number of useful variable creation functions here: `https://r4ds.had.co.nz/transform.html#mutate-funs`.

Activities. On your own, working with the `flights` data:

- Currently `dep_time` and `sched_dep_time` are convenient to look at, but hard to compute with because they're not really continuous numbers. Convert them to a more convenient representation of number of minutes since midnight.
- What does `1:3 + 1:10` return? Why do you think it returns this?

Create grouped summaries of data frames: `summarise()`. The last key verb function is `summarise()`. It's most useful when combined with `group_by()`, so that it produces a summary for each level or value of a variable/group. Notice what happens if used without grouping:

```
mpg %>%
  summarise(avg_hwy = mean(hwy))
```

```
# A tibble: 1 x 1
  avg_hwy
    <dbl>
1    23.4
```

This value represents the average highway mileage across *all cars in the data frame*.

We can see immediately that, while this has certainly reduced the size of the data frame, it's not very useful. Instead, we might prefer the average highway mileage by class of car, or by manufacturer. We can view these, and even `arrange()` by highway mileage:

```
mpg %>%
  group_by(class) %>%
  summarise(avg_hwy = mean(hwy)) %>%
  arrange(avg_hwy)

mpg %>%
  group_by(manufacturer) %>%
  summarise(avg_hwy = mean(hwy)) %>%
  arrange(avg_hwy)
```

```
# A tibble: 7 x 2
  class      avg_hwy
  <chr>        <dbl>
1 pickup        16.9
2 suv           18.1
3 minivan       22.4
4 2seater       24.8
5 midsize       27.3
6 subcompact    28.1
7 compact       28.3

# A tibble: 15 x 2
   manufacturer avg_hwy
   <chr>          <dbl>
 1 land rover      16.5
 2 lincoln         17
 3 jeep            17.6
 4 dodge           17.9
 5 mercury         18
 6 ford            19.4
 7 chevrolet       21.9
 8 nissan          24.6
 9 toyota          24.9
10 subaru          25.6
11 pontiac         26.4
12 audi            26.4
13 hyundai         26.9
14 volkswagen      29.2
15 honda           32.6
```

The following code finds the average highway mileage by manufacturer, counts the number of cars produced by each manufacturer, and prints the top 10 manufacturers with largest numbers of cars, arranged by mileage:

```
mpg %>%
  group_by(manufacturer) %>%
  summarise(avg_hwy = mean(hwy),
            count = n()) %>%
  filter(count >= 9) %>%
  arrange(avg_hwy)
```

```
# A tibble: 10 x 3
   manufacturer avg_hwy count
   <chr>          <dbl> <int>
 1 dodge           17.9    37
 2 ford            19.4    25
 3 chevrolet       21.9    19
 4 nissan          24.6    13
 5 toyota          24.9    34
 6 subaru          25.6    14
 7 audi            26.4    18
 8 hyundai         26.9    14
 9 volkswagen      29.2    27
10 honda           32.6     9
```

13.3 Exploratory Data Analysis

The last section of this lab will guide you through practicing exploratory data analysis on the dataset `diamonds` (contained in the `ggplot2` package).

Diamonds. We'll start with the `diamonds` data set. First, let's take a look at the first few lines of it, to get a feel for it:

```
diamonds %>%
  head()
```

```
# A tibble: 6 x 10
  carat cut       color clarity depth table price     x     y     z
  <dbl> <ord>     <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>
1  0.23 Ideal     E     SI2      61.5    55   326  3.95  3.98  2.43
2  0.21 Premium   E     SI1      59.8    61   326  3.89  3.84  2.31
3  0.23 Good      E     VS1      56.9    65   327  4.05  4.07  2.31
4  0.29 Premium   I     VS2      62.4    58   334  4.2   4.23  2.63
5  0.31 Good      J     SI2      63.3    58   335  4.34  4.35  2.75
6  0.24 Very Good J     VVS2     62.8    57   336  3.94  3.96  2.48
```

Think about which of these variables we might want to predict with a machine learning model. `price`makes intuitive sense; it's not something we can simply directly measure from a diamond, and we would likely be very interested in knowing how much a given diamond is worth.

Activities.

- How many observations are there in `diamonds`?
- How many variables? Of these, how many are features we could use for predicting `price`?

Run `?diamonds` and look at the variable definitions.

First, let's make a correlation matrix to see which continuous variables are correlated with `price`. *[Figure: `corrplot` (lower triangle, coloured squares, -1 dark red to $+1$ dark navy). Besides the strong positive carat/price/x/y/z block: light salmon (moderate negative, about -0.3): depth–table, the only clearly negative cell. Near white (≈ 0): depth with carat, price, x, y. Pale blue (weak positive, ≈ 0.1–0.2): table with carat, price, x, y, z; depth with z.]*

Take a moment and look at the arguments in the `corrplot()` function. What does each one do? What happens if you change `diag` to `TRUE` and `method` to `'square'`?

Activities.

- Which features are positively correlatedwith `price`? Do these make sense?
- Are any features negatively correlated with `price`?
- Which features are correlated with *each other*? Why do you think this might be?

Let's make a boxplot of the distribution of `price` per level of `cut` and `color`, to see if there appears to be a relationship between it and these predictors.*[Figure: Horizontal boxplots of price by cut (y, ordered by `reorder(cut, price)`, bottom to top: Ideal, Good, Very Good, Fair, Premium), 7 boxes per cut filled by `color` (viridis: D dark purple … J yellow). All boxes sit at low prices (lower quartiles ≈ 800–2,500) with long right whiskers and many outlier dots up to ≈18,800; Fair has far fewer outlier dots. In every cut the worse colours (I, J) have higher medians and wider boxes than D/E: e.g. Premium J median ≈5,000 with box to ≈9,000, while Ideal D/E medians are only ≈1,500–1,800 with boxes ending near 3,000. Ideal (the best cut) has the lowest prices overall; Premium the highest.]*

Activities.

- What do you learn from this plot about the relationship between `price`, `cut`, and `color`?
- Refer back to the definitions of the variables in `?diamonds`. Does anything you learned surprise you?

Since `J` is the worst color for diamonds, why do you think they tend to cost more?

Let's take a look at the relationship between `color` and `carat` to explore further. Remember from the correlation plot that `carat` is highly positively correlatedwith `price`.

```
diamonds %>%
  ggplot(aes(x = carat, y = reorder(color, carat))) +
  geom_boxplot() +
  theme_bw() +
  labs(x = "Carat", y = "Color")
```

[Figure: Horizontal boxplots of carat by color (`theme_bw`, unfilled boxes). x-axis "Carat" 0–5; y-axis "Color" ordered by median carat, bottom to top: D, E, F, G, H, I, J. Medians increase steadily: D ≈0.53, E ≈0.53, F ≈0.70, G ≈0.70, H ≈0.90, I ≈1.0, J ≈1.1; upper quartiles from ≈0.9 (D, E) to ≈1.5 (I, J). Many outliers to the right for every colour (D–G from ≈1.8–2 up to ≈3); the largest stones are J (outliers up to 5.01 carat) and I (up to ≈4.0); H has one at ≈4.1. Message: worse colours (I, J) tend to be much bigger diamonds.]

Activities.

- Explain why lower-quality colors tend to cost more.

Now we'll assess the distribution of our outcome variable `price`. Let's make a histogram:*[Figure: Histogram of `price`, 60 bins. Very tall narrow peak near price ≈700 (≈8,200 diamonds), next bin ≈7,000, then a sharp drop to ≈3,200 and ≈2,300–2,600 around 1,500–2,000; a small secondary bump of ≈1,600 near 4,500; then a long, slowly decaying right tail with only ≈100–300 diamonds per bin from about 10,000 to the maximum ≈18,800.]*

Notice that we increased the number of bins here; this allows us to get a more fine-grained picture of the distribution.

Activities.

- Create a single plot to visualize the relationship between `cut`, `carat`, and `price`.

13.4 Data Splitting

Now we're going to walk through the process of splitting the `diamonds` data set into two, a training setand a test set. Note that in the future, after we discuss the concept of resampling, we'll use a resampling technique called cross-validation, but for now, we'll work with the entire training set.

The textbook(s) describe a way to split data using base R functions, but `tidymodels` makes the process a lot easier.

In general, set a seed to whatever number you like. People often use birthdates, anniversaries, or lucky numbers, etc.

Activities.

- How many observations are now in the training and testing sets, respectively? Report the exact number, not a proportion.
- What do you think the `strata = price` argument does? Take a guess, then use `?initial_split` to verify.

14 Lab 2 — Model fitting with tidymodels: linear regression, KNN, missing data

[Parts already covered in Part I are omitted: package loading and the train/test split chunk; recipe/`step_dummy()` code and `prep()`/`bake()` explanation; `lm` spec/workflow/fit code; `bind_cols()` and predicted-vs-actual code; RMSE outputs and the test metric set (1146, 0.918, 751); KNN engine/`set_mode()` explanation and spec code; `extract_fit_parsnip()` output (Best k: 5); `vis_miss()` percentages, `mean()`/NA and `drop_na()` (153 → 111) discussion, imputation choices, linear-imputation recipe, final `vis_miss()`, missing-data rules.] **Lab 2** PSTAT 131/231. *[Rmd header: `html_document` with `toc: true`, `toc_float: true`, `code_folding: show`. Setup chunk (not shown in output): `knitr::opts_chunk$set(echo = TRUE, message = FALSE, warning = FALSE)`.]*

14.1 Introduction: Model Fitting

Creating a Recipe. To fit any model with `tidymodels`, the first step is to create a recipe. The structure of this recipe is similar to that of `lm()`; the outcome is listed first, then the features are added:

```
simple_diamonds_recipe <-
  recipe(price ~ ., data = diamonds_train)
```

Note that `.` is a placeholder for "all other variables." If we call the recipe object now, we can see some information:

```
simple_diamonds_recipe
```

```
## Recipe
##
## Inputs:
##
##       role #variables
##    outcome          1
##  predictor          9
```

More specifically, we see that there are 9 predictors.

We'll also use `kbl()` and `kable_styling()` from the `kableExtra` package (`https://cran.r-project.org/web/packages/kableExtra/vignettes/awesome_table_in_html.html`), which we installed above. It's not necessary to use these functions, but doing so allows the table of data to display more neatly, so that all the columns and rows are actually legible. We also use `head()` to select only the first few rows; otherwise the entire data frame would print, which would be time-consuming. Also note the use of `scroll_box()`, allowing us to scroll through the entire data set.

```
prep(diamonds_recipe) %>%
  bake(new_data = diamonds_train) %>%
  head() %>%
  kable() %>%
  kable_styling(full_width = F) %>%
  scroll_box(width = "100%", height = "200px")
```

[note: the HTML table has 24 columns; shown here split into three blocks of the same 6 rows. `cut`, `color`, `clarity` are ordered factors, so `step_dummy()` produced orthogonal-polynomial contrast columns (`cut_1`..`cut_4`, `color_1`..`color_6`, `clarity_1`..`clarity_7`), not 0/1 columns.]

carat	depth	table	x	y	z	price	cut_1	cut_2
0.23	61.5	55	3.95	3.98	2.43	326	0.6325	0.5345
0.21	59.8	61	3.89	3.84	2.31	326	0.3162	-0.2673
0.23	56.9	65	4.05	4.07	2.31	327	-0.3162	-0.2673
0.29	62.4	58	4.20	4.23	2.63	334	0.3162	-0.2673
0.31	63.3	58	4.34	4.35	2.75	335	-0.3162	-0.2673
0.24	62.8	57	3.94	3.96	2.48	336	0.0000	-0.5345

cut_3	cut_4	color_1	color_2	color_3	color_4	color_5
0.3162	0.1195	-0.3780	0.0000	0.4082	-0.5641	0.4364
-0.6325	-0.4781	-0.3780	0.0000	0.4082	-0.5641	0.4364
0.6325	-0.4781	-0.3780	0.0000	0.4082	-0.5641	0.4364
-0.6325	-0.4781	0.3780	0.0000	-0.4082	-0.5641	-0.4364
0.6325	-0.4781	0.5669	0.5455	0.4082	0.2417	0.1091
0.0000	0.7171	0.5669	0.5455	0.4082	0.2417	0.1091

color_6	clar_1	clar_2	clar_3	clar_4	clar_5	clar_6	clar_7
-0.1974	-.3858	.0772	.3077	-.5238	.4922	-.3077	.1195
-0.1974	-.2315	-.2315	.4308	-.1209	-.3638	.5539	-.3585
-0.1974	.0772	-.3858	-.1846	.3626	.3210	-.3077	-.5974
-0.1974	-.0772	-.3858	.1846	.3626	-.3210	-.3077	.5974
0.0329	-.3858	.0772	.3077	-.5238	.4922	-.3077	.1195
0.0329	.2315	-.2315	-.4308	-.1209	.3638	.5539	.3585

[note: values rounded to 4 places (output has 7); `clar_k` = `clarity_k`.]

Activities:

- Use the Internet to find documentation about the possible `step` functions. Name three `step` functions that weren't used here and describe what they do.

14.2 Linear Regression

We can view the model results:

```
lm_fit %>%
  # This returns the parsnip object:
  extract_fit_parsnip() %>%
  # Now tidy the linear model object:
  tidy()
```

```
## # A tibble: 24 x 5
##    term         estimate std.error statistic   p.value
##    <chr>           <dbl>     <dbl>     <dbl>     <dbl>
##  3 depth          -58.4       6.25     -9.33  1.07e- 20
##  4 table          -24.1       3.23     -7.45  9.40e- 14
##  7 z             -127.       73.2      -1.74  8.23e-  2
##  9 cut_2          -300.      19.9      -15.0   4.77e- 51
## 10 cut_3           153.      17.2        8.90  5.99e- 19
## # ... with 14 more rows
```

[note: rows 1–2, 5–6 and 8 (`(Intercept)`, `carat`, `x`, `y`, `cut_1`) are in Part I and omitted here.]

Activities:

- Explain what the intercept represents.
- Describe the effect of `carat`. Is it a significant predictor of `price`? Holding everything else constant, what is the effect on `price` of a one-unit increase in `carat`?

The following code generates predicted values for `price` for each observation in the training set:

```
diamond_train_res <- predict(lm_fit, new_data = diamonds_train %>%
    ↪  select(-price))
diamond_train_res %>%
  head()
```

```
## # A tibble: 6 x 1
##    .pred
##    <dbl>
## 1 -1363.
## 2  -667.
## 3   199.
## 4  -815.
## 5 -3472.
## 6 -1373.
```

We might be interested in a plot of predicted values vs. actual values, here for the training data: *[Figure: Square scatterplot (`theme_bw`, `coord_obs_pred` so both axes share the range ≈ -5000 to 40000): x = `.pred` (linear-model prediction), y = `price`; semi-transparent black points; dashed 45° line $y = x$. Points form a curved (convex) band, not a line: for actual prices below ≈ $2000 many predictions are negative (down to ≈ -4000); between ≈$5000 and $10000 actual, predictions are too low; the band then flattens at the price ceiling ≈ $18,800, with predictions spreading from ≈7000 to ≈40000 (scattered points at .pred 25000–40000 with price 12000–18500). Shows a nonlinear relationship the linear model misses.]*

It's fairly clear that the model didn't do very well. If it predicted every observation accurately, the dots would form a straight line. We also have predicted some negative values for price, and once the actual price is approximately over $5,000, the model does a pretty poor job.

We can create and view a "metric set" of RMSE, MSE, and R^2 as shown:

```
diamond_metrics <- metric_set(rmse, rsq, mae)
diamond_metrics(diamond_train_res, truth = price,
                estimate = .pred)
```

```
## # A tibble: 3 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rmse    standard    1126.
## 2 rsq     standard       0.920
## 3 mae     standard     736.
```

Activities:

- Is there a difference between the three metrics for the training data and the testing data?
- Do your best to explain why this difference does (or does not) exist.

14.3 *k*-Nearest Neighbors

Now we'll take the recipe we've already created and try fitting a KNN (*k*-nearest neighbors) model with it (`https://parsnip.tidymodels.org/reference/nearest_neighbor.html`)! To do this, we'll use the `nearest_neighbor()` function, rather than the `linear_reg()` function, but we'll still need to select an engine. There is only one *R* package, or engine, that works with `nearest_neighbor()`, and that is the `kknn` package.

[Figure: Cartoon by Allison Horst (images/parsnip.png): title "parsnip::set_your_engine()!". Four race cars on a track at a checkered start line, labelled STAN (purple), keras (red, flag), GLMNET (blue) and lm (yellow); a purple fuzzy creature in sunglasses holds a "parsnip" hex flag; text "3…2…1… Run model!". Caption: "a standardized interface for fitting models and making predictions."]

Unlike a linear regression, however, there is a parameter – more specifically, a *hyperparameter* – that we have to set to fit a *k*-nearest neighbors model. That hyperparameter is k – the number of neighbors for the model to consider.

How do we know the "right" or the optimal value of k to use? Well, we don't! Eventually we'll discuss the concepts of resampling, cross-validation, and tuning, which will allow us to determine the optimal value of a hyperparameter with relative ease. For now, we'll go with the default value of k.

Activities:

- What IS the default value of k here, if we do not specify a value? How did you find out?

Now we add the model and recipe to the workflow and fit the model to the training data set:

```
knn_wflow <- workflow() %>%
  add_model(knn_model) %>%
  add_recipe(simple_diamonds_recipe)

knn_fit <- fit(knn_wflow, diamonds_train)
```

We'll generate the predictions from this model for the training set and testing set, and then compare the metrics for each, as before:

```
diamond_metrics(diamond_train_knn, truth = price,
                estimate = .pred)
```

```
## # A tibble: 3 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rmse    standard     386.
## 2 rsq     standard       0.991
## 3 mae     standard     201.
```

```
diamond_metrics(diamond_test_knn, truth = price,
                estimate = .pred)
```

```
## # A tibble: 3 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rmse    standard     711.
## 2 rsq     standard       0.968
## 3 mae     standard     373.
```

Let's try a different value for *k*. We can manually specify a value by adding it inside the nearest_neighbor() function. Later, when we introduce cross-validation, that's also where we'll specify the parameter to be tuned. The value we try is arbitrary; we can try basically any value of *k* from 1 to *n*. Then we have to fit the new model, etc.:

```
knn_model2 <- nearest_neighbor(neighbors = 2) %>%
  set_engine("kknn") %>%
  set_mode("regression")

knn_wflow2 <- workflow() %>%
  add_model(knn_model2) %>%
  add_recipe(diamonds_recipe)

knn_fit2 <- fit(knn_wflow2, diamonds_train)
```

[predict()/bind_cols() lines for train and test omitted (as in Part I).]

```
diamond_metrics(diamond_train_knn2, truth = price,
                estimate = .pred)
```

```
## # A tibble: 3 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rmse    standard     152.
## 2 rsq     standard       0.999
## 3 mae     standard      74.6
```

```
diamond_metrics(diamond_test_knn2, truth = price,
                estimate = .pred)
```

```
## # A tibble: 3 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rmse    standard     851.
## 2 rsq     standard       0.955
## 3 mae     standard     421.
```

Activities:

- Which of the three models – linear regression, *k*-nearest neighbors with $k = 5$, or *k*-nearest neighbors with $k = 2$ – performed better on the testing data? Why do you think this is so?
- What do you think explains the difference between the training and testing metrics for the KNN models?
- How many predictors are included in our KNN model(s)?
- Do you think changing the number of predictors might improve the performance of the KNN model(s)?

14.4 Missing Data

We'll use the airquality data set, which comes installed with base R, to illustrate some ways of handling missing data. Let's start by loading the naniar package (https://cran.r-project.org/web/packages/naniar/vignettes/getting-started-w-naniar.html). Usually it's considered good practice to load all packages in the same place – at the beginning of the Markdown file – but we'll make an exception. If this is your first time using the package, you'll need to uncomment and run the install.packages() line.

Try running ?airquality to learn about the variables in the data set. There are 153 observations on six variables.

[Figure: vis_miss(airquality) heat map (grey = present, black = missing; rows 1–153). Ozone has many black stripes (rows ≈5, 10, 25–27, big blocks ≈32–37 and ≈52–61, plus ≈65, 72, 75, 84, 102–103, 107, 115, 119, 150); Solar.R has a few (rows ≈5–6, 11, 27, 96–98); Wind, Temp, Month, Day are fully grey.]

We can see that these observations are missing also by directly looking at the data:

```
airquality %>%
  kable() %>%
  kable_styling(full_width = F) %>%
  scroll_box(width = "100%", height = "200px")
```

Ozone	Solar.R	Wind	Temp	Month	Day
41	190	7.4	67	5	1
36	118	8.0	72	5	2
12	149	12.6	74	5	3
18	313	11.5	62	5	4
NA	NA	14.3	56	5	5
28	NA	14.9	66	5	6
23	299	8.6	65	5	7
19	99	13.8	59	5	8
8	19	20.1	61	5	9
NA	194	8.6	69	5	10
7	NA	6.9	74	5	11
16	256	9.7	69	5	12

... (141 more rows; e.g. rows 25-27 and 32-37 have Ozone NA)

Since the data set is meant to measure air quality, it makes sense that we might want to predict the ozone levels. We can look at a histogram of the variable:

```
ggplot(aes(x = Ozone), data = airquality) +
  geom_histogram()
```

[Figure: Histogram (default 30 bins, grey ggplot theme) of Ozone: x 0–170, y count 0–17. Strongly right-skewed: tallest bar ≈17 at Ozone ≈10–15, bars of 13 at ≈15–25, 10 at ≈35, 8 at ≈30 and ≈45, then mostly 1–6 per bin from 50 to 125, isolated bars at ≈130–135 and ≈165–170 (count 1).]

If there is even one NA value, for instance, mean() will report NA. We can tell it to drop any rows with NA and get a value instead:

```
airquality %>%
  summarise(mean(Ozone, na.rm = T))
```

```
##   mean(Ozone, na.rm = T)
## 1               42.12931
```

Since only two variables have missingness, we'll take a relatively easy route and use linear imputation to handle them. This can be incorporated in the recipe like so:

```
airquality_split <- initial_split(airquality, prop = 0.80,
                                  strata = Ozone)
airquality_train <- training(airquality_split)
airquality_test <- testing(airquality_split)
```

[air_recipe with two step_impute_linear() steps omitted (in Part I).] And we can prep() and bake() the recipe to verify that it worked – these missing values were imputed, or predicted, using the complete variables!

```
prep(air_recipe) %>%
  bake(new_data = airquality_train) %>%
  kable() %>%
  kable_styling(full_width = F) %>%
  scroll_box(width = "100%", height = "200px")
```

Solar.R	Wind	Temp	Month	Day	Ozone	
149	12.6	74	5	3	12	
313	11.5	62	5	4	18	
19	20.1	61	5	9	8	
206	6.9	74	5	11	7	# Solar.R imputed
256	9.7	69	5	12	16	
290	9.2	66	5	13	11	

... (121 rows in all; Ozone is now the last column) ...

162	14.3	56	5	5	-11	# both imputed

```
194  8.6   69     5  10    37   # Ozone imputed
 66 16.6   57     5  25   -12   # Ozone imputed
286  8.6   78     6   1    47   # Ozone imputed
266 14.9   58     5  26    -3   # Ozone imputed
128  8.0   57     5  27    22   # both imputed
269  4.1   84     7   1   135
```

[note: the "imputed" comments are added here by comparing with the raw data; the knitted table has no comments. Linear imputation produced some impossible negative Ozone values (-11, -12, -3).]

Activities

- Fit a linear regression model using the imputed data.
- Compare the results of that model to a linear regression fit using mean imputation for both variables. Discuss the similarities and differences.

14.5 Resources

The free book *Tidy Modeling with R* (`https://www.tmwr.org/`) is strongly recommended.

You can view all the ISLR textbook code written with `tidymodels` here (`https://emilhvitfeldt.github.io/ISLR-tidymodels-labs/index.html`).

15 Lab 3 — Intro to binary classification (logistic, LDA, QDA, naive Bayes)

[Parts already covered in Part I are omitted: intro paragraph; `tidymodels/discrim/corrr/corrplot/klaR` loading; `correlate()`/`rplot()` and `corrplot()` code; `Direction`/`Purchase` bar plots and `table()` counts (602/648, 653/417); stratified 70/30 splits; `oj_recipe`, `logistic_reg()` spec and workflows; all `conf_mat()` / `accuracy()` code and the 8 training accuracies; LDA/QDA/naive Bayes specs and workflows; model-comparison tables; why choosing by training accuracy is bad; Smarket QDA and OJ logistic test confusion matrices and metrics.] **Lab 3: Intro to Binary Classification** PSTAT 131/231

Loading Packages. We load `tidymodels` for modeling functions, `ISLR` and `ISLR2` for data sets, `discrim` to give us access to discriminant analysis models such as LDA and QDA, as well as the Naive Bayes model, and `poissonreg` for Poisson regression. Finally, we load `corrr` for aid in visualizing correlation matrices.

```
library(ISLR) # For the Smarket data set
library(ISLR2) # For the Bikeshare data set
library(poissonreg)
```

Data. We'll start out by working with the `Smarket` data set for this lab, which contains daily percentage returns for the S&P 500 stock index between 2001 and 2005. It contains 1,250 observations on 8 numeric variables, plus a factor variable called `Direction` which has two levels, "Up" and "Down".

Before we go on to modeling, we'll explore the data a little, at least to look at possible correlations between the variables. `Direction` is our outcome variable of interest; we'll attempt to predict it using the volume of shares traded (`Volume`) and the five lag variables (which represent the percentage return for the previous days).

The other data set we'll work with for this lab is the `OJ`, or Orange Juice, data set. It contains 1,070 observations, each of which represents a customer's purchase of either Citrus Hill or Minute Maid Orange Juice. `Purchase` is our outcome variable, which indicates the brand purchased. We'll attempt to predict it using the prices charged for each brand, the discounts offered, and a metric of customer loyalty.

Activities

- Access the help page for `Smarket`. What does the `Direction` variable represent? What predictors do you think might be correlated with it?
- Access the help page for `OJ` and familiarize yourself with it.

To look at correlations among the continuous variables, we'll try using the `corrr` package. The `correlate()` function will calculate the correlation matrix between all the variables that it is given. We choose to remove `Direction`, it is not numeric. Then we pass the results to `rplot()` to visualize the correlation matrix. *[Figure: `rplot()` of the `Smarket` correlation matrix: rows and columns Year, Lag1–Lag5, Volume, Today; each cell a dot whose size and colour show the correlation (colour bar -1 red, 0 white, $+1$ blue; diagonal left blank). Every dot is essentially invisible (correlation ≈0) except two light-blue dots at Year–Volume and Volume–Year ($r \approx 0.54$).]*

Activities

- What do you notice from this correlation matrix?
- With this data set, do you need to worry about predictors being highly linearly correlated with each other?
- Does this matrix tell you anything about relationships between predictors and the outcome?

We could also use `ggplot` and the `geom_tile()` function to create a heatmap-style correlation plot, with a few more lines of code:

```
cor_Smarket %>%
  stretch() %>%
  ggplot(aes(x, y, fill = r)) +
  geom_tile() +
  geom_text(aes(label = as.character(fashion(r))))
```

[Figure: Heatmap (`geom_tile()`) of the stretched correlations: x and y both Lag1–Lag5, Today, Volume, Year; fill = r (dark navy ≈0 to light blue 0.5); diagonal tiles grey (NA); each tile labelled with r to two decimals. All labels are between $-.05$ and $.04$ (e.g. Lag1–Lag2 $-.03$, Volume–Lag4 $-.05$, Year–Lag4 $.04$, Volume–Today $.01$), except the two bright-blue tiles Year–Volume $= .54$.]

Notice that the upper and lower triangles of the matrix are identical; that's a common feature of correlation matrices. Again, we see that only `Year` and `Volume` have much of any correlation with each other, and it's only about 0.54.

Let's investigate that correlation a little further. Here's a boxplot of `Volume` by `Year`:

```
ggplot(Smarket, aes(factor(Year), Volume)) +
  geom_boxplot() +
  geom_jitter(alpha = 0.1) +
  xlab("Year")
```

[Figure: Boxplots of Volume (y, ≈0.4–3.2) for each Year 2001–2005 (x), with faint jittered points. Medians rise: 2001 ≈1.2, 2002 ≈1.38, 2003 ≈1.40, 2004 ≈1.42, 2005 ≈1.90 (box ≈1.65–2.15, max outlier ≈3.15). 2001 and 2002 have many high outliers (≈1.8–2.8). Volume clearly increases with Year, especially in 2005.]

We'll also consider a correlation plot for the orange juice data, restricting our analysis to the continuous predictor variables we'll be including: *[Figure: Lower-triangle `corrplot` (no diagonal) of the 7 `OJ` predictors PriceCH, PriceMM, DiscCH, DiscMM, LoyalCH, PctDiscMM, PctDiscCH (colour bar -1 red to $+1$ blue along the bottom). Strongest: PctDiscMM–DiscMM and PctDiscCH–DiscCH, huge dark-blue circles ($r \approx 1$); PriceMM–PriceCH a large medium-blue circle (≈0.6–0.7). Small light-blue circles (≈0.1–0.2) for DiscCH, DiscMM, LoyalCH, PctDiscMM, PctDiscCH with PriceCH, for LoyalCH with PriceMM and DiscCH, and for PctDiscCH–LoyalCH; the rest are tiny (≈0, a few faintly negative/orange).]*

Here, we set `diag = F` and `type = 'lower'`, which simplifies the plot a bit and may make it easier to read. We can practice interpreting these correlations; for example, `PriceMM` and `PriceCH` are positively correlated. This implies that higher prices for one brand of orange juice are correlated with higher prices for the other, and makes sense; when orange juice in general is cheaper to make, both brands would lower their prices, and vice versa.

Activities

- Describe the relationship between `Year` and `Volume`.

The orange juice data set is definitely somewhat unbalanced; the Citrus Hill brand appears to be more popular overall, with 653 observations representing customers purchasing Citrus Hill and only 417 Minute Maid. For now, we won't adjust for this, but we'll discuss methods of **upsampling** and **downsampling** in next week's lab. Also, note that this is by no means the most extreme example of an unbalanced outcome; some outcomes have incredibly rare levels.

15.1 Logistic Regression

Creating a Recipe. First, just as with our linear regression lab last week, we want to create a recipe to represent the model we'll be fitting – one for `Direction` and one for `Purchase`. Neither model has categorical predictors, so we won't need to include `step_dummy`.

```
smarket_recipe <- recipe(Direction ~ Lag1 + Lag2 + Lag3 + Lag4 +
    ↪
                           Lag5 + Volume, data = smarket_train)
```

Model Results. We can view the results – for the `Smarket` data:

```
smarketlog_fit %>%
  tidy()
```

```
# A tibble: 7 x 5
  term        estimate std.error statistic p.value
  <chr>          <dbl>     <dbl>     <dbl>   <dbl>
1 (Intercept) -0.197      0.293     -0.672    0.502
2 Lag1        -0.00479    0.0601    -0.0797   0.937
3 Lag2        -0.0537     0.0619    -0.868    0.386
4 Lag3         0.00265    0.0611     0.0435   0.965
5 Lag4        -0.0440     0.0594    -0.740    0.459
6 Lag5         0.0381     0.0605     0.630    0.529
7 Volume       0.184      0.193      0.952    0.341
```

And for the `OJ` data:

```
ojlog_fit %>%
  tidy()
```

```
# A tibble: 8 x 5
  term        estimate std.error statistic  p.value
  <chr>          <dbl>     <dbl>     <dbl>    <dbl>
2 PriceCH         3.68      1.49      2.48  1.31e- 2
3 PriceMM        -4.17      0.980    -4.26  2.03e- 5
4 DiscCH        -10.0      23.2      -0.431 6.66e- 1
5 DiscMM         27.1      10.1       2.69  7.20e- 3
7 PctDiscMM     -51.8      21.0      -2.47  1.36e- 2
8 PctDiscCH       8.43     43.5       0.194 8.46e- 1
```

[note: rows 1 and 6 ((Intercept) 4.47, LoyalCH −6.07) are in Part I and omitted here.]

Assessing Model Performance. We can use each of these models to generate probability predictions for the training data:

```
predict(smarketlog_fit, new_data = smarket_train, type = "prob")
```

```
# A tibble: 874 x 2
   .pred_Down .pred_Up
        <dbl>    <dbl>
 1      0.521    0.479
 2      0.470    0.530
 3      0.498    0.502
 4      0.498    0.502
 5      0.467    0.533
 6      0.518    0.482
 7      0.468    0.532
 8      0.478    0.522
 9      0.517    0.483
10      0.521    0.479
# ... with 864 more rows
```

Each row represents the probability predicted by the model that a given observation belongs to a given class. Notice this is redundant, because one could be calculated directly from the other, but it's useful in multiclass situations.

We haven't stored these, just looked at them to illustrate what's going on "behind the scenes." For the first row, or the first observation in the `Smarket` **training** set, the logistic regression model generated a predicted probability of `Down` of about 0.56 and a probability of `Up` of about 0.44. The other rows could be interpreted in much the same way. One thing to note is that none of these first 10 probabilities is below 0.40 or above 0.59; this is a clue that the model is not able to discriminate between the two outcome classes very well. Compare that to the probabilities for the `OJ` data: *[note: the knitted output above shows 0.521 / 0.479 for row 1; the 0.56 / 0.44 in the text are from an earlier knit.]*

```
predict(ojlog_fit, new_data = oj_train, type = "prob")
```

```
# A tibble: 748 x 2
   .pred_CH .pred_MM
      <dbl>    <dbl>
 3    0.970  0.0300
 4    0.958  0.0424
 5    0.951  0.0494
 6    0.952  0.0484
 7    0.994  0.00625
 8    0.840  0.160
 9    0.846  0.154
10    0.795  0.205
# ... with 738 more rows
```

However, it's more useful to summarize the predicted values. We can use `augment()` to attach the predicted values to the data, then generate a confusion matrix for `Smarket`:

```
          Truth
Prediction Down  Up
      Down  104  98
      Up    317 355
```

We'll create a confusion matrix for the orange juice data as well: *[Figure: Heatmap of the OJ logistic-regression training confusion matrix (rows Prediction CH/MM, columns Truth CH/MM): Pred CH: 403 (Truth CH), 71 (Truth MM); Pred MM: 54 (Truth CH), 220 (Truth MM). The diagonal (403, 220) is dark — mostly correct predictions ($623/748 = 0.833$).]*

Activities

- What do you notice from each of these confusion matrices?
- Consider the data sets. For each outcome, do the models tend to overpredict one class or the other? If so, which one?
- Do you think the models are doing well? Why or why not?

15.2 LDA

Assessing Performance. This can be done almost exactly the same way. We can view a confidence matrix and calculate accuracy on the **training data**: *[note: "confidence matrix" (sic) = confusion matrix.] [Smarket, training data]*

```
          Truth
Prediction Down  Up
      Down  103  98
      Up    318 355
```

[OJ, training data]

```
          Truth
Prediction  CH  MM
        CH 398  70
        MM  59 221
```

Activities

- Compare the results of the LDA models to the results of the logistic regression models.

15.3 QDA

Assessing Performance. *[Smarket, training data]*

```
          Truth
Prediction Down  Up
      Down  125  99
      Up    296 354
```

[OJ, training data]

```
          Truth
Prediction  CH  MM
        CH 370  65
        MM  87 226
```

15.4 Naive Bayes

Assessing Performance. *[Smarket, training data]*

```
          Truth
Prediction Down  Up
      Down  107  98
      Up    314 355
```

[OJ, training data]

```
          Truth
Prediction  CH  MM
        CH 338  47
        MM 119 244
```

15.5 Comparing Model Performance

Activities

- Which model performed the best on the training data for each data set?
- Which model would you choose? Why?

15.6 Fitting to Testing Data

Finally, let's look at an ROC curve on the testing data: *[Figure: ROC curve of the Smarket QDA model on the test set (x "1 - specificity", y "sensitivity", both 0–1, dotted diagonal): a jagged staircase that hugs the 45° diagonal the whole way, dipping slightly below it (e.g. sensitivity ≈0.06 at FPR 0.12, ≈0.25 at FPR 0.28, ≈0.71 at FPR 0.75) and only marginally above it around FPR 0.4–0.6 — essentially no discrimination (AUC ≈0.5; not printed).]*

We'll fit the logistic regression to the `OJ` testing set, since it also performed best on the training set, with the same caveats, and view the same things: *[code (`conf_mat()`, `metric_set()`, `roc_curve(Purchase, .pred_CH)`) and the confusion matrix / metrics are in Part I.][Figure: ROC curve of the OJ logistic regression on the test set (same axes, dotted diagonal): bows strongly toward the top-left corner — rises steeply to sensitivity ≈0.68 by FPR ≈0.06, ≈0.81 by FPR ≈0.13, ≈0.90 by FPR ≈0.25, ≈0.95 by FPR ≈0.37, then flattens (≈0.97 at 0.45–0.55, ≈0.99 at 0.75) to 1.00 at FPR ≈0.89. Far above the diagonal = good discrimination (AUC not printed).]*

Activities

- What do the sensitivity and specificity values mean? Interpret them in terms of the concepts and data.
- How well did the models perform on the **testing** data?
- Why do you think the models performed like they did?
- Which model performed better?

15.7 Resources

The free book *Tidy Modeling with R* is strongly recommended.

15.8 Source

Several parts of this lab come directly from the "ISLR Tidymodels Labs". Credit to Emil Hvitfeldt for writing and maintaining the open-source book.

16 Homework 1–3: prompts, my submissions, solutions

[Parts already covered in Part I are omitted: HW1 instructions + setup chunks, Q1–Q6 (all), EDA intro, most plotting code; HW2 Abalone photo, setup chunk, Q5, Q7, Q10–Q12 (231; Q12 proof removed), most recipe/workflow/metric code and metric tables; HW3 photo, setup chunk, Q7, Q11–Q12 (231; proofs removed), model-spec/workflow code, AUC tables, confusion matrix + metrics. See d5 "HW answers at a glance", d2 HW2 pipeline, d2b HW3 pipeline, d4 templates.]

*How to read: each box = one question. **Prompt.** = assignment text (verbatim); **My answer.** = my graded submission, verbatim (HW1, HW2); **Answer.** = correction/note (HW1, HW2) or my own worked solution, run in R 4.5.3 (HW3, not yet submitted).*

16.1 Homework 1 – ML main ideas; EDA on mpg

HW1 Ex1

Prompt. We are interested in highway miles per gallon, or the `hwy` variable. Create a histogram of this variable. Describe what you see/learn.

My answer. *[Figure: grey histogram, white bar outlines, bin width 2. x = "Highway MPG" (ticks 10–40), y = "Count" (0–40+). Bin counts (centres 12, 14, …, 44): 5, 12,*

38, 23, 13, 14, 28, 46, 29, 11, 6, 3, 3, 0, 1, 0, 2. Two peaks: 38 cars at 15–17 mpg and the tallest bar, 46 cars, at 25–27 mpg; a few cars above 40.]
The two peaks are around 16-18mpg and 24-28mpg, with a median of 24 and a mean of about 23.4. Most cars fall between 15 and 30mpg. There are a few outliersabove 40mpg.

HW1 Ex2

Prompt. Create a scatterplot. Put `hwy` on the x-axis and `cty` on the y-axis. Describe what you notice. Is there a relationship between `hwy` and `cty`? What does this mean?
My answer. *[Figure: black-dot scatterplot, x = "Highway MPG" (12–44), y = "City MPG" (9–35). Points lie in a tight, straight, upward band from (12, 9) to (44, 35); they sit on a grid of whole numbers, so many cars overlap (only 78 distinct points for 234 cars).]*
City MPG is consistently lower than highway MPG; the fitted line has a slope of about 0.7, so each extra highway MPG goes with about 0.7 extra city MPG. Even though there are 234 cars in the data there aren't 234 dots on the chart due to rounding to the nearest whole number and multiple cars overlapping

HW1 Ex3

Prompt. Make a bar plot of `manufacturer`. Flip it so that the manufacturers are on the y-axis. Order the bars by height. Which manufacturer produced the most cars? Which produced the least?
My answer.

```
mpg %>% count(manufacturer, sort = TRUE)
```

```
# A tibble: 15 x 2
   manufacturer     n
   <chr>        <int>
 1 dodge           37
 2 toyota          34
 3 volkswagen      27
 4 ford            25
 5 chevrolet       19
 6 audi            18
 7 hyundai         14
 8 subaru          14
 9 nissan          13
10 honda            9
11 jeep             8
12 pontiac          5
13 land rover       4
14 mercury          4
15 lincoln          3
```

HW1 Ex4

Prompt. Make a boxplot of `hwy`, grouped by `cyl`. Use `geom_jitter()` and the `alpha` argument to add points to the plot.
Describe what you see. Is there a relationship between `hwy` and `cyl`? What do you notice?
My answer.

```
ggplot(mpg, aes(x = factor(cyl), y = hwy)) +
  geom_boxplot(outlier.shape = NA) +
  geom_jitter(alpha = 0.3, width = 0.2) +
  labs(x = "Number of Cylinders", y = "Highway MPG")
```

[Figure: white boxplots of "Highway MPG" for x = "Number of Cylinders" 4, 5, 6, 8, with semi-transparent grey jittered points. 4 cyl: box 26–31, points 20–44 (three above 40); 5 cyl: only 4 cars, flat box at 28.75–29; 6 cyl: box 19–26, points 17–29; 8 cyl: box 16–19, points 12–26.]
This makes sense since if cylinders are not fully utilized their efficiency goes down, and with more cylinders the chance of full utilization drops. The 4cyl group has the widest spread and includes the high-mileage outliers above 40 mpg. There aren't many cars with 5cyl so its not a reliable box. There is also some overlap between groups they're not all disjoint, like some 4cyl get worse mileage than some 8cyls, though on average cylinders are negatively correlated to mpg
Answer. (note) 4-cyl has the widest *range* (20–44), but the widest *box* (IQR) is 6-cyl (19–26, IQR 7 vs. 5 for 4-cyl). `outlier.shape = NA` hides the boxplot's own outlier dots so they are not drawn twice on top of the jittered points.

HW1 Ex5

Prompt. Use the `corrplot` package to make a lower triangle correlation matrix of the `mpg` dataset. (Hint: You can find information on the package here [link: `cran.r-project.org/web/packages/corrplot/vignettes/corrplot-intro.html`].)
Which variables are positively or negatively correlated with which others? Do these relationships make sense to you? Are there any that surprise you?
My answer. *[Figure: lower-triangle corrplot of the 5 numeric variables (displ, year, cyl, cty, hwy), coefficients printed as coloured numbers (dark blue = $+1$, white = 0, dark red = -1; colour bar -1 to 1 underneath; labels in red). Values: year–displ 0.15; cyl–displ 0.93, cyl–year 0.12; cty–displ -0.80, cty–year -0.04, cty–cyl -0.81; hwy–displ -0.77, hwy–year 0.00, hwy–cyl -0.76, hwy–cty 0.96; diagonal 1.00.]*
`displ` and `cyl` are strongly positively correlated with each other, since the more cylinders you add the more volume the engine displaces on each cycle. Both mileage variables are strongly negatively correlated with both engine size variables, since more displacement means more volume (and in turn mass) that needs to be heated, increasing the fuel burn rate. The only metric that isn't correlated with mpg is the year. That one surprised me, since I expected newer cars to be more fuel efficient, the EPA had been gradually increasing efficiency requirements, but the data only covers 1999 and 2008 so there isn't much of a trend even after 9 years.

Prompt (231 Students Only) — from the 131/231 version `homework-1-1.html`; *231 only, not required for 131, not in my submission*.

HW1 Ex6 (231 only)

Prompt. USing `ggplot()`, recreate the following graphic as closely as you can. *Hint: Use the* `ggthemes` *package.*
[Figure: target graphic: horizontal boxplots of x = "Highway MPG" (ticks 20, 30, 40) for y = "Vehicle Class" (bottom to top: 2seater, compact, midsize, minivan, pickup, subcompact, suv). Unfilled black boxes, outliers as solid black dots, plus grey semi-transparent jittered points. White background, light-grey horizontal and vertical grid lines, a black x-axis line along the bottom, grey sans-serif labels (a Google-Docs-like ggthemes theme). Pickup and suv lowest (≈17), compact/midsize/subcompact highest (≈26–27 medians), subcompact widest.]
Answer. (my recreation, checked in R)

```
ggplot(mpg, aes(x = hwy, y = class)) +
  geom_boxplot() +
  geom_jitter(alpha = 0.3) +
  theme_gdocs() +
  labs(x = "Highway MPG", y = "Vehicle Class")
```

HW1 Ex7 (231 only)

Prompt. Recreate the following graphic.
[Figure: target graphic: default grey ggplot theme; side-by-side boxplots of y = hwy (ticks 20, 30, 40) for x = class (2seater, compact, midsize, minivan, pickup, subcompact, suv), boxes filled by legend "drv" (4 = red, f = green, r = blue). 2seater: r only (≈25); compact: 4 (≈25) and f (≈29, outliers 37, 44); midsize: 4 (≈24), f (≈27); minivan: f (≈23); pickup: 4 (≈17); subcompact: 4 (flat at 26), f (≈29, up to 44), r (≈23); suv: 4 (≈18, many outliers) and r (≈17).]
Answer. (my recreation, checked in R)

```
ggplot(mpg, aes(x = class, y = hwy, fill = drv)) +
  geom_boxplot()
```

HW1 Ex8 (231 only)

Prompt. Recreate the following graphic.
[Figure: target graphic: default grey theme; scatterplot y = hwy vs. x = displ (1.6–7), points coloured by "drv" (4 = red, f = green, r = blue) and three blue loess curves without ribbons, one per drv, distinguished by linetype (4 = solid, decreasing from ≈27 to ≈16 then flat; f = short dashes, from ≈33 down to ≈25 then flat; r = long dashes, ≈25 dipping to ≈18 at displ 5.3 then rising to ≈25). One merged legend "drv" shows coloured point + line type.]
Answer. (my recreation, checked in R)

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(aes(color = drv)) +
  geom_smooth(aes(linetype = drv), se = FALSE)
```

16.2 Homework 2 – linear regression and KNN (abalone)

Prompt (Linear Regression and KNN). For this assignment, we will be working with a data set from the UCI (University of California, Irvine) Machine Learning repository (see website here [link: `archive.ics.uci.edu/ml/datasets/Abalone`]). The full data set consists of $4{,}177$ observations of abalone in Tasmania. (Fun fact: Tasmania supplies about 25% of the yearly world abalone harvest.)
The age of an abalone is typically determined by cutting the shell open and counting the number of rings with a microscope. The purpose of this data set is to determine whether abalone age (**number of rings + 1.5**) can be accurately predicted using other, easier-to-obtain information about the abalone.
Codebook (`abalone_codebook.txt`; Name / Data Type / Measurement Unit / Description):

Sex	nominal	–	M, F, and I (infant)
Length	continuous	mm	Longest shell measurement
Diameter	continuous	mm	perpendicular to length
Height	continuous	mm	with meat in shell
Whole weight	continuous	grams	whole abalone
Shucked weight	continuous	grams	weight of meat
Viscera weight	continuous	grams	gut weight (after bleeding)
Shell weight	continuous	grams	after being dried
Rings	integer	–	+1.5 gives the age in years

[note: in `abalone.csv` *the columns are named* `type`, `longest_shell`, `diameter`, `height`, `whole_weight`, `shucked_weight`, `viscera_weight`, `shell_weight`, `rings`*.]*

HW2 Q1

Prompt. Your goal is to predict abalone age, which is calculated as the number of rings plus 1.5. Notice there currently is no age variable in the data set. Add age to the data set.
Assess and describe the distribution of age.
My answer.

```
head(abalone)
```

```
# A tibble: 6 x 10
  type  longest_shell diameter height whole_weight shucked_weight
    ↪ viscera_weight
  <fct>         <dbl>    <dbl>  <dbl>        <dbl>           <dbl>
    ↪     <dbl>
1 M             0.455    0.365  0.095        0.514          0.224
    ↪    0.101
2 M             0.35     0.265  0.09         0.226          0.0995
    ↪    0.0485
3 F             0.53     0.42   0.135        0.677          0.256
    ↪    0.142
4 M             0.44     0.365  0.125        0.516          0.216
    ↪    0.114
5 I             0.33     0.255  0.08         0.205          0.0895
    ↪    0.0395
6 I             0.425    0.3    0.095        0.352          0.141
    ↪    0.0775
# i 3 more variables: shell_weight <dbl>, rings <dbl>, age <dbl>
```

```
abalone %>%
  summarise(n = n(),
            mean = mean(age),
            median = median(age),
            sd = sd(age),
            min = min(age),
            q1 = quantile(age, 0.25),
            q3 = quantile(age, 0.75),
            max = max(age))
```

```
# A tibble: 1 x 8
      n  mean median    sd   min    q1    q3   max
  <int> <dbl>  <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1  4177  11.4   10.5  3.22   2.5   9.5  12.5  30.5
```

[Figure: "Distribution of Abalone Age": steel-blue histogram, bin width 1, x = "Age (years)" (2.5–30.5), y = "Count" (0–≈690). Unimodal, peak ≈690 at age 10.5 (next 11.5 ≈630, 9.5 ≈570, 12.5 ≈490), long thin right tail; bars barely visible beyond 24.] [Figure: "Boxplot of Abalone Age": one horizontal steel-blue box, x = "Age (years)"; box 9.5–12.5, median 10.5, whiskers 5.5–16.5; outlier dots at 2.5–4.5 on the left and many from 17.5 to 30.5 on the right (meaningless y-axis -0.4 to 0.4).]
The max is almost 3 times the median, but the 25th and 75th percentiles are only 3 years apart meaning the data is tightly bound around the median.
Answer. (check) Correct: `boxplot.stats()` flags 278 of 4177 = 6.66% of ages as outliers (outside 9.5 − 1.5·3 = 5 and 12.5 + 1.5·3 = 17).

HW2 Q2

Prompt. Split the abalone data into a training set and a testing set. Use stratified sampling. You should decide on appropriate percentages for splitting the data.
My answer. I chose an 85/15 split since when I took a course on machine learning from Andrew Ng, *I think* he said 85/15 was the split he uses. 629 test rows is perfectly acceptable and the model should train well on 3548 rows.

HW2 Q3

Prompt. Using the **training** data, create a recipe predicting the outcome variable, age, with all other predictor variables. Note that you **should not** include `rings` to predict age. *Explain why you shouldn't use* `rings` *to predict* age.
My answer.

```
abalone_recipe %>%
  prep() %>%
  bake(new_data = NULL) %>%
  head()
```

```
# A tibble: 6 x 14
  longest_shell diameter height whole_weight shucked_weight
    ↪ viscera_weight
          <dbl>    <dbl>  <dbl>        <dbl>           <dbl>          <
    ↪ dbl>
1         -1.62    -1.54  -1.41        -1.27           -1.22
    ↪ -1.29
2         -0.828   -1.09  -1.05        -0.975          -0.985
    ↪ -0.943
3         -1.41    -1.29  -1.29        -1.10           -1.19
    ↪ -1.29
4         -0.496   -0.537 -0.817       -0.715          -0.599
    ↪ -0.519
5         -0.621   -0.537 -0.817       -0.627          -0.554
    ↪ -0.587
6         -2.66    -2.60  -1.99        -1.61           -1.50
    ↪ -1.51
# i 8 more variables: shell_weight <dbl>, age <dbl>, type_I <dbl>,
#   type_M <dbl>, type_I_x_shucked_weight <dbl>,
    ↪ type_M_x_shucked_weight <dbl>,
#   longest_shell_x_diameter <dbl>, shucked_weight_x_shell_weight <dbl>
```

Answer. (note) The key term is target leakage: age = rings $+1.5$ exactly, so a model with rings fits perfectly ($R^2 = 1$) and tells us nothing; and rings is exactly the hard-to-obtain measurement we want to avoid.

HW2 Q4

Prompt. Create and store a linear regression object using the "lm" engine.
My answer.

```
Linear Regression Model Specification (regression)

Computational engine: lm
```

HW2 Q6

Prompt. Now, for each of these models (linear regression and KNN):
1. set up an empty workflow,
2. add the model, and
3. add the recipe that you created in Question 3.

Note that you should be setting up two separate workflows.
Fit both models to the training set.
My answer. (`lm_fit %>% extract_fit_parsnip() %>% tidy()`, then `knn_fit`):

```
# A tibble: 14 x 5
   term                        estimate std.error statistic  p
     ↪ .value
   <chr>                          <dbl>     <dbl>     <dbl>
     ↪  <dbl>
 1 (Intercept)                   11.5      0.0367   312.     0
 2 longest_shell                  0.466    0.281      1.66
     ↪ 9.67e- 2
 3 diameter                       2.20     0.308      7.16
     ↪ 1.00e-12
 4 height                         0.262    0.0689     3.81
     ↪ 1.43e- 4
 5 whole_weight                   4.98     0.390     12.8
     ↪ 1.71e-36
 6 shucked_weight                -4.26     0.247    -17.3
     ↪ 3.07e-64
 7 viscera_weight                -1.02     0.155     -6.54
     ↪ 7.22e-11
 8 shell_weight                   1.61     0.209      7.72
     ↪ 1.52e-14
 9 type_I                        -0.919    0.111     -8.26
     ↪ 1.99e-16
10 type_M                        -0.223    0.101     -2.21
     ↪ 2.71e- 2
11 type_I_x_shucked_weight        0.468    0.0843     5.54
     ↪ 3.16e- 8
12 type_M_x_shucked_weight        0.237    0.106      2.23
     ↪ 2.59e- 2
13 longest_shell_x_diameter      -2.79     0.394     -7.06
     ↪ 1.94e-12
14 shucked_weight_x_shell_weight -0.149    0.201     -0.742
     ↪ 4.58e- 1
```

[note: term column shortened by 3 spaces to fit; values unchanged.]

```
-- Model ---------------------------------------------

Call:
kknn::train.kknn(formula = ..y ~ ., data = data,
  ks = min_rows(7, data, 5))

Type of response variable: continuous
minimal mean absolute error: 1.654218
Minimal mean squared error: 5.598762
Best kernel: optimal
Best k: 7
```

HW2 Q8

Prompt. Now you want to assess your models' performance. To do this, use the `yardstick` package:
1. Create a metric set that includes R^2, RMSE (root mean squared error), and MAE (mean absolute error).
2. Use `augment()` to create a tibble of your model's predicted values from the **testing data** along with the actual observed ages (these are needed to assess your model's performance).
3. Finally, apply your metric set to the tibble, report the results, and interpret the *R*\^2 value.

Repeat these steps once for the linear regression model and for the KNN model.
My answer. (`head()` of `augment(...) %>% select(age, .pred)`, LM then KNN):

```
    age .pred(LM) .pred(KNN)
1   8.5   9.51       9.31
2  11.5  12.5       12.0
3  13.5  12.9       13.4
4   8.5  10.5       10.9
5  12.5   9.91       8.68
6  13.5  12.0       11.7
```

[note: the two 6-row tibbles printed side by side to save space.] [Figure: "Predicted

vs. Observed Age (Testing Data)": two square facets, "KNN (k = 7)" (left) and "Linear Regression" (right); x = "Observed Age", y = "Predicted Age" (both ≈4–23), semi-transparent black points in vertical stripes (ages are whole numbers + 0.5), red dashed 45° line $y = x$. Both clouds follow the line for young abalone but fall below it for old ones (predictions too low above age ≈15); the KNN cloud is more spread out.]

HW2 Q9

Prompt. Which model performed better on the testing data? Explain why you think this might be. Are you surprised by any of your results? Why or why not?
My answer.

```
bind_rows(
  abalone_metrics(lm_test_results, truth = age, estimate = .pred
    ↪ ) %>%
    mutate(model = "Linear Regression", data = "Testing"),
  abalone_metrics(knn_test_results, truth = age, estimate = .
    ↪ pred) %>%
    mutate(model = "KNN (k = 7)", data = "Testing"),
  augment(lm_fit, new_data = abalone_train) %>%
    abalone_metrics(truth = age, estimate = .pred) %>%
    mutate(model = "Linear Regression", data = "Training"),
  augment(knn_fit, new_data = abalone_train) %>%
    abalone_metrics(truth = age, estimate = .pred) %>%
    mutate(model = "KNN (k = 7)", data = "Training")
) %>%
  select(model, data, .metric, .estimate) %>%
  pivot_wider(names_from = .metric, values_from = .estimate) %>%
    ↪
  arrange(data, model)
```

```
# A tibble: 4 x 5
  model             data       rsq  rmse   mae
  <chr>             <chr>    <dbl> <dbl> <dbl>
1 KNN (k = 7)       Testing  0.483  2.21  1.62
2 Linear Regression Testing  0.585  1.95  1.44
3 KNN (k = 7)       Training 0.811  1.46  1.02
4 Linear Regression Training 0.552  2.18  1.57
```

16.3 Homework 3 – binary classification (Titanic)

Prompt (Binary Classification). For this assignment, we will be working with part of a Kaggle data set [link: `kaggle.com/c/titanic/overview`] that was the subject of a machine learning competition and is often used for practicing ML models. The goal is classification; specifically, to predict which passengers would survive the Titanic shipwreck.

Codebook (`titanic_codebook.txt`):

survival:	Survival
pclass:	Ticket class 1 = 1st, 2 = 2nd, 3 = 3rd
sex:	Sex
Age:	Age in years
sibsp:	# of siblings / spouses aboard the Titanic
parch:	# of parents / children aboard the Titanic
ticket:	Ticket number
fare:	Passenger fare
cabin:	Cabin number
embarked:	Port of Embarkation C = Cherbourg, Q = Queenstown, S = Southampton

[note: CSV columns: `passenger_id`, `survived` (Yes/No), `pclass`, `name`, `sex`, `age`, `sib_sp`, `parch`, `ticket`, `fare`, `cabin`, `embarked`; 891 rows. My solutions use `set.seed(3435)`.]

HW3 Q1

Prompt. Split the data, stratifying on the outcome variable, `survived`. You should choose the proportions to split the data into. Verify that the training and testing data sets have the appropriate number of observations. Take a look at the training data and note any potential issues, such as missing data.
Why is it a good idea to use stratified sampling for this data?

```
colSums(is.na(titanic_train))
```

Answer. Issues in the training data: missing data — `age` is missing for 128 of 623 (20.5%; handled by `step_impute_linear()` in Q4), `cabin` for 481 (77%, too sparse to use), `embarked` for 2. `fare` is right-skewed (max 512).

HW3 Q2

Prompt. Using the **training** data set, explore/describe the distribution of the outcome variable `survived`.
Create a percent stacked bar chart [link: `r-graph-gallery.com/48-grouped-barplot-with-ggplot2`] (recommend using `ggplot`) with `survived` on the *x*-axis and `fill = sex`. Do you think `sex` will be a good predictor of the outcome?
Create one more percent stacked bar chart of `survived`, this time with `fill = pclass`. Do you think passenger class will be a good predictor of the outcome?
Why do you think it might be more useful to use a percent stacked bar chart as opposed to a traditional stacked bar chart?

```
titanic_train %>% count(survived) %>% mutate(prop = n / sum(n))
  ↪
titanic_test %>% count(survived) %>% mutate(prop = n / sum(n))
```

```
# A tibble: 2 x 3
  survived     n  prop
  <fct>    <int> <dbl>
1 Yes        239 0.384
2 No         384 0.616
# A tibble: 2 x 3
  survived     n  prop
  <fct>    <int> <dbl>
1 Yes        103 0.384
2 No         165 0.616
```

Numbers behind the charts (row % within each bar), plus survival rate per group:

```
titanic_train %>% count(survived, sex) %>%
  group_by(survived) %>% mutate(prop = n / sum(n))
titanic_train %>% group_by(sex) %>%
  summarise(n = n(), surv_rate = mean(survived == "Yes"))
```

```
  survived sex        n  prop       sex        n surv_rate
1 Yes      female   156 0.653      1 female   214      0.729
2 Yes      male      83 0.347      2 male     409      0.203
3 No       female    58 0.151
4 No       male     326 0.849
  survived pclass     n  prop       pclass     n surv_rate
1 Yes      1         97 0.406      1 1        154      0.630
2 Yes      2         61 0.255      2 2        128      0.477
3 Yes      3         81 0.339      3 3        341      0.238
4 No       1         57 0.148
5 No       2         67 0.174
6 No       3        260 0.677
```

[note: four tibbles printed side by side to save space; same code with `pclass` for the second pair. The two percent stacked bar charts show exactly these `prop` values.]

HW3 Q3

Prompt. Using the **training** data set, create a correlation matrix of all continuous variables. Visualize the matrix and describe any patterns you see. Are any predictors correlated with each other? Which ones, and in which direction?

```
cor_titanic <- titanic_train %>%
  select(age, sib_sp, parch, fare) %>%
  cor(use = "pairwise.complete.obs")
round(cor_titanic, 3)
corrplot(cor_titanic, method = "number", type = "lower")
```

```
          age sib_sp  parch  fare
age     1.000 -0.327 -0.173 0.113
sib_sp -0.327  1.000  0.433 0.145
parch  -0.173  0.433  1.000 0.254
fare    0.113  0.145  0.254 1.000
```

Answer. Continuous variables: `age`, `sib_sp`, `parch`, `fare` (`passenger_id` is an ID; `pclass` is now a factor). Patterns (Pearson correlation): `sib_sp`–`parch` **positive** (0.43: families travel together); `age`–`sib_sp` **negative** (-0.33: children travel with siblings), `age`–`parch` weakly negative (-0.17); `fare` weakly positive with `parch` (0.25) and `sib_sp` (0.15) (fare is per ticket, families share a ticket) and with `age` (0.11).

HW3 Q4

Prompt. Using the **training** data, create a recipe predicting the outcome variable `survived`. Include the following predictors: ticket class, sex, age, number of siblings or spouses aboard, number of parents or children aboard, and passenger fare.
Recall that there were missing values for `age`. To deal with this, add an imputation step using `step_impute_linear()`. Next, use `step_dummy()` to **dummy** encode categorical predictors. Finally, include interactions between:

- Sex and passenger fare, and
- Age and passenger fare.

You'll need to investigate the `tidymodels` documentation to find the appropriate step functions to use.

```
titanic_recipe <- recipe(survived ~ pclass + sex + age + sib_sp
    ↪ +
                           parch + fare, data = titanic_train)
    ↪ %>%
  step_impute_linear(age,
                     impute_with = imp_vars(pclass, sex, sib_sp,
    ↪
                                            parch, fare)) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_interact(terms = ~ starts_with("sex"):fare + age:fare)
titanic_recipe
```

```r
baked <- titanic_recipe %>% prep() %>% bake(new_data = NULL)
baked
sum(is.na(baked$age))
```

```
-- Recipe ----------------------------------------------
-- Inputs
Number of variables by role
outcome:   1
predictor: 6
-- Operations
* Linear regression imputation for: age
* Dummy variables from: all_nominal_predictors()
* Interactions with: starts_with("sex"):fare + age:fare
# A tibble: 623 x 10
     age sib_sp parch    fare survived pclass_X2 pclass_X3
     ↪ sex_male
   <dbl>  <dbl> <dbl>  <dbl> <fct>        <dbl>     <dbl>
     ↪ <dbl>
 1    22      1     0   7.25 No               0         1
     ↪     1
 2    35      0     0   8.05 No               0         1
     ↪     1
 3    54      0     0  51.9  No               0         0
     ↪     1
 4     2      3     1  21.1  No               0         1
     ↪     1
 5    39      1     5  31.3  No               0         1
     ↪     1
 6    14      0     0   7.85 No               0         1
     ↪     0
 7     2      4     1  29.1  No               0         1
     ↪     1
 8    31      1     0  18    No               0         1
     ↪     0
 9     8      3     1  21.1  No               0         1
     ↪     0
10    19      3     2 263    No               0         0
     ↪     1
# i 613 more rows
# i 2 more variables: sex_male_x_fare <dbl>, fare_x_age <dbl>
[1] 0
```

Answer. Imputation = a linear model of age on the other five predictors, fit on training rows with age observed. `step_dummy()` renames sex → sex_male (female = reference level) and pclass → pclass_X2, pclass_X3 (class 1 = reference). Result: 9 predictors, no missing age.

HW3 Q5

Prompt. Specify a **logistic regression** model for classification using the "glm" engine. Then create a workflow. Add your model and the appropriate recipe. Finally, use `fit()` to apply your workflow to the **training** data.

Answer. (`log_fit %>% extract_fit_parsnip() %>% tidy()`):

```
# A tibble: 10 x 5
   term              estimate std.error statistic  p.value
   <chr>                <dbl>     <dbl>     <dbl>    <dbl>
 1 (Intercept)      -4.40       0.683       -6.45  1.13e-10
 2 age               0.0629     0.0136       4.62  3.77e- 6
 3 sib_sp            0.437      0.132        3.30  9.52e- 4
 4 parch             0.151      0.153        0.989 3.23e- 1
 5 fare             -0.00116    0.0107      -0.108 9.14e- 1
 6 pclass_X2         1.25       0.363        3.46  5.48e- 4
 7 pclass_X3         2.44       0.382        6.39  1.62e-10
 8 sex_male          2.15       0.303        7.09  1.32e-12
 9 sex_male_x_fare   0.0139     0.00836      1.66  9.65e- 2
10 fare_x_age       -0.000360   0.000206    -1.75  8.03e- 2
```

Because "Yes" is the first level, `glm` models $P(\texttt{survived} = \text{"No"})$, so these log odds are of *dying*: sex_male $= +2.15$ means a male's odds of not surviving are $e^{2.15} \approx 8.6$ times a female's (at fare = 0, others fixed); pclass_X3 $= +2.44$: 3rd class $\approx e^{2.44} \approx 11.5\times$ the odds of dying vs. 1st class; age $+0.063$/year. Significant (p-value < 0.05): age, sib_sp, pclass, sex; the two interactions are borderline ($p \approx 0.08$–0.10).

HW3 Q6

Prompt. Repeat Question 5, but this time specify a linear discriminant analysis (LDA) model for classification using the "MASS" engine.

Answer. (`lda_fit`, model part):

```
Call:
lda(..y ~ ., data = data)

Prior probabilities of groups:
      Yes        No
0.3836276 0.6163724

Group means:
         age    sib_sp     parch     fare pclass_X2 pclass_X3
Yes 28.74378 0.4393305 0.4267782 48.81424 0.2552301 0.3389121
No  29.64519 0.5807292 0.3463542 22.99299 0.1744792 0.6770833
     sex_male sex_male_x_fare fare_x_age
Yes 0.3472803        12.91229   1568.232
No  0.8489583        19.63018    703.509

Coefficients of linear discriminants:
                      LD1
age            0.0358259965
sib_sp         0.2231961671
parch          0.0876726369
fare           0.0032525962
pclass_X2      0.8724616082
pclass_X3      1.6983328837
sex_male       1.9274805785
sex_male_x_fare 0.0020072010
fare_x_age     -0.0001487374
```

[note: the group-means table was printed 9 columns wide in R; re-wrapped here into two blocks.] Group means show survivors paid more (mean fare 48.8 vs. 23.0), were less often male (0.35 vs. 0.85) and less often 3rd class (0.34 vs. 0.68). LD1: positive weights on sex_male, pclass_X3 point toward "No".

HW3 Q8

Prompt. Repeat Question 5, but this time specify a *k*-nearest neighborsmodel for classification using the "kknn" engine. Choose a value for *k* to try.

Answer. $k = 10$; `knn_fit %>% extract_fit_parsnip()`:

```
Type of response variable: nominal
Minimal misclassification: 0.1990369
Best kernel: optimal
Best k: 10
```

(≈80% leave-one-out accuracy on training.) Note: KNN is distance-based, so predictors should be on comparable scales; the Q4 recipe has no `step_normalize()`, but kknn standardizes predictors internally by default (`scale = TRUE`), so fare does not dominate the Euclidean distance.

HW3 Q9

Prompt. Now you've fit four different models to your training data.

Use `predict()` and `bind_cols()` to generate predictions using each of these 4 models and your **training** data. Then use the metric of **area under the ROC curve** to assess the performance of each of the four models.

Answer. `head(log_train) (predict(..., type = "prob") + bind_cols(survived))`:

```
# A tibble: 6 x 3
  .pred_Yes .pred_No survived
      <dbl>    <dbl> <fct>
1     0.114    0.886 No
2     0.0835   0.916 No
3     0.310    0.690 No
4     0.115    0.885 No
5     0.0219   0.978 No
6     0.755    0.245 No
```

Extra (training accuracy, `predict()` default `type = "class"` → .pred_class, then `accuracy()`):

```
  model    .metric  .estimator .estimate
1 Logistic accuracy binary         0.814
2 LDA      accuracy binary         0.793
3 QDA      accuracy binary         0.778
4 KNN      accuracy binary         0.891
```

HW3 Q10

Prompt. Fit all four models to your **testing** data and report the AUC of each model on the **testing** data. Which model achieved the highest AUC on the **testing** data?

Using your top-performing model, create a confusion matrix and visualize it. Create a plot of its ROC curve.

How did your best model perform? Compare its **training** and **testing** AUC values. If the values differ, why do you think this is so?

```r
test_pred <- function(f) predict(f, titanic_test, type = "prob")
     ↪  %>%
  bind_cols(predict(f, titanic_test)) %>%
  bind_cols(titanic_test %>% select(survived))
log_test <- test_pred(log_fit); lda_test <- test_pred(lda_fit)
qda_test <- test_pred(qda_fit); knn_test <- test_pred(knn_fit)
```

Extra — testing accuracy: Logistic 0.832, LDA 0.806, QDA 0.810, KNN 0.843. *[Figure: ROC curve of LDA on the testing set: x = "1 - specificity", y = "sensitivity", both 0–1, dotted diagonal = random guessing. The step curve rises steeply to sensitivity ≈0.70 at 1-spec ≈0.05 and ≈0.85 at 1-spec ≈0.18, then flattens (≈0.90 from 0.25 to 0.6) and reaches (1, 1); far above the diagonal (AUC 0.881).]*

Prompt (Required for 231 Students) — In a binary classification problem, let p represent the probability of class label 1, which implies that $1 - p$ represents the probability of class label 0. The *logistic function* (also called the "inverse logit") is the cumulative distribution function of the logistic distribution, which maps a real number *z* to the open interval $(0, 1)$. *[Q11–Q12 and answers: see Part I.]*

17 Practice exam (Fall 2026) and Sai's graded midterm (last quarter)

[Parts already covered in Part I are omitted: PE Q1–Q6, Q10–Q12, Q15–Q19 entirely, and the official solutions of every kept PE question (Q7, Q8, Q9, Q13, Q14, Q20); Sai Q1, Q4, Q8a–d entirely; Sai's answers, the Q3 and Q6 figures/tables and the rubric lines restated in Part I; the Gradescope header, most of the cover sheet and the blank scratch pages.]

17.1 Practice exam with official solutions

*[note: the official key prints the correct choice(s) of every multiple-choice question and all solution text in red bold; here the correct choices are in **green bold**; the solution texts are condensed in Part I. The question prompts are identical in the exam and in the key.]*

PE Q7

7. A researcher wants to use KNN to predict whether or not a bank customer will default on a loan. They write the following code:

```
loan_recipe <-
  recipe(defaulted ~ ., data = loans) |>
  step_dummy(all_nominal_predictors(), one_hot = TRUE)

loan_model <-
  nearest_neighbor(neighbors = 5) |>
  set_engine("kknn") |>
  set_mode("classification")

loan_fit <-
  loan_model |>
  fit(defaulted ~ ., data = loans)
```

Their data contain the following variables:

```
application_id
age
income
employment_type
credit_score
default_date
defaulted
```

Identify at least four problems with this analysis. For each problem, explain why it matters and how you would correct it.

PE Q8

8. A scatterplot of X and Y shows a very strong U-shaped relationship. Their Pearson correlation is 0.03. Which statement is best?

a. X and Y are essentially unrelated
b. The scatterplot is likely wrong
c. **There is little linear correlation, despite a strong nonlinear relationship**
d. X will not be a useful variable for predicting Y
e. Both C and D

PE Q9

9. Which EDA finding is evidence of an **interaction**?

a. X1 and X2 have a Pearson correlation of 0.91
b. **The relationship between X1 and Y is positive for one level of X2 and negative for the other**
c. X1 is strongly right-skewed and X2 is strongly left-skewed
d. X1 and Y have a Pearson correlation of 0.80
e. The relationship between X1 and Y is the same across both levels of X2

For Questions 10–12, suppose a university wants to predict whether a student will graduate within four years, using information available **at the end of the student's first year**.

Variable	Description
StudentID	Unique identifier
HS_GPA	High school GPA
FirstYearGPA	GPA at end of first year
Major	Major at end of first year
CreditsAttempted	First-year credits attempted
CreditsCompleted	First-year credits completed
Dorm	First-year residence hall name
GraduationDate	Date of graduation, if graduated
FinalGPA	GPA upon leaving the university
Graduated4Years	Outcome

PE Q13

13. A student wants to predict "salary" using "experience" and "department." They fit the following model:

```
salary_recipe <- recipe(salary ~ experience + department,
                        data = employees) %>%
  step_dummy(department, one_hot = TRUE)

salary_model <- linear_reg() %>%
  set_engine("lm")

salary_workflow <- workflow() %>%
  add_recipe(salary_recipe) %>%
  add_model(salary_model)

salary_fit <- fit(salary_workflow, data = employees)
```

Suppose "department" has three levels, "Sales," "IT," and "HR."
After fitting the model, R reports that the model matrix is rank-deficient and that one or more coefficients cannot be uniquely identified.

a. What caused the problem?
b. Why does "one_hot = TRUE" create this problem for an ordinary linear regression model with an intercept?
c. How should the code be changed?
d. After making this change, how many indicator variables will be created for "department"?

PE Q14

14. A logistic regression model predicts whether a patient has pneumonia based on symptoms and clinical measurements. For one patient, the model produces a predicted probability of 0.78. Which interpretation is correct?

a. The model is 78% accurate for this patient.
b. The patient definitely has pneumonia.
c. **The fitted model assigns this patient a probability of 0.78 of belonging to the pneumonia class.**
d. 78% of the patient's symptoms indicate pneumonia.

PE Q20

20. You have 50,000 observations, two predictors, and clear evidence that the covariance structures differ substantially across levels of your outcome variable. Which would you expect to have the greater potential to capture the true decision boundary?

a. LDA
b. **QDA**
c. Both would do equally well
d. Neither would do well because they are both parametric models

17.2 Sai's midterm (graded, 20/26)

[note: source = Gradescope export `submission_422008762.pdf`. PDF pp. 1–3 = Gradescope rubric summary; pp. 4–8 = the scanned exam booklet (printed questions + Sai's pencil answers); pp. 9–10 = blank scratch pages. Gradescope numbers the graded items "Question 1"–"Question 12", the booklet numbers them 1–9 (Q8 has parts a–d), so both are given ("Gradescope Q12 / Q9"). In each rubric list, ✓ marks the rubric item the grader applied; green = −0 pts, red = deductions, as on Gradescope.]

Booklet cover sheet (PDF p. 4), excerpt

PSTAT 131 Midterm Exam — Summer 2026.

3. **Marking Answers:** You must indicate your answers to **all questions** (multiple choice and free response) on this exam booklet.
4. **Personal Aids:** This is an open-book and open-note exam. Any and all **physical materials** may be used during this exam. **Absolutely no** electronic or digital tools may be used.
8. **Final Answers:** Round answers to 3–4 decimal places when appropriate. Answers must be legible; if multiple answers are given, the first will be graded, except for multiple choice questions; unless otherwise specified, multiple answers marked for Multiple Choice questions will receive a score of zero.

Multiple Choice Questions (booklet pp. 2–3 = PDF pp. 5–6)

Sai Q2 = Gradescope Question 2 — 2 / 2 pts

2. (2 points) Why can KNN represent nonlinear decision boundaries even without explicitly including polynomial or interaction terms?

(A) It assumes predictors are normally distributed
(B) It does not assume equal variances across levels of the outcome
(C) Classification is determined locally by neighboring observations
(D) It estimates a different regression coefficient for every observation
(E) It automatically creates squared terms
(F) Both A and B

Grader / rubric.

✓ **−0 pts** Correct answer is C. KNN does not make any assumptions about normality or equality of variances, it also doesn't create squared terms.
−1 pt Partial credit for an answer of B; students could be remembering that this assumption allows QDA to represent nonlinear decision boundaries
−2 pts A, D, E, or F

Sai Q3 = Gradescope Question 3 — 2 / 2 pts

3. (2 points) Moving along the ROC curve from Point A toward Point C corresponds generally to:
(A) Increasing the classification threshold
(B) Decreasing the classification threshold
(C) Increasing the AUC
(D) Decreasing the AUC

Grader / rubric.
−**1 pt** Partial credit for an answer of A (wrong direction, but still recognizing that the threshold would be changing).
−**2 pts** C or D.

Sai Q5 = Gradescope Question 5 — 2 / 2 pts

5. (2 points) Suppose the population covariance matrices really are identical across all classes. What advantage does QDA gain from estimating a separate covariance matrix for every class?
(A) A closer fit to the population decision boundary
(B) No inherent advantage from the extra covariance flexibility; it may simply increase estimation variance
(C) Perfect classification
(D) QDA, unlike LDA, is nonparametric

Sai Q6 = Gradescope Question 6 — 0 / 2 pts

6. (2 points) Which feature of these results most strongly suggests an interaction?
(A) Treatment B always has the larger fitted value
(B) Increasing dosage changes the fitted value by 10 under Treatment A but by 25 under Treatment B
(C) All four fitted values are different
(D) Increasing dosage increases the fitted value across both treatments
(E) Dosage has two levels
(F) Both B and D

Sai Q7 = Gradescope Question 7 — 0 / 2 pts

7. (2 points) In a logistic regression model, the coefficient for `age` is positive. Increasing one customer's age from 40 to 41 does not change that customer's predicted class. Why is this possible?
(A) A positive coefficient does not affect predicted probabilities
(B) The predicted probability can increase without crossing the classification threshold
(C) Logistic regression coefficients affect only training observations
(D) Age must interact with another predictor before it can affect classification

Free Response Questions (booklet pp. 4–5 = PDF pp. 7–8)

Sai Q9 = Gradescope Question 12 — 3 / 4 pts

9. (4 points) Consider the following recipe. The model's goal is to predict the sale price of a home. `square_feet` represents the house size measured in feet; `square_meters` represents the house size measured in meters:

```
house_recipe <-
  recipe(sale_price ~ ., data = houses) %>%
  step_mutate(
    square_meters = square_feet * 0.092903
  ) %>%
  step_normalize(all_numeric_predictors())
```

A student argues that because both `square_feet` and `square_meters` are normalized, it is appropriate to keep both in an ordinary linear regression. Is this reasoning correct or incorrect? Explain. *[note: on the scan a thin horizontal scanner line runs through "measured in meters".]*

Grader / rubric.
−**2 pts** Logic is partially correct.
−**4 pts** Logic is completely incorrect.
✓ −**1 pt** Correct answer (that the reasoning is wrong) but doesn't specifically mention collinearity

INDEX – page : grid cell

How to read: **12:C4,D7** = distilled notes, page 12, grid cells C4 and D7; 45:B2 = raw transcription, page 45, column B, row 2; pages are separated by ';'. Columns A–H run left→right across the sheet (A–D ≈ left text column, E–H ≈ right text column); rows 1–11 run top→bottom, one inch each. The small grey letters/numbers in the page margins mark the grid.

Symbols & operators

%>% **15:A6,B8,C8,D10**; 28:E1; 34:C4; 38:D11; 39:D7
%in% **15:G6**
`.estimate` **13:C3,C4**; **17:F2**; **20:D3**; 36:C4
`.estimator` **13:C3,C4**; **17:H2**
`.metric` **13:C3**; **17:G2**
`.pred` **13:B2**; **15:H2,B3**; **17:B3**; 30:F5
`.pred_CH` 33:C5,F6
`.pred_class` **4:H11**; **10:D8,C9**; **14:H4**; **15:A1,G4**; **17:B4**; **19:F9**; **20:C3**; 27:C7; 37:G7
`.pred_Down` **17:D5**; 33:C2
`.pred_MM` 33:C5
`.pred_No` **14:E11**; **15:G4**; **17:B4**
`.pred_Up` 33:C2
`.pred_Yes` **10:F2**; **12:D9**; **14:H4**; **15:D1**, **G4**; **17:D4**; **19:E9**; **20:A3**; 27:C5
`.threshold` **14:G8**; 27:C3,G3
<- 28:A5,B5
== 28:G1
|> **15:D6**; 38:D4

A

abalone **5:C10**; **7:D4**; **8:D10**; **15:C9**; **20:E6**; 34:E8
accuracy **2:B6**; **3:E6**; **12:E2**; **13:A10**; **18:F3**; **19:A3**; **20:C2,E3**; 24:A7; 33:C9
`accuracy()` **5:C1**; **12:D2**; **14:H5**; **15:D1**, **H5**; **17:E2**; 37:E7
add_formula() **15:H5**
`add_model()` **4:G10**; **8:G2**; **10:D8**; **15:G5**, **B8,D11**; **19:G9**; 31:C4; 38:H2
`add_recipe()` **4:G10**; **8:G2**; **10:D8**; **15:C8**, **D11**; **18:C9**; **19:F9**; 31:C4; 38:H2
adjusted R-squared **6:G3**
`aes()` **5:D11**; **15:F11**; 27:C8,D9; 32:C11
`age` 35:C1; 36:D8
AIC **9:E6**
`airquality` **5:G8**; 31:E2,G4
`all_nominal_predictors()` **4:G9**; **7:B6**; **8:C11**; **12:D7**; **15:D10**; **16:C7,A10**; **20:G10**; 26:G11; 36:G11; 38:D4
`all_numeric_predictors()` **4:G9**; **15:D10**; **16:E2,C10**; 39:D8
all_outcomes() **16:D10**
`all_predictors()` **8:G1**; **16:G2,A10**
`alpha` 34:A7,C7
AlphaGo 22:F7
`arrange()` **15:E7,H9**; 29:D1,B2; 36:B5
artificial intelligence (AI) 22:G3,H7,A10
assignment operator 28:B5
AUC *see* AUC (area under the ROC curve)
AUC (area under the ROC curve) **2:A6**; **10:C4**; **12:A10**; **13:F9**; **14:A5**; **18:G4**; **19:A5,A6**; **20:A10,E11**; 26:A2,A3,E9; 37:G6; 39:A1,B2
`augment()` **4:G11**; **8:H3**; **10:D8,B10**; **14:G4**; **15:H2,A3,A9**; **16:F4**; **17:A4,E6**; **20:E1,C3,E7**; 27:D6; 33:D7; 35:E9
`autoplot()` **12:C9**; **14:F5**; **15:H5**; **21:B2**

B

backward selection **6:H6**; 25:A3
`bake()` **8:B9**; **16:H3,F4**; 30:C7; 31:E9,G9; 35:B9; 37:D1
bar plot **5:B8**; **20:H5**; 34:B4; 36:G1
Bayes classifier **3:E8**; **10:E3**; **20:A6**; 24:E1, E3
Bayes decision boundary **3:E9**; **10:E10**; 24:E1,G2
Bayes error rate **3:E8,E9**; **20:B7**
Bayes' theorem **10:E2,F4**
best subset selection **6:F5**
$\hat{\beta}$ 24:E10
bias **3:A7**; **6:E1**; **11:G3**; **12:F4**; **19:B7**; **20:G4**; 24:C4,A7,B10,D11
bias-variance decomposition 24:A7,A9,A10
bias-variance tradeoff **1:E9**; **3:A6,A8**; **8:H8**; **11:A10**; **12:G4**; **19:A7**; **20:B5**; 23:E6,F8; 24:A8; 31:F1
Bikeshare 32:C6
`bind_cols()` **5:A2**; **12:D8**; **13:B2**; **15:B3**, **E8**; **17:D5,C6**; **20:G11**; 37:F5
`bind_rows()` **15:E9**
`bins` 29:E8
boxplot **5:B8**; **20:G5**; 25:D11; 27:F1,E6; 29:E4,F5; 32:G3; 34:B7

C

`c()` 28:B3
`cabin` 36:B8
CART 22:G3
categorical predictor **6:E11**; **7:A1**; **18:F8**, **D11**; 25:A4; 32:G7
causation **9:C10**
centering **8:A8**
class imbalance **5:A8**; **6:C5**; **12:A5**; **13:E8**; **20:A2,F9**; 32:G6
class label 24:E3
class overlap 26:B11
classical programming 22:G3
classification **1:F9**; **2:A6**; **8:E9,G10**; **20:G3**, **B5**; 23:G5; 24:A11; 25:A9; 27:D6; 32:A5; 36:A6
clustering 23:G3,G5
code chunk 28:C4
coefficient **6:F8**; **18:H11**; 25:G5; 30:E4; 32:F8; 33:A1; 35:H7; 37:B9; 38:G3; 39:A5
coefficient interpretation **9:A10**
`coin_flip` **14:B9**
collinearity **5:F2,E7**; **7:E10**; **8:A1**; **17:H9**; **18:G1,E7**; **19:F5**; **20:A2,E10**; 29:H4; 32:B10; 39:A7
`color` 34:F8
`colSums()` 36:C9
`conf_mat()` **10:D8**; **14:H5**; **15:A1,B2,H4**; **17:E3**; **19:C2**; **21:D1**; 27:D7
confidence interval **6:G2**
confounding **5:H3,F7**; **9:E8,E10,G10**; **20:A9**; 25:E4; 29:E6
confusion matrix **10:C4**; **13:A8,B8**; **19:A1**; **21:C1**; 25:E9,E10; 33:C7,D8,C9; 37:G8
`coord_obs_pred()` **16:C3**
`cor()` **5:H4,C9,D11**; **16:D3**; 36:G6
`correlate()` **16:D3**; 32:A9
correlation **5:E3**; 25:E8; 29:F3,F4,H6; 32:E2,H5,B7; 34:D3
correlation matrix **5:B9**; **8:C2**; **20:G5**; 29:E2; 32:F2,D6,C9; 34:D10; 36:G6
`corrplot` **15:D5**; 34:B10
`corrplot()` **5:D9,D11**; **16:D3**; **20:F5**; 29:G3; 36:H6
`corrr` **15:D5**; 32:B6,D8
`count()` **5:D8**; **15:E8**; 34:C4; 36:E2
covariance matrix **11:A2**; **18:F4**; **20:B8**; 38:E5,F5; 39:A3,C3
Credit (dataset) 25:A6
cross-validation **2:E11**; **4:F2**; **11:F4**; **12:D4**, **G5**; 29:G10; 31:C2,D7
`cty` 34:C2
curse of dimensionality **4:E6,G6**; **8:F6**; **20:A6**; 31:F2
`cyl` 34:C7

D

data processing 28:A9
data science workflow 23:C2
data splitting **6:A2**; **16:A5**; 27:E9; 29:E10; 35:D8
decision boundary **4:A9**; **10:E9**; **11:C7,A8**; **20:B8**; 24:E2; 26:G2; 38:E5,H5,E9,G9
decision tree 22:D3; 23:G3
deep learning 22:G2,G3,A10
`Default` **8:F10**; **9:H4**; 25:H2
Default data set 25:A10
density function **10:G4**
density plot **5:C8**
descriptive model **2:A8,B8**; **20:G3**
deviance **9:E6**
diagnostic odds ratio 26:C2
`diameter` 34:H11
`diamonds` **3:A3**; **5:H2,A10**; **6:E6**; **12:E7**; **15:B9**; 27:E9; 29:B9,B10; 30:C3
`dim()` **15:E8**
dimension reduction 23:G3,G5; 26:E4,C7, H7
Direction (Smarket) 32:B7
`discrim` **10:D6**; **11:E10**; **15:D5**; **16:H8**; **20:H10**; 32:A5
`discrim_linear()` **11:G10**; **12:C2**; **16:E8**; **20:E10**
`discrim_quad()` **11:G10**; **12:A2**; **16:E9**; **20:F10**
discriminant analysis **8:G10**; **10:E2**; 26:A5
discriminant function **10:F7**; 37:G3
discriminative model **11:G2,C9**
distance between means 26:F6
distance metric **4:A5**; **18:D8**
downsampling 32:G6
`dplyr` **15:C4,E5**; 27:E9
`drop_na()` **5:F9**; **15:E8**
dummy variable **6:E11**; **7:C1**; **9:F6**; **19:E2**; **20:C7**; 25:A4; 26:G11; 30:C8; 32:E7; 36:G9; 38:A10

E

EDA *see* exploratory data analysis (EDA)
`embarked` 36:C8
empirically-driven model **2:G1**; **20:E4**
`engine` **16:E7,F9**; 30:H11
error rate **2:B6**; **3:E5,F6**; **12:F2**; **13:C10**; 24:A11
ESL 22:C3; 24:F5
`estimate` **5:C1**; **8:G3**; **13:C2**; **15:D1,G2**
Euclidean distance **4:A5**; **18:C8**; **19:A9**; 37:F5
`eval` 28:D10
event level **10:A11**; **14:E10**; **20:B10**
`event_level` **17:F4**
`everything()` **15:F6**
exam patterns **18:A4**; **19:A1**
exploratory data analysis 29:D9
exploratory data analysis (EDA) **5:A4,A5**; **15:E10**; 24:E4,E5; 27:E9; 29:A9; 33:E10; 38:A7
`extract_fit_parsnip()` **5:A2**; **6:G7**; **15:H1**; **17:C7**; 30:F2

F

F-statistic **6:G4**
F1 score **13:E2**
`facet_wrap()` **16:A4**
factor level **10:A11**
`factor()` **2:A7**; **12:D5**; **14:C9**; **15:H2,E8**; **20:B10**; 27:C7; 32:G3; 34:C7
false discovery rate (FDR) 25:G11
false negative **13:B9**
false omission rate (FOR) 25:G11
false positive **10:A4**; **13:C9**; **19:A1**
false positive rate (FPR) **13:F10,A11**; **18:F3**; **19:B3**; **20:B9**; 27:F2,C8
`family = "binomial"` 26:C6
`family = binomial` **9:F4,H4**; **17:D10**; 25:G2
`fare` 36:B8
`fashion()` 32:G1
feature **2:C2**; **8:D4**; 22:G4,G10; 29:G2; 30:B4
feature engineering **5:A4,B5**; **8:A4**; **16:A6**; **19:E6**; 24:E4; 38:A9; 39:A7
feature extraction 26:A7
feature selection 26:C7
`fill` 34:G6
`filter()` **15:E6,G9**; 28:G1,A11; 29:B7
`fit()` **4:G10**; **8:H2**; **10:D8**; **14:F4**; **15:G1**, **A8**; **16:F4**; **17:F6**; **18:A9**; **20:G1**; 31:C4; 38:H2,D5
flexibility **2:E4,G5**; **4:A9**; **11:A10**; **12:E3**; **19:A7**; **20:B6**; 23:H7,E8,E10; 24:A2,A3,A9
`flights` 28:G6
formula (R) 30:C4
forward selection **6:E6**; 25:A3
FPR *see* false positive rate (FPR)

G

Gauss, Carl Friedrich 24:F7
Gauss-Markov theorem **8:E8**
gene-expression example 26:A8
generative AI 22:F6
generative model **11:G2,C9**
`geom_abline()` **16:C3**
`geom_bar()` **5:C8**; **16:D2**; **20:E9**
`geom_boxplot()` **5:C8**; **16:C2**; 27:C11; 32:G3; 34:C7
`geom_histogram()` **5:C8**; **16:D1**; 27:C9; 29:E8; 31:G6
`geom_jitter()` **16:C2**; 32:G3; 34:C7
`geom_point()` **5:C9**; **16:D1**; 27:C8
`geom_smooth()` **5:C9**; **16:D2**; 34:G8
`geom_text()` 32:G1
`geom_tile()` 32:G1,B10
`ggplot()` **5:D11**; **15:E11**; 27:D9; 31:G6; 32:G3,C11; 34:E3
`ggplot2` **15:C4,E10**; 29:C9; 32:B10
`ggthemes` **15:D5**; 34:E3
`glimpse()` **15:E8**
`glm` **10:D7,A11**; **15:G4**; **16:H8**; **20:A10,E10**
`glm()` **9:E4,H4**; **17:D10**; 25:H2; 26:C6
`group_by()` **15:E7,G9**; 28:H10; 29:B2; 36:G3

H

`head()` **15:E8**; 27:C5; 29:A10; 30:D6,C7; 35:B2
`height` 34:E11
heteroscedasticity **7:F8**
hierarchical clustering 23:G3
hierarchical principle **7:G3**; **20:D7**
high leverage point **7:E9**
high-dimensional data 23:A1
histogram **5:B8**; **20:E5**; 27:E4,B9; 29:E8; 31:G6; 33:E11
homework **20:E2**; 33:E9
`hwy` **20:E5**; 33:G10; 34:B2,B7
hyperparameter **4:E2,A9**; 31:B2
hyperplane 26:E4
hypothesis test **2:B9**

I

identifier variable **5:B2**; **8:A7**; **17:E7**; **18:B8**, **H8**; **19:E5,F5**; **20:E9**; 38:A3
`if_else()` **15:H7**; 27:C7

`imp_vars()` **6:D1**; **12:B6**,**B10**; **15:F3**; **16:C8**; **19:G7**; **20:H10**; 26:H11; 36:E10
imputation **5:E8**,**F10**; **12:B10**; 36:G9
`impute_with` 26:H11; 36:E10
Income data 23:H7
indicator function **3:E5**
inference **1:F10**; **18:B6**; **20:E5**
inferential model **2:A8**,**B8**; **20:F3**
influential observation **7:E10**
`initial_split()` **6:D3**; **10:D6**; **13:F9**; **15:B7**,**D9**; **16:A5**; **17:F5**; **19:G10**; **20:C4**, **E6**; 30:B2; 31:H8
`install.packages()` 23:C3; 31:F3
interaction term **3:A5**; **5:F7**; **7:A8**,**B10**; **12:C10**; **18:F5**,**B7**; **19:E4**; **20:B7**,**G10**; 25:A5,A6; 26:G11; 36:F9; 38:A7,C7; 39:A4, D4
intercept **6:C9**; **7:B7**; **18:F2**; 30:E4; 32:E9; 38:F3
interpretability **1:E10**; **2:E4**; **11:B10**; **18:D6**; 24:G8
irreducible error **3:E2**,**E3**; **20:D5**,**F8**; 24:B1, A7,A9
`is.na()` 36:C9
`ISLR` 22:B3; 24:F5; 28:B10,C10; 32:D5,C6
`ISLR2` 32:D5,C6

K

k-means clustering 23:G3
k-nearest neighbors *see* k-nearest neighbors (KNN)
K-nearest neighbors *see* k-nearest neighbors (KNN)
k-nearest neighbors *see* k-nearest neighbors (KNN)
k-nearest neighbors (KNN) **1:E9**; **4:A3**; **8:E5**, **F10**; **11:A9**; **12:E8**; **16:E8**; **18:A5**; **19:A9**; **20:A5**; 23:G3,F8; 30:C3,E10,G10; 34:E8; 35:G3; 37:G4,H4; 38:A2,A3,E9,F9
k-NN *see* k-nearest neighbors (KNN)
k-NN *see* k-nearest neighbors (KNN)
`kable()` 30:C7; 31:G4
`kable_styling()` 30:B6,C7
`kableExtra` 30:C6
Kaggle 22:E10
`kbl()` 30:B6
`kknn` **4:E8**,**F9**; **8:G2**; **12:D8**; **15:D5**,**D11**; **20:E7**; 30:E11; 38:D4
`klaR` **10:D6**; **11:F10**,**H11**; **15:D5**; **16:G9**
KNN *see* k-nearest neighbors (KNN)

L

`labs()` **15:E11**; **16:D2**
lasso 23:G3
LD1 26:H7,G8
LD2 26:E8
LDA *see* linear discriminant analysis (LDA)
least squares **6:D10**; 24:E8
`levels()` **15:E8**
`library()` **15:C5**; 28:D10
`library(discrim)` **11:E10**
likelihood ratio 26:A2
linear discriminant (LD1, LD2) 26:F7,E8
linear discriminant analysis (LDA) **8:E10**; **10:E4**,**E6**; **11:A2**; **16:E8**; **18:H4**; **19:E10**; **20:A8**; 26:A5,A6,A8; 32:A5,D5; 33:A9,B11; 36:A6; 37:C9; 38:E5; 39:A3
linear imputation **5:E10**; **20:F10**; 26:H11; 31:E8; 32:C2; 36:G9
linear model 24:E4
linear probability model 25:A9
linear regression **2:E3**; **3:A3**; **5:A4**; **6:A6**; **9:B1**; **16:E8**; **19:E2**; 23:G3,F9; 24:C2,E4, E7,G8; 25:A9; 30:E2,C3; 32:B2,F7; 34:E8; 38:A10; 39:A7,C8
`linear_reg()` **6:E7**; **7:C6**; **8:G2**; **15:D10**; **16:E8**; **20:E7**; 30:E2,E11; 38:H1
`linetype` 34:G8
`lm` **8:G2**; **15:D10**; **16:H8**; 38:H2
`lm()` **17:D10**
log odds **9:C5**,**C11**; **17:C9**; **18:E3**,**G10**; **19:A11**,**B11**; 37:D8; 39:A5
log transformation **5:C10**; **7:G6**
logistic function **9:C4**; **21:B2**; 37:F11
logistic regression **2:A4**; **8:F10**; **9:A3**; **10:E3**; **13:A4**,**B4**; **16:E8**; **18:B11**; **19:A11**; **20:A8**; 23:G3; 25:A8; 32:A5,E7; 33:B4,F6, D11; 36:A6; 37:B6; 38:E4; 39:A5,B5
`logistic_reg()` **10:D7**,**A11**; **15:G4**; **16:E8**; **20:G10**
logit **9:D5**,**C7**; **20:B8**; **21:A2**
`longest_shell` 34:G11

M

machine learning **1:E9**; 22:G3,A8; 23:G5
MAE **2:B6**; **8:H3**; **12:E9**,**H10**; **20:H3**,**D7**; 30:G8; 31:A6; 35:F9
`mae()` **8:G3**; **13:C2**; **15:G2**; **17:E2**; 30:G8
main effect **18:F9**; 25:A5; 39:A4
Manhattan distance **4:A6**
`manufacturer` 34:B4
`MASS` **11:G10**; **16:G9**; **20:G10**
maximum likelihood **9:E3**,**F3**; **11:B11**; **20:B8**; 24:F8; 25:E2,F2
mean imputation **5:F10**; 32:A3
mean squared error (MSE) **2:A6**,**E8**; **12:F2**, **E9**; **20:B5**
`mean()` 28:A2,B3,F10; 31:F7
mechanistic model **2:E1**; **20:F4**
`metric_set()` **5:C2**; **8:G3**; **12:B4**; **13:C2**; **15:G2**,**B9**; **17:E3**; **20:F7**; 30:G8; 31:D4
Minkowski distance **4:A6**
missing at random (MAR) **5:F9**; 24:G6
missing completely at random (MCAR) **5:H8**; 24:F6
missing data **5:B7**,**E8**; **20:E9**; 30:C3; 31:E2, F2; 36:C9
missing not at random (MNAR) **5:E9**; 24:H6
missingness pattern 24:G6
mixed selection **6:G6**
MLE 25:F2
`mode` **16:E7**,**E9**
model comparison **11:A8**; **12:A11**; **18:A4**; 31:B11; 33:E4,F5; 36:D2; 38:A1
model complexity 23:H7
model evaluation **12:A11**; **17:E1**; **18:A4**; 23:G6; 25:A8,E9; 27:E7; 33:A1; 37:G9; 38:A1,E6
`mpg` **5:F2**,**A10**; **15:G9**; **20:E3**; 28:A10; 33:E10
MSE *see* mean squared error (MSE)
multi-class classification **10:G3**
multiclass classification 25:E9; 27:G8; 33:D4
multiclass confusion matrix **13:E4**
multicollinearity **8:B2**
multiple linear regression **6:C8**; 25:A2
multiple logistic regression **9:E8**; 25:H6
multivariate normal distribution **11:D2**
`mutate()` **7:E7**; **8:D10**; **15:H2**,**E7**; **20:F6**; 27:C7; 28:E8; 36:E2

N

`n()` **15:F7**,**G9**; 29:B7; 35:B4
`na.rm` **15:H10**; 31:G8
naive Bayes **8:F10**; **11:E8**; **16:E9**; 23:G3; 32:A5,B5; 33:E3
`naive_Bayes()` **11:G11**; **12:B2**; **16:E9**
`naniar` **5:F8**; **15:D5**; **16:A4**; 31:E2
`nearest_neighbor()` **4:E8**,**G10**; **8:G2**; **12:D8**; **15:D10**; **16:E8**; **19:E9**; **20:F7**; 30:E10,G10; 31:A7,D7; 38:D4
negative predictive value (NPV) **13:F2**
`neighbors` **4:G8**,**G10**; **8:G2**,**G6**; **12:D8**; **15:D10**; **16:G8**; **20:G11**; 31:D7; 38:D4
Netflix Challenge 22:E8
neural network 22:G3
`new_data` 30:C7; 33:D2; 35:B9
non-parametric model **2:E1**,**F2**; **4:F7**; **8:G5**; **11:A8**,**A10**; **20:F4**,**D5**; 38:E9; 39:C4
nonlinear relationship **5:E6**; **7:A8**; **18:A9**; 24:A3; 25:A7; 30:E7; 38:A6,G9
normalization **4:E4**; **8:A8**; **20:B6**; 39:A7,A8
`nycflights13` 28:E6

O

observation 23:B5,B6
odds **9:C4**,**D8**; **21:D2**
odds ratio **9:C8**; **17:B9**; **19:A11**; **20:C8**; **21:A2**; 37:B9
`OJ` **10:B6**; **12:B2**; **13:G9**; **14:G11**; **15:G2**; 32:C7,B8,E10; 33:F6
one-hot encoding **7:B6**; **16:C7**; **20:C7**; 38:A10
`one_hot` **5:A3**; **7:A5**,**A6**; **16:B7**; **18:C9**; 38:H1,D4
`opts_chunk$set()` 28:B6
orthogonal polynomial contrast 30:C8
outcome **2:B2**; **5:E9**; **18:B10**; **19:G5**; 29:F1, F8; 30:D4; 32:D7
outlier **5:C10**; **7:E9**; **20:E6**; 34:A2
`outlier.shape` 34:C7
overfitting **2:F3**,**F6**,**E9**; **6:G3**; **11:E8**; **12:E4**, **D10**; **18:A6**; **19:C9**; **20:F4**,**G7**; 23:H7,E8; 24:A5,A7; 25:C8; 31:F1,A10; 38:A2

P

p-value **2:B9**; **6:E5**; **9:H5**; 25:G5; 30:E4; 32:E9; 33:A1; 35:H7; 37:A9
parametric model **2:E1**; **6:C8**; **8:E5**; **11:A8**, **C10**; **20:H4**
`parch` 36:A8
`parsnip` **4:E8**; **15:B4**; **16:E7**; 23:A3; 30:F2; 31:D2
`passenger_id` 36:B8
`pclass_X2` 37:C6
`pclass_X3` 37:C6
Pearson correlation **3:B5**; **5:E3**; **18:C7**; **20:A2**,**H10**; 34:F2; 36:F7; 38:A6
percent stacked bar chart **20:E9**; 36:G1
perfect separation **9:E7**; 26:A6
`pivot_wider()` 36:B5
plane 26:F4
`poissonreg` 32:C5,C6
polynomial contrasts **6:G10**
polynomial regression **7:E5**; 25:A5
pooled variance **10:G8**
`position = "fill"` **16:D2**
posterior probability **10:F5**; **20:F2**; 27:B4; 33:B4; 38:E4
practice exam **18:A5**
precision **2:C6**; **13:E2**; **19:D3**; **20:B9**
`precision()` **17:F3**
`predict()` **4:H11**; **5:D1**; **8:H2**,**C9**; **10:D8**, **A9**; **13:B2**; **15:C3**,**A9**; **16:E4**; **17:D5**,**E6**; **19:F9**; **20:E1**,**A3**,**E7**,**E11**; 30:F5; 33:D2,D5; 37:E5
predicted probability **9:E11**; **10:A1**,**B3**
predicted vs. actual plot 30:H6
prediction **1:E9**; **17:A3**; **20:F5**; 22:D5,G8; 23:H2; 30:E5
predictive model **2:A8**,**B9**; **20:F3**
predictor **2:B2**; 23:A9; 24:E3; 29:H4; 30:C6; 31:G1; 38:A9
`prep()` **6:B6**; **8:A9**; **16:G3**,**E4**,**B7**; 30:C7; 31:E9,G9; 35:B9; 37:D1
principal component analysis (PCA) 23:G3
prior probability **10:E4**; 37:G3
projection 26:F6,H7,A8,E8
`prop` **6:A3**,**D3**; **13:F9**; **15:D9**; **16:A5**; **20:H6**; 31:H8
Purchase (OJ) 32:C8

Q

QDA *see* quadratic discriminant analysis (QDA)
quadratic discriminant analysis (QDA) **8:E10**; **11:A5**,**C5**; **16:E9**; **18:H4**; **19:E10**; **20:C8**; 26:A5; 32:A5; 33:E1; 36:A6; 38:E5,F11; 39:A3,C3
qualitative response **8:E10**
`quantile()` 35:C5

R

R code **15:A3**
R Markdown 28:A3
R output **20:A3**
R project 28:B7
R-squared **2:B6**; **6:E3**,**F10**; **12:E9**,**E10**; **20:H3**,**A7**; 30:G8; 31:A6; 35:F9
random seed **6:C4**
rank deficiency **7:C6**; **18:F1**; 38:H3,A10
`read_csv()` **8:D10**; **12:D5**; **15:G2**
`readr` **15:D4**
recall *see* true positive rate (TPR, sensitivity, recall)
`recipe()` **4:G9**; **7:D4**; **15:B8**,**D10**; 26:H10; 30:A4,C4; 32:E7,E8; 36:E10; 38:D4,D11; 39:D7
`recipes` **8:A4**; **15:D4**; **16:A6**; **19:E6**; 23:A3; 25:B4; 38:A3
reducible error **2:B9**; **3:E2**,**F2**; **20:H7**
reference level **7:A2**,**A5**; **9:G6**; **16:B7**; **18:H2**; **19:F3**; 37:A6
regression **2:A5**; **12:E8**; **20:F3**,**D5**; 23:G5, B10
regression line 22:B5
regression plane 25:A2
regularization 23:G3
reinforcement learning 23:G5
`reorder()` 29:G7
reproducibility **6:C4**; **16:D5**; **17:E5**; **20:F1**
resampling 29:E10; 31:C2
residual **6:C9**; 23:G2; 24:F9; 25:A2
residual plot **7:E7**,**F8**
residual standard error (RSE) **6:G3**; **12:E10**
residual sum of squares (RSS) **6:D10**
ridge regression 23:G3
`rings` 34:E11; 35:D8
`rmarkdown` 23:A4
RMSE *see* root mean squared error (RMSE)
`rmse()` **8:G3**; **13:C2**; **15:G2**; **17:E2**
ROC AUC *see* AUC (area under the ROC curve)
ROC curve **10:B4**; **13:E10**; **17:G3**; **18:F4**,**E7**; **19:A5**; **20:A10**; 26:A2,A3,E9; 27:A1,B1,E2, F6,E7,G7; 33:F5; 37:G6; 39:A1,B1
`roc_auc()` **10:G2**; **12:D9**; **14:E6**,**E10**; **15:B1**,**G5**; **17:E3**; **19:E9**; **20:H11**
`roc_curve()` **12:B4**,**C9**; **14:G6**,**E10**,**E11**; **15:H5**; **17:E3**; **21:A2**; 27:B4; 33:F6
root mean squared error (RMSE) **2:B6**; **6:E10**; **12:E9**,**E10**; **20:G3**,**D7**; 31:A6; 35:F9
`rplot()` **16:D3**; 32:B9
`rsample` **15:C4**; **16:A5**; 23:A3
`rsq()` **8:G3**,**E4**; **12:E10**; **13:C2**; **15:G2**; **17:E2**; 30:G8
RStudio 27:E11

S

Sai midterm **18:E5**
sampling variability 25:C1
`scale_fill_manual()` 27:D9
scaling **8:C8**
scatter 26:F6
scatterplot **5:B9**; **20:F5**; 22:A4; 25:F8,C10; 26:E2; 30:F7; 34:B2; 38:A6
`scroll_box()` 30:C7,C8
`se = FALSE` 34:G8
`select()` **5:D11**; **15:E6**; 27:C5; 28:E7,H8; 30:F5; 36:G6
self-driving car 22:H7
sensitivity *see* true positive rate (TPR, sensitivity, recall)
`sensitivity()` **14:G5**,**F10**; **17:E2**
`seq()` 28:B5
`set.seed()` **6:D3**,**A4**; **15:B7**,**D9**; **16:A5**; **17:E5**; **19:F10**; **20:E1**
`set_args()` **11:G11**; **16:G10**

`set_engine()` **4:F9**,**G10**; **6:E7**; **8:G2**; **10:D7**; **11:G10**; **15:C8**,**D10**; 31:D2; 38:H2, D4
`set_mode()` **2:D8**; **4:F9**,**G10**; **8:G2**; **10:D7**; **15:C8**,**D11**; **19:F9**; 38:D4
`sex` 36:D8
`sex_male` 37:D6
`shell_weight` 34:G11
`Shiny` 23:A4
`shucked_weight` 34:F11
`sib_sp` 36:A8
sigmoid **9:C5**; 25:C10
simple linear regression **6:C7**; 24:E8
simple logistic regression 25:G5
Simpson's paradox **9:G10**; 25:E7,E8
skewness **5:D10**; **20:E5**; 31:F7
slope **6:B9**
`Smarket` **10:B6**; **12:B2**; **13:E6**,**E9**; **14:G10**; **15:G2**; 32:C6,B8,G8
smoothing spline 23:E9
specificity *see* specificity (TNR)
specificity (TNR) **10:A4**; **13:F9**,**A11**; **18:E6**; **19:D3**,**B4**; **20:D9**; **21:D1**; 27:E7,C8; 33:F7
`specificity()` **14:G5**,**G10**; **17:E3**
standard error **6:F1**; 25:G5; 30:E4; 32:E9; 35:H7
standardization **4:E4**; **8:B8**; **18:C6**; 37:G5
`starts_with()` **7:H4**,**E5**; **12:D7**,**A10**; **15:H4**,**G6**; **16:B10**; **20:G10**; 26:G11; 36:H11
statistical learning 22:A8
statistical machine learning **1:E9**,**F10**
`step_center()` **8:G1**,**B8**; **16:C8**; **19:G7**; **20:G6**
`step_dummy()` **4:E8**,**G9**; **5:B3**; **6:F6**,**F10**; **7:B4**,**G5**,**A6**; **12:D7**,**D9**; **15:D10**; **16:F1**,**A7**; **18:F1**; **19:F7**,**E10**; **20:F6**,**G10**; 25:A5; 26:G11; 32:E7; 36:G9,G11; 37:B6; 38:H1, D4
`step_impute_bag()` **5:F10**; **16:A9**
`step_impute_knn()` **5:G10**; **16:A9**
`step_impute_linear()` **5:E10**; **6:D1**; **12:B6**; **15:F3**; **16:A8**; **19:F7**; **20:E10**; 26:H11; 36:E9,B10,F10
`step_impute_mean()` **5:E10**; **16:A8**; **19:F7**; 32:A3
`step_impute_mode()` **5:G10**; **16:A8**
`step_interact()` **7:E4**,**H4**; **12:D7**; **15:H4**; **16:A8**; **19:F7**,**G10**; **20:G6**,**G10**; 26:G11; 36:H11
`step_log()` **7:G6**; **16:A9**; **19:G8**
`step_mutate()` **16:B9**; 39:D7
`step_normalize()` **4:G5**,**G9**; **5:A3**; **8:F2**, **A8**; **15:D10**; **16:A8**; **17:E7**; **18:C9**,**G10**; **19:F7**,**G9**; **20:E2**; 37:G5; 39:D8
`step_poly()` **7:E6**; **16:A9**; **19:F8**
step_rm() **16:A9**
`step_scale()` **8:F2**,**D8**; **16:B8**; **19:F7**; **20:E6**
stepwise selection 25:A3
`strata` **6:D3**,**A4**; **10:D6**; **12:D5**; **13:F9**; **14:B10**; **15:D9**; **16:A5**; **19:E10**; **20:A2**,**H6**, **G8**; 30:B2; 31:H9
stratified sampling **6:B4**; **12:B5**; **13:E9**; **16:B5**; **20:H6**,**H9**,**A11**; 30:B2; 35:D7; 36:D9
`stretch()` 32:C11
structural collinearity **8:B3**
studentized residual **7:G9**
`summarise()` **15:E7**,**G9**; 28:E9,E10,F10; 35:B4; 36:H4
`summary()` **9:H4**; **15:E8**; **17:D10**
supervised learning **2:A3**; **20:E3**,**A5**; 23:G3, G5
SVM 22:G3

T

t-statistic **6:E5**; **19:E3**; 30:E4; 35:H7
`table()` **15:E9**
target leakage **5:C2**; **8:A5**; **14:C7**; **17:E5**,**H7**; **18:D8**; **19:E5**,**H5**,**B6**; **20:H6**,**B10**; 35:F1; 38:A3
test MSE **2:E8**,**G9**; 23:E8,E10; 24:A2,A5,A9
testing data **11:G7**; **20:B6**; **21:C1**; 29:F10; 33:E5; 37:G9
testing error **2:E7**; **3:E5**; **12:E1**,**F2**; **18:B6**; **19:A8**; 30:E10; 31:A10
`testing()` **6:C3**; **15:B7**,**D10**; **16:A6**; 31:H9
`theme_bw()` 27:C9
`theme_gdocs()` 34:F4
threshold **9:E11**; **10:B3**; **13:A4**; **18:F2**,**G6**; **19:A3**; **20:A9**; 27:F2,F3,D4,G5,B6,F7; 39:A1,B2,A5
`tibble` **15:D4**
`tibble()` **17:C6**
tidy data 23:C2
`tidy()` **6:G7**; **10:D8**,**D10**; **15:H1**,**A4**; **17:A7**; **19:E3**; 30:F2; 32:F8,F10
`tidymodels` **4:E8**; **10:D6**; **15:A3**,**B4**,**C5**,**A7**; 23:A2,C3,A4; 25:A4; 30:C3; 32:B5
`tidymodels_prefer()` **15:A5**,**D5**
`tidyverse` **15:A3**,**A4**,**C5**; 23:B2,A4; 27:E9
tie-breaking **4:E4**
`titanic` **12:A4**,**A5**; **14:A9**; **15:F2**; **20:E8**; 26:E9; 36:A6
TPR *see* true positive rate (TPR, sensitivity, recall)
training data 22:G4; 24:H10; 29:F10; 33:D9
training error **2:E7**; **3:E5**; **11:E7**; **12:E1**,**F2**, **B4**; **18:D6**; **19:A8**; 30:E10; 31:A10
training MSE **2:E8**,**E9**; 23:E8,E10; 24:A2,A5
training set 22:F4
`training()` **6:C3**; **15:A7**,**D10**; **16:A6**; 31:H9
training/testing split **6:A2**; 29:E10; 35:D8; 36:D9
transformation **7:G6**
`transmute()` **15:E7**
true negative **13:D9**
true positive **10:C3**; **13:A9**; **19:A1**
true positive rate (TPR, sensitivity, recall) **10:D4**; **13:E9**,**A10**,**B10**,**C10**,**E10**; **18:H3**,**F6**; **19:C3**,**A4**; **20:F3**,**A9**,**B9**; **21:C1**; 27:E2,E7,A8; 33:E7
`truth` **5:C1**; **8:G3**; **13:C2**; **15:B1**,**G2**; **17:F2**; 31:D4
tuning parameter **4:F2**,**A9**; **8:F6**; 31:B2,D2
`type` 34:G11
`type = "prob"` **4:G11**; **10:D8**,**A9**; **12:D8**; **14:F4**; **17:B4**,**D5**; 33:D2,D5
Type I error **13:A9**
Type II error **13:D9**

U

underfitting **2:E10**; **12:E4**; **19:B9**; 24:A5,C6
`ungroup()` **15:F7**
unsupervised learning **2:A3**,**C3**; **20:G3**,**B5**; 23:G3,G5
upsampling 32:F6
`use = "pairwise.complete.obs"` 36:G6
`usekernel` **11:E9**,**G11**

V

validation set 24:C6
variance **3:A7**; **11:E3**; **18:B6**,**G6**; **19:C7**; **20:H4**; 24:A7,B10,D11; 39:A3,B3
variance inflation factor (VIF) **8:D2**
`vis_miss()` **5:F8**; **6:D2**; **16:B4**; 31:G4
`viscera_weight` 34:G11

W

`where()` **15:G6**
`whole_weight` 34:E11
`workflow()` **4:G10**; **5:B3**; **8:G2**; **10:D8**; **15:B8**,**D11**; **18:C9**; **19:E8**,**F9**; **20:F1**,**F7**; 31:C3; 38:H2
`workflows` **15:C4**,**A7**

X

`xlab()` 27:D9; 32:G3

Y

`yardstick` **12:E11**; **14:E4**; **15:C4**,**B9**; **17:E1**; **20:A10**; 23:A3; 35:E9

Z

z-statistic **9:F5**; 25:G5